

Scientific Workflows with DAGonStar

Practice and Experience

Orchestrating AI-HPC Workflows for Science and Engineering

Version 2.2 — July 17, 2026

Dante Domizzi Sánchez-Gallegos Ciro Giuseppe De Vita
Raffaele Montella

High-Performance Scientific Computing Laboratory
Department of Science and Technology
University of Naples “Parthenope”

Contributors:

Juan Armando Barrón Lugo, Genaro Domizzi Sánchez-Gallegos, Catherine Alessandra Torres Charles

Compiled from the DAGonStar project documentation (main branch)

July 17, 2026

Document Information

Scientific Workflows with DAGonStar

Practice and Experience

Orchestrating AI-HPC Workflows for Science and Engineering

Version	2.2
Date	July 2026
Repository branch	main
Repository	https://github.com/DagOnStar/dagonstar

Authors

Dante Domizzi Sánchez-Gallegos

High-Performance Scientific Computing Laboratory, Department of Science and Technology,
University of Naples “Parthenope”

`dante.sanchezgallegos@uniparthenope.it`

Ciro Giuseppe De Vita

High-Performance Scientific Computing Laboratory, Department of Science and Technology,
University of Naples “Parthenope”

`cirogiuseppe.devita@uniparthenope.it`

Raffaele Montella

High-Performance Scientific Computing Laboratory, Department of Science and Technology,
University of Naples “Parthenope”

`raffaele.montella@uniparthenope.it`

Contributors

Juan Armando Barrón Lugo

Centro Nacional de Investigación y Desarrollo Tecnológico (CENIDET), Mexico

Genaro Domizzi Sánchez-Gallegos

Benemérita Universidad Autónoma de Puebla (BUAP), Mexico

Catherine Alessandra Torres Charles

High-Performance Scientific Computing Laboratory, Department of Science and Technology,
University of Naples “Parthenope”

Version History

Version	Date	Changes
1.0	June 2025	Initial release. Fourteen-lesson tutorial, architecture, environmental case study, AI-HPC integration (LLM, Web, Native). Branch development-native-web .
2.0	January 2026	Extended narrative, deeper theory, richer exercises, appendices.
2.1	June 2026	Branch master . Chapter 18 (Jupyter/Colab). Appendices E and F. PyPI installation path. Full author attribution.
2.2	July 2026	Branch main . Tutorial restructured to 19 lessons (L00–L18), five-module curriculum, assessment model. Five new task types: FAAS, IOT, KUBERNETES, APPTAINER, NOMAD. New Workflow capabilities: FAIR by design (enable_fair), CWL v1.2 export (saveAsCWL), portable emulation. Chapters 19–23 and Appendix G. Part IV added. Architecture, reference guide, and developer guide updated.

Licence and Reuse

This textbook is compiled from the official DAGonStar project documentation (<https://github.com/DagOnStar/dagonstar>) and from original pedagogical material contributed by the authors. Readers are encouraged to use, adapt, and redistribute this material for academic and educational purposes with appropriate attribution.

Abstract

This textbook presents *DAGonStar* (Direct Acyclic Graph On anything, also written *DAGon**), a lightweight Python workflow engine designed to orchestrate computational tasks across local machines, on-premise high-performance computing (HPC) clusters, container runtimes, cloud virtual infrastructures, and — in its current evolution — artificial-intelligence (AI) and web services. *Scientific Workflows with DAGonStar: Practice and Experience* is a self-contained academic treatment of workflow-based scientific and engineering computing, structured as a progressive course from foundations to advanced capabilities and grounded exclusively in the documentation and source examples found in the official `DagOnStar/dagonstar` repository.

The book is organized in four parts. **Part I** establishes the theoretical foundations: directed acyclic graphs (DAGs), task- and data-oriented dependency models, the `workflow://` schema, configuration management, scratch directories, generated launchers, cycle detection, checkpointing, and data staging. **Part II** extends the model to heterogeneous execution backends: containers, remote and Slurm-scheduled HPC execution, meta-workflows, asynchronous orchestration, and first-class *AI-HPC* integration through Large Language Model (LLM) tasks, Web tasks, and Native Python tasks. **Part III** applies the accumulated framework to computational science and engineering applications, presenting the environmental forecasting case study as a representative end-to-end pipeline, cataloguing the examples library, synthesizing the operational architecture, and introducing the interactive development environment through Jupyter Notebook and Google Colab. **Part IV** covers advanced capabilities: FAIR by design (`enable_fair`), Common Workflow Language v1.2 export (`saveAsCWL`), portable emulation, Function-as-a-Service tasks, IoT and compute-continuum tasks, and three additional infrastructure backends (Kubernetes, Apptainer, Nomad). Seven appendices collect the complete API reference, developer practices, configuration model, checkpoint semantics, Google Colab reference, GitHub contribution guide, and external service provider configuration.

Every code listing, configuration fragment, and lesson narrative reproduced in this book is drawn directly from the *DAGonStar* documentation and tutorial sources; the book adds pedagogical structure, formal definitions, discussion, and original explanatory figures, while remaining descriptive of, and consistent with, the current implementation. Version 2.2 reflects the `main` branch of the repository.

Note on scope. Although the motivating applications throughout this book are drawn from computational environmental science — numerical weather prediction, ocean modelling, particle tracking — *DAGonStar*’s mechanisms are entirely domain-neutral. The dependency model, execution backends, FAIR recorder, CWL exporter, and portable emulation apply identically to bioinformatics pipelines, materials science simulations, financial data workflows, signal processing chains, and any other computational science or engineering domain where heterogeneous tasks must be orchestrated reproducibly. This edition adopts the broadened title *Scientific Workflows with DAGonStar: Practice and Experience* to reflect that scope.

Contents

List of Figures

Listings

Preface

Scientific and engineering computing increasingly depends on the orchestration of heterogeneous resources: numerical models compiled for high-performance computing (HPC) clusters, data preprocessing and post-processing scripts, container images that freeze a fragile software stack, cloud instances provisioned for a single analysis run, and — more recently — artificial intelligence (AI) services consumed through web APIs for summarization, anomaly detection, and decision support. The challenge addressed in this textbook is not how to write any one of these components, but how to make their *composition* explicit, reproducible, portable, and auditable — regardless of the scientific or engineering domain in which they are deployed.

DAGonStar (also written DAGon*) answers this challenge with a deliberately small abstraction: a workflow is a directed acyclic graph (DAG) of tasks, and a task is a Python object that executes a command — local, remote, containerized, scheduler-submitted, an HTTP request, an LLM inference call, or an importable Python function — in a particular environment. This minimalism is the engine’s central design virtue. It keeps workflow authoring close to ordinary Python while still providing the scientific guarantees that ad hoc shell scripts cannot: explicit dependency graphs, automatic data staging, cycle detection, checkpointing, and a uniform mechanism — the `workflow://` schema — for expressing that one task consumes a file produced by another.

About this textbook and its authors. *Scientific Workflows with DAGonStar: Practice and Experience* grew from tutorial and documentation material developed by the DAGonStar team at the High-Performance Scientific Computing Laboratory, Department of Science and Technology, University of Naples “Parthenope.” The three principal authors are Dante Domizzi Sánchez-Gallegos, Ciro Giuseppe De Vita, and Raffaele Montella, with contributions from Juan Armando Barrón Lugo, Genaro Domizzi Sánchez-Gallegos, and Catherine Alessandra Torres Charles. Each chapter and appendix has been reviewed against the repository’s own implementation and documentation to ensure fidelity to the actual, deployed system. This edition (Version 2.2) reflects the `main` branch of the repository, which introduces five new task types (FAAS, IOT, KUBERNETES, APPTAINER, NOMAD), the FAIR by design recorder, CWL v1.2 export, and portable emulation.

Scope and intended audience. The title *Scientific Workflows with DAGonStar: Practice and Experience* reflects a deliberate broadening. Earlier editions were organized around the workshop title “*Orchestrating AI-HPC Workflows for Computational Environmental Science*” and used meteorological applications exclusively as motivating examples. This edition retains those examples where they illuminate the mechanisms most clearly, but adopts a domain-neutral framing: every workflow mechanism described here — dependency resolution, data staging, checkpointing, FAIR recording, CWL export, FaaS invocation, IoT integration — applies with equal fidelity to bioinformatics, materials science, structural engineering, signal processing, and any other computational discipline that composes heterogeneous tasks. The subtitle *Practice and Experience* signals the progression: structured lessons build practical

competence; the synthesis chapters, case study, and exercises build the experience needed to design and deploy new workflows independently.

This book is written for graduate students, researchers, and research software engineers in any scientific or engineering discipline who need to compose heterogeneous computational pipelines reproducibly. Working knowledge of Python and of basic Unix/HPC concepts (shells, SSH, job schedulers) is assumed; no prior exposure to workflow engines is required.

How this book is organized. Part I (Chapters 1–8) builds the conceptual and mechanical foundations: workflow theory, the DAGonStar architecture, installation, the `workflow://` schema, configuration, and Lessons 1–7 from the official tutorial (local workflows through data staging). Part II (Chapters 9–14) covers heterogeneous execution backends and the AI–HPC integration layer: Lessons 8–14 (Docker, remote/Slurm execution, meta-workflows, asynchronous launch, LLM, Web, and Native tasks), plus a chapter on wrapping external scientific software. Part III (Chapters 15–18) applies the accumulated framework to an end-to-end case study, surveys the examples catalog, synthesizes the operational architecture, and introduces the interactive deployment layer (Jupyter Notebook and Google Colab). Part IV (Chapters 19–23) covers FAIR by design, CWL export and portable emulation, FaaS tasks, IoT and compute-continuum tasks, and extended infrastructure backends. Appendices A–G collect reference material.

A note on fidelity to source material. Every code listing in this book is reproduced, without semantic alteration, from the DAGonStar documentation and tutorial sources on the main branch of <https://github.com/DagOnStar/dagonstar>. Where the documentation explicitly flags a limitation, an open technical-debt item, or a constraint of the current implementation, this book preserves that caveat rather than smoothing it away, in keeping with the project’s own stated principle: *“Documentation must remain consistent with the software in this repository.”* Readers who detect a discrepancy between a code listing in this book and the repository’s current source are encouraged to open a GitHub issue, as described in Appendix F.

Part I

Foundations of Scientific Workflow Orchestration

Chapter 1

Introduction to Scientific Workflows

If these steps remain implicit in shell history, personal notebooks, or site-specific scripts, the experiment is difficult to audit. Workflow systems make the structure explicit.

DAGonStar documentation, Introduction to Scientific Workflows

1.1 Why Workflows Matter in Computational Science

Modern computational science rarely consists of a single program executed once. A realistic environmental-science experiment, for instance, combines many heterogeneous activities:

- preprocessing observational data;
- running models or simulations;
- staging intermediate files between machines;
- executing tools on HPC clusters or cloud instances;
- post-processing model products;
- publishing scientific outputs;
- preserving sufficient context to reproduce a result.

When these steps are encoded only in shell history, ad hoc notebooks, or site-specific scripts passed from one student to the next, the resulting experiment is opaque: it cannot easily be audited, repeated by a different researcher, or ported to a different machine. A *workflow system* exists precisely to make this implicit structure explicit, executable, and inspectable.

DAGonStar approaches the problem with a deliberately small abstraction: a **workflow** is a **directed acyclic graph (DAG) of tasks**, and each **task** is a Python object that executes a command in a particular environment. The remainder of this chapter develops this abstraction formally and explains the scientific benefits it is designed to deliver.

1.1.1 The Scientific Benefits of Explicit Workflows

The DAGonStar documentation identifies five principal benefits that follow from making computational structure explicit:

Reproducibility

the sequence of computations is encoded as source code, not as institutional memory.

Traceability

inputs, outputs, and task dependencies are visible as first-class graph structure rather than buried in script side effects.

Portability

the same workflow structure can target different execution backends — a laptop today, an HPC cluster tomorrow — without rewriting the scientific logic.

Parallelism

independent tasks can run concurrently when resources permit, because the dependency graph makes independence explicit.

Recoverability

checkpointing can avoid recomputing finished work after a partial failure, which matters enormously for expensive simulations.

Scientific Practice Note

Even apparently small automation gains compound at scale. A workflow that takes ten extra minutes to encode explicitly, instead of being run by hand from a terminal, pays for itself the first time it must be rerun — with a different parameter, on a different machine, or six months after the original researcher has left the project.

1.2 Directed Acyclic Graphs

Definition: Directed Acyclic Graph (DAG)

A *directed acyclic graph* $G = (V, E)$ consists of a finite set of **nodes** V and a set of directed **edges** $E \subseteq V \times V$, such that there is no sequence of edges $v_1 \rightarrow v_2 \rightarrow \cdots \rightarrow v_k \rightarrow v_1$ that returns to its starting node. In scientific workflow terminology:

- a **node** is a computational *task*;
- an **edge** $(u, v) \in E$ means that task v *depends on* task u , i.e. u must complete before v may begin;
- **acyclic** means the graph contains no circular dependency.

The acyclicity constraint is not a mathematical nicety: it is the precise condition under which a valid execution order (a *topological sort*) exists. If a cycle were permitted, no task in the cycle could be scheduled first without violating some dependency in the cycle, and the workflow would be unrealizable.

?? illustrates the canonical example used throughout the DAGonStar project, expressed as four BATCH tasks: **Tokio** writes a file consumed implicitly by **Mosco**, while **Berlin** and **Nairobi** are independent siblings that both depend only on **Tokio**. The graph immediately

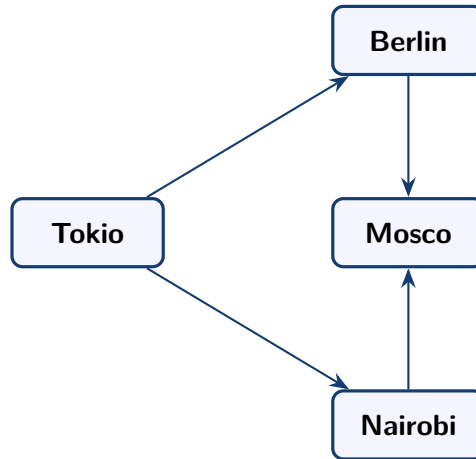


Figure 1.1: A four-task DAG corresponding to the engine’s canonical “Batch Task Flow” example. **Tokio** has no predecessors and runs first; **Berlin** and **Nairobi** depend only on **Tokio** and may run concurrently; **Mosco** depends on both and runs last.

exposes the available parallelism: a scheduler is free to run **Berlin** and **Nairobi** concurrently once **Tokio** finishes.

DAGonStar validates the workflow graph before execution, either implicitly when dependencies are constructed through `Workflow.make_dependencies()`, or explicitly via `Workflow.Validate_WF()`. A cycle is treated as a *modeling error*: the engine refuses to run a workflow whose dependency graph is not a true DAG.

1.3 Task-Oriented and Data-Oriented Workflows

DAGonStar supports two complementary ways of describing a workflow, and the distinction between them is the single most important conceptual choice a workflow author makes.

1.3.1 Task-Oriented Workflows

In a **task-oriented workflow**, dependencies are stated explicitly by the author, independent of any file the tasks happen to exchange:

```
task_b.add_dependency_to(task_a)
```

Listing 1.1: Explicit dependency declaration.

This statement reads: “**task_b** must wait for **task_a**.” Explicit dependencies are appropriate when the relationship between two tasks is *conceptual or procedural* rather than a literal file consumption — for example, “validate parameters before launching the model.”

1.3.2 Data-Oriented Workflows

In a **data-oriented workflow**, dependencies are *inferred* from the command text using the `workflow://` schema (developed fully in ??):

```
task_b = DagonTask(TaskType.BATCH, "B", "cat workflow:///A/output/result.txt")
```

Listing 1.2: Implicit, data-driven dependency.

When `Workflow.make_dependencies()` is called, DAGonStar scans every task’s command string, finds the reference to task *A*, records the dependency $A \rightarrow B$, and later stages the referenced file into *B*’s working directory before *B* executes.

Scientific Practice Note

The data-oriented style is especially natural for scientific pipelines, because dependency edges then follow the *data products* being consumed — which is, after all, how a scientist already thinks about a processing chain: “the post-processing step needs the simulation’s NetCDF output,” not “task three must follow task two for an unstated reason.”

Both styles may, and routinely do, coexist in the same workflow: explicit edges encode procedural ordering, while `workflow://` edges encode genuine data lineage. ?? develops both patterns with complete, runnable examples.

1.4 DAGonStar’s Computational Model

The current implementation centers on a small number of cooperating abstractions, summarized here and developed individually across the remainder of Part I and Part II:

Workflow

the container and scheduler for a set of tasks; owns dependency construction, execution, checkpoint state, and configuration.

DagonTask

a *factory* that constructs a concrete task implementation from a `TaskType` value.

Task

the base class providing threaded task execution, dependency bookkeeping, and `workflow://` resolution.

Batch

a local shell task — the simplest concrete execution backend.

Slurm

a task submitted through `sbatch` to an HPC job scheduler.

RemoteTask

a task executed on a remote host through SSH.

DockerTask

a task executed inside a Docker container.

CloudTask

a task bound to an Apache Libcloud-managed cloud instance.

Stager

the component responsible for generating or invoking the data-movement operation between two dependent tasks.

Operationally, the workflow engine runs tasks as Python `threading.Thread` objects. Each task thread waits for its predecessors, prepares its scratch directory, resolves data references, executes its generated launcher script, updates its status, and finally releases the tasks that depend on it. This execution model is examined in full mechanical detail in ??.

1.5 The Role of Scratch Directories

Scientific workflows frequently produce many intermediate files. DAGonStar therefore gives *every* task its own working directory, called a **scratch directory**. The base path for all scratch directories is set once, in configuration:

```
1 [batch]
2 scratch_dir_base=/tmp/
```

Listing 1.3: Minimal scratch directory configuration.

If a task is not given an explicit `working_dir`, DAGonStar constructs one automatically from the workflow’s scratch base and a timestamped task name (for example, `/tmp/1710000000000-prepare`). Every task additionally receives an internal `.dagon/` directory containing its generated launcher script and execution metadata — a mechanism examined closely in ??.

1.6 Execution Backends at a Glance

DAGonStar’s public task types are enumerated in `dagon.task.TaskType`:

TaskType	Execution environment
BATCH	Local (or remote, via SSH keyword arguments) shell command
SLURM	HPC job-scheduler submission via <code>sbatch</code>
CLOUD	Apache Libcloud-managed virtual machine
DOCKER	Containerized execution via the Docker SDK
LLM	OpenAI-compatible Chat Completions request
NATIVE	Importable Python <code>module:function</code> call
WEB	Arbitrary HTTP/HTTPS request
CHECKPOINT	Explicit checkpoint-preservation task

Remote variants of several task types are selected not by a different `TaskType` value, but by *keyword arguments*: for example, a `BATCH` task constructed with `ip=...` becomes a remote batch task transparently. This design keeps workflow code compact while still exposing several execution environments through one uniform factory, `DagonTask`. It also means that several integrations are *optional*: using Docker, cloud, Globus, or remote SSH execution requires the corresponding site-specific service or credential, while the core local-batch engine has no such external dependency.

Scientific Practice Note

LLM tasks — the engine’s principal AI integration point — call configured OpenAI-compatible Chat Completions providers and can consume local UTF-8 producer files through the same `workflow://` schema used by ordinary file-based tasks. This is the technical mechanism that allows an AI inference step to sit inside the same DAG as a numerical simulation, with the same dependency, staging, and provenance guarantees. ?? is devoted entirely to this mechanism.

1.7 Workflows in the Landscape of Computational Science

It is useful, before going further, to situate the workflow-orchestration problem within the broader history of computational science tooling, so that the design choices examined in later

chapters can be understood not as arbitrary engineering preferences but as responses to a recurring, well-documented set of pressures.

Three historically distinct strategies have been used to compose computational steps into a pipeline. The first, and oldest, is the *shell script*: a linear sequence of commands, executed in source-code order, with control flow expressed through exit codes and conditional branches. Shell scripts are easy to write and require no additional software, but they conflate two logically separate concerns — the *order* in which steps must run and the *mechanism* by which they run — into a single, brittle, line-by-line narrative. A shell script cannot, without substantial additional engineering, express that two steps are mutually independent and may run concurrently, nor can it express, in a form a second program can inspect, which step produced which file.

The second strategy is the *build system*, exemplified historically by **make** and its descendants. Build systems were designed to compile software, not to run scientific experiments, but they introduced an idea central to this book: a target (a file to be produced) depends on a set of prerequisites (other files), and a recipe (a command) explains how to produce the target from the prerequisites once they exist. This is, in essence, a restricted form of the data-oriented dependency model developed in ???. Build systems are excellent at avoiding redundant recomputation — a target whose prerequisites have not changed is not rebuilt — but they are typically confined to a single machine and a single execution environment, and their dependency language is tied tightly to file modification timestamps rather than to a richer notion of provenance.

The third strategy, and the one this book is concerned with, is the *dedicated scientific workflow engine*. Such engines generalize the build-system insight — dependencies as a directed graph over named artifacts — while relaxing its constraints: a task may run on a different machine than its consumer, may be backed by a container, a job scheduler, a remote web service, or an AI inference call, and the engine is expected to manage the data movement, environment provisioning, and failure recovery that a single-machine build tool can ignore. DAGOnStar belongs to this third family, and the remainder of this book can be read as an extended case study in how one engine in this family resolves the tension between *generality* (supporting arbitrary execution backends) and *simplicity* (keeping the authoring surface close to ordinary Python).

Scientific Practice Note

A useful heuristic for deciding whether a computational pipeline has outgrown a shell script and needs a workflow engine: if you find yourself writing comments in a shell script that explain *why* step three must follow step one, or maintaining a parallel README describing which script produced which file, the dependency structure you are documenting informally is exactly the structure a DAG-based engine would make executable.

1.8 A Formal Note on Topological Ordering

??? asserted, without proof, that acyclicity is precisely the condition under which a valid execution order exists. It is worth making this claim mathematically precise, both because it explains *why* `Validate_WF()` is sufficient to guarantee an executable workflow, and because it grounds the engine’s threaded execution model (??) in a standard result of graph theory.

Definition: Topological Order

A *topological order* of a directed graph $G = (V, E)$ is a linear ordering $v_1, v_2, \dots, v_n$ of V such that, for every edge $(v_i, v_j) \in E$, $i < j$. Equivalently, every node appears in the

ordering only after all nodes it depends on.

A standard theorem of graph theory states that a directed graph admits a topological order *if and only if* it is acyclic. The forward direction is immediate by contradiction: if a cycle $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k \rightarrow v_1$ existed, no linear ordering could place every predecessor before its successor along the entire cycle, since the cycle returns to its starting node. The converse can be shown constructively, by repeated removal of *source* nodes — nodes with no remaining incoming edges — from the graph: at every step, an acyclic graph with at least one node must contain at least one source (otherwise, following incoming edges indefinitely would, in a finite graph, eventually revisit a node, producing a cycle), and repeatedly removing sources and appending them to the output list produces a valid topological order.

This constructive procedure is, in fact, an algorithmic description of exactly what a DAG-based scheduler does at runtime: a task with no unsatisfied predecessor is, in DAGOnStar’s vocabulary, in the **READY** state (??); the engine’s threaded execution model effectively performs the source-removal procedure *concurrently*, starting every **READY** task as a thread, and transitioning a waiting task to **READY** the moment its last predecessor reaches **FINISHED**. `Validate_WF()`’s cycle rejection is therefore not an arbitrary safety check; it is the exact precondition the underlying scheduling theorem requires for *any* valid execution order, sequential or concurrent, to exist at all.

Scientific Practice Note

This formal correspondence has a practical consequence worth internalizing: because topological order is generally *not unique* when a DAG contains independent branches (as in ??, where **Berlin** and **Nairobi** could legally appear in either order, or interleaved), a DAGOnStar workflow author should never write task logic that silently depends on one sibling task happening to run before another, unless that ordering is encoded as an explicit or inferred dependency. If two tasks are not connected by an edge, the engine offers no guarantee about their relative execution order or interleaving.

1.9 Granularity: How Big Should a Task Be?

A question every new workflow author eventually confronts, and one the project’s documentation does not answer prescriptively, is how finely a computation should be decomposed into tasks. This is a genuine engineering trade-off, and it is worth reasoning about explicitly rather than leaving it to instinct.

Coarse-grained tasks — for instance, a single **BATCH** task that runs an entire multi-stage shell pipeline internally — minimize the number of scratch directories, launcher scripts, and `workflow://` references the author must manage, and minimize the staging overhead incurred at each dependency edge. Their cost is a loss of the very properties ?? opened by praising: traceability and recoverability degrade, because a failure partway through a coarse task provides no checkpoint boundary (??) and no intermediate, individually inspectable scratch directory.

Fine-grained tasks — one task per logical step, connected by explicit `workflow://` edges — maximize traceability (every intermediate product is a named, inspectable file in its own scratch directory), maximize recoverability (checkpointing can skip any individual completed step), and maximize the parallelism the engine can exploit (??). Their cost is increased staging overhead — every edge triggers a data-movement operation (??) — and a larger number of artifacts for a human reader to track.

Scientific Practice Note

A reasonable default, consistent with the five-stage prepare/run/validate/postprocess/publish pattern this book returns to repeatedly (??), is to draw task boundaries at points where a scientist would naturally want to *inspect* an intermediate product, or at points where the engine’s checkpoint or parallelism guarantees provide a concrete operational benefit — not at the boundary of every individual shell command. A model run that internally invokes three tightly coupled binaries with no scientifically meaningful intermediate product is usually one task; a model run followed by an independently useful post-processing step is usually two.

1.10 Chapter Summary

This chapter established the conceptual vocabulary used throughout the rest of the book: a workflow is a DAG of tasks; dependencies may be declared explicitly or inferred from data references; the engine materializes every task into an isolated scratch directory; and a small, uniform set of task types — spanning local shells, HPC schedulers, containers, clouds, web requests, native Python functions, and AI completions — share one dependency and staging mechanism. It further situated this design within the broader history of computational pipeline tools (shell scripts, build systems, workflow engines), gave a formal grounding for why acyclicity guarantees an executable schedule, and offered a first discussion of task granularity as a recurring design decision. ?? now turns this vocabulary into a precise account of the implementation that realizes it.

Exercise 1.1. Using the source-removal construction of ??, manually compute *all* valid topological orders of the DAG in ?. How many are there, and what does this number tell you about the maximum parallelism the engine could exploit if it had unlimited threads available?

Exercise 1.2. Using only the vocabulary of this chapter, classify the following dependency as task-oriented or data-oriented, and justify your answer: “the post-processing script must not run until the simulation has written its restart file.”

Chapter 2

The DAGonStar Computational Model and Architecture

This chapter develops, in precise and implementation-faithful terms, the internal architecture introduced informally in ???. The DAGonStar documentation states a strict principle that this chapter respects throughout: “*documentation must remain consistent with the software in this repository*” — this is a descriptive account of the current implementation, not an aspirational design document.

2.1 Package Layout

The engine’s source tree is organized as follows:

1	da gon/	
2	__init__.py	Workflow, Status, DataMover, Stager
3	task.py	TaskType, DagonTask, Task base class
4	batch.py	Batch, RemoteBatch, Slurm, RemoteSlurm
5	remote.py	RemoteTask, CloudTask
6	docker_task.py	DockerTask, DockerRemoteTask
7	kubernetes_task.py	KubernetesTask
8	apptainer_task.py	ApptainerTask
9	nomad_task.py	NomadTask
10	llm.py	LLMTask
11	native.py / native_runner.py	NativeTask and subprocess runner
12	web/	WebTask
13	faas.py	Provider-neutral FaaS lifecycle and artifacts
14	faas_providers.py	Lazy mock, HTTP, AWS, Azure, GCP adapters
15	faas_models.py	Dependency-free invocation/provider contracts
16	fair/	Optional FAIR recorder, exporters, validator
17	checkpoint.py	Checkpoint tasks
18	config.py	INI configuration reader
19	dag_tps.py	Meta-workflow coordination
20	api/	HTTP client and workflow service (optional)
21	cloud/	Apache Libcloud helpers
22	communication/	SSH, SCP, Globus, SKYCDS helpers
23	ftp_publisher/	FTP publishing helper
24	peer2peer/	Experimental peer-to-peer service functions

Listing 2.1: da gon/ package layout (v2.2).

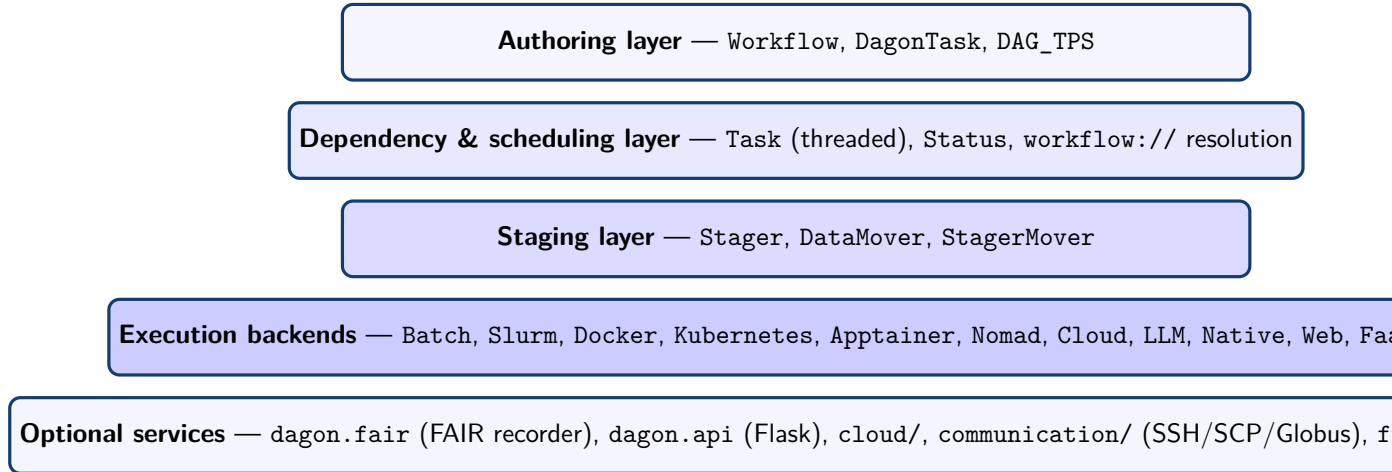


Figure 2.1: Layered view of the DAGonStar architecture. Each layer depends only on the layer(s) below it; optional services are loaded lazily so that core local-batch workflows never require Docker, cloud, or Globus dependencies to be installed.

2.2 Main Abstractions

2.2.1 Workflow

`dagon.Workflow` owns a list of tasks and coordinates dependency construction, execution, checkpoint state, configuration, and optional service registration. Its most important methods are:

- `add_task(task)` — register a task with the workflow.
- `make_dependencies()` — parse `workflow://` references and validate the resulting graph.
- `run(resume_checkpoint_file=None)` — execute the workflow synchronously, in the calling thread.
- `launch(resume_checkpoint_file=None)` and `wait(timeout=None)` — execute asynchronously, in a background thread (??).
- `as_json()` — serialize the workflow's structure.
- `find_task_by_name(workflow_name, task_name)` — locate a task, possibly across workflow boundaries.
- `Validate_WF()` — reject cyclic graphs (??).
- `set_data_mover(data_mover)` / `set_stager_mover(stager_mover)` — configure how files move between tasks.

2.2.2 DagonTask: the Task Factory

`DagonTask` is a *factory*: it receives a `TaskType` value and returns an instance of the concrete task class mapped to that type.

```
task = DagonTask(TaskType.BATCH, "A", "echo A")
```

Listing 2.2: `DagonTask` as a factory over concrete task classes.

This single line hides the dispatch logic: depending on `TaskType` and on auxiliary keyword arguments (such as `ip` for remote execution), `DagonTask` returns the appropriate subclass without the workflow author needing to import or instantiate it directly.

2.2.3 Task: the Common Base

`Task` extends `threading.Thread` and provides, uniformly across every execution backend:

- task status (see ??);
- predecessor (`prevs`) and successor (`nexts`) lists;
- a back-reference to the owning `Workflow`;
- scratch-directory creation;
- `workflow://` parsing (??);
- launcher-script generation;
- reference counting and garbage-collection hooks;
- checkpoint interaction (??, Appendix D).

Concrete task classes override `on_execute()` and related environment-specific hooks to specialize *how* a command is actually run, while inheriting all dependency, staging, and checkpoint behavior from `Task`. This is the architectural reason a heterogeneous DAG — mixing a local shell command, a Slurm submission, and an LLM completion — behaves uniformly with respect to dependencies and data movement: heterogeneity lives only in `on_execute()`.

2.3 Execution Lifecycle

A typical dataflow run follows a fifteen-step sequence, traced here exactly as the architecture documentation specifies it:

1. The user creates a `Workflow`.
2. The user creates tasks through `DagonTask`.
3. The user adds tasks with `workflow.add_task()`.
4. The user calls `workflow.make_dependencies()`.
5. Each task parses its command for `workflow://` references.
6. Dependencies are recorded in `prevs` and `nexts`.
7. `Workflow.Validate_WF()` rejects cycles.
8. The user calls `workflow.run()`.
9. Tasks are started as Python threads.
10. Each task waits for predecessor completion.
11. Each task creates or reuses a scratch directory.

12. Input data is staged.
13. The task launcher script is executed.
14. Status and checkpoint metadata are updated.
15. Dependent tasks proceed.

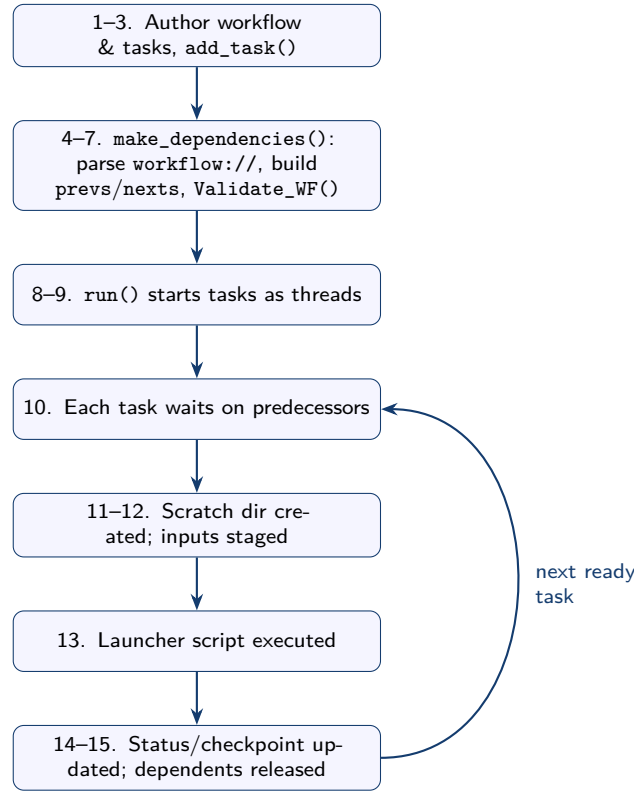


Figure 2.2: The fifteen-step execution lifecycle, grouped into seven logical phases. The loop at the right indicates that step 15 (releasing dependents) re-enters the per-task lifecycle for every task whose predecessors have just completed.

2.4 The Status Model

`dragon.Status` defines five task states:

Status	Meaning
READY	Eligible to run; no unmet dependency.
WAITING	Blocked on at least one predecessor.
RUNNING	Launcher script currently executing.
FINISHED	Completed with a successful exit code.
FAILED	Completed with a non-zero exit code or raised exception.

Status changes are local to the running process and, when the optional DAGon service is enabled, are additionally propagated through the service API (??).

2.5 The Dependency Model

Every task carries three pieces of dependency bookkeeping:

- **prevs**: the tasks that must finish before this task may run;
- **nexts**: the tasks that depend on this task;
- **reference_count**: the number of downstream data references, used by garbage-collection hooks for scratch-directory cleanup.

Explicit dependencies are created with `consumer.add_dependency_to(producer)`; implicit dependencies are created automatically by parsing `workflow:///producer/path/to/file` tokens in a task's command, as introduced in ?? and developed fully in ??.

2.6 Data Staging

`Stager.stage_in()` selects a data-movement strategy for a given dependency edge. The implemented `DataMover` enumeration is:

DataMover	Mechanism
DONTMOVE	No staging is performed.
LINK	Symbolic link to the producer's file.
COPY	Independent file copy.
SCP	Secure copy over SSH.
HTTP / HTTPS	Movement over plain or secured HTTP.
FTP / SFTP	Movement over (S)FTP.
GRIDFTP	GridFTP-based movement (Globus).
SKYCDS	SKYCDS-backed catalog/data movement.

The currently most exercised local paths are `COPY` and `LINK`; remote and specialized movers require external services and site-specific configuration. A complementary enumeration, `StagerMover`, controls the *staging mode*: `NORMAL`, `PARALLEL`, or `SLURM`.

2.7 Execution Backends

2.7.1 Batch

`Batch` executes a generated launcher script locally through `subprocess.Popen`. This is the default, dependency-free execution path used by nearly every example in this book.

2.7.2 Remote Batch

When a batch task is constructed with `ip=...`, the factory `Batch.__new__` transparently returns a `RemoteBatch` instance instead, which executes the same generated launcher through an `SSHManager`-mediated connection.

2.7.3 Slurm

`Slurm` generates an `sbatch` command from optional parameters such as partition, task count, memory, wall time, nodes, and tasks per node — the natural interface for compiled HPC models (??).

2.7.4 Docker

`DockerTask` creates or reuses a Docker container through the Docker Python SDK; the task command is wrapped inside `docker exec`.

2.7.5 Cloud

`CloudTask` extends remote execution with Apache Libcloud-backed instance management, allowing a task to provision or reuse a cloud virtual machine as its execution target.

2.7.6 Checkpoint

Checkpoint tasks preserve task working directories and checkpoint metadata. The base `Workflow` can resume from a checkpoint file independently of any explicit `CHECKPOINT` task (Appendix D).

2.8 Service Architecture

The optional `tagon.api` package contains two components:

- **API:** a client used by workflows to register and update remote service state.
- **WorkflowServer:** a Flask-based server wrapper exposing workflow operations over HTTP.

The default, documented configuration disables this service entirely:

```
1 [tagon_service]
2 use = False
```

When disabled — the recommended mode for introductory examples, unit tests, and the entirety of this textbook’s local exercises — DAGonStar runs as a purely local, in-process workflow engine with no external service dependency.

2.9 Known Architectural Constraints

In the interest of scientific and engineering honesty, this section reproduces, verbatim in substance, the constraints the project itself documents as open or partially mitigated:

Caution

- Tasks are Python *threads*, not isolated operating-system processes.
- Some command strings are still assembled by string concatenation; shell-command safety and path quoting remain an active improvement priority, although new staging helpers already quote generated staging paths.
- Optional integrations (Docker, cloud, Globus, API) are exposed through package *extras* and use lazy imports, so that core local-batch workflows do not require those dependencies to be installed.
- The repository has an initial unit-test and CI baseline, but comprehensive workflow, checkpoint, staging, and integration coverage is still being expanded.
- Public APIs have broad type annotations across workflow, task, configuration, staging, task-subclass, and API-client code; stricter static checking remains a maintainability goal.

- Some examples depend on external systems and cannot be verified in a generic continuous-integration environment.

2.10 The Factory Pattern in Depth

`DagonTask` deserves a closer reading than ?? gave it, because the factory pattern it implements is the architectural device that lets Part II of this book introduce four further execution backends (Docker, remote/Slurm, LLM, Web, Native) without ever changing the authoring surface a workflow writer interacts with.

In classical object-oriented design terms, a *factory* is a function or method that returns an instance of one of several related classes, selected by a parameter, without the caller needing to know — or import — the concrete class being instantiated. `DagonTask(TaskType.BATCH, ...)` and `DagonTask(TaskType.SLURM, ...)` return instances of entirely different classes (`Batch` and `Slurm` respectively, per the `tasks_types` mapping reproduced in Appendix A), yet both are constructed through an identical calling convention, and both are usable interchangeably as nodes in a `Workflow`’s task list. This is what makes the heterogeneous DAGs explored from ?? onward possible at all: a single `workflow.add_task()` call accepts any concrete task type with no special-casing required at the workflow level.

The remote-variant dispatch described in ?? — a `BATCH` task constructed with `ip=...` silently becoming a `RemoteBatch` — is a second, nested instance of the same pattern, implemented one level lower, inside `Batch.__new__` rather than inside `DagonTask` itself. The practical lesson for a workflow author is that *keyword arguments, not just the `TaskType` value, are part of the dispatch key*: two superficially similar `DagonTask` calls that differ only in the presence of an `ip` keyword can construct objects of entirely different concrete classes, executing in entirely different environments, while remaining structurally identical from the perspective of the dependency graph.

2.11 Threads, Not Processes: Consequences and Caveats

?? flagged, among the documented architectural constraints, that DAGonStar tasks are Python `threading.Thread` objects rather than isolated operating-system processes. This single design decision has consequences pervasive enough to merit a dedicated discussion, because it shapes what a workflow author may safely assume about isolation, concurrency, and resource accounting.

In CPython, the standard implementation most DAGonStar deployments will run on, the Global Interpreter Lock (GIL) serializes execution of Python bytecode across threads within a single process: at any instant, only one thread is actually executing Python code, even on a multi-core machine. This does *not*, however, prevent DAGonStar’s concurrent task execution from delivering real parallelism for the cases that matter most in scientific computing, for a structural reason: a `Batch` task’s actual computational work happens inside a *child process* spawned via `subprocess.Popen`, entirely outside the GIL’s purview. The controlling Python thread for that task spends almost all of its lifetime blocked waiting for the child process to exit — a blocking I/O-bound wait, which CPython threads handle well, releasing the GIL for other threads while blocked. The same reasoning extends to `RemoteTask` (blocked on a network socket during SSH execution), `DockerTask` (blocked on the Docker daemon’s API), `WebTask` (blocked on an HTTP response), and `LLMTask` (blocked on the same). Only `NativeTask`, whose computation runs as actual in-process Python code (??), is potentially GIL-bound, and therefore is the one task type whose *computational*, rather than purely I/O, concurrency

benefits most directly from the engine’s `executor="slurm"` escape hatch to out-of-process execution.

Caution

Thread-based isolation is weaker than process-based isolation in ways that matter for defensive workflow design. Threads share the controlling Python process’s memory space, open file descriptors, environment variables, and current working directory at the moment of thread creation. A bug in one task’s Python-level pre/post-processing code — as opposed to the external program or subprocess it launches — could in principle corrupt shared interpreter state in a way a process-isolated design would prevent entirely. This is part of why the developer guide (Appendix B) flags defensive shell quoting and careful handling of paths and user-controlled values as an active, ongoing engineering priority rather than a solved problem.

2.12 Comparing the Layered Architecture to Alternative Designs

It is instructive to contrast the four-layer architecture of ?? with two alternative architectural strategies a workflow engine could have adopted, because the contrast clarifies what DAGonStar is optimizing for.

A **monolithic engine** would fold dependency resolution, staging, and execution into a single class hierarchy, with each concrete task type re-implementing its own version of dependency tracking and data movement. This would reduce the indirection a reader must follow to understand any one task type in isolation, but it would multiply the surface area for bugs in dependency resolution — a defect in cycle detection, for instance, would need to be fixed and tested independently in every concrete task class, rather than once in the shared `Task` base class examined in ??.

A **microservice-oriented engine**, at the opposite extreme, would externalize scheduling and staging into separate, independently deployed services communicating over a network protocol, with the Python library reduced to a thin client. This would more cleanly separate concerns and would scale naturally to very large workflows or multi-tenant deployments, but at the cost of introducing a mandatory network dependency and operational complexity (service discovery, authentication, availability) for even the simplest single-machine workflow — exactly the cost the optional, disable-by-default DAGon service architecture (??) is designed to avoid imposing on the common case.

Scientific Practice Note

DAGonStar’s layered-but-in-process design (??) occupies a deliberate middle position: dependency resolution and staging are shared, reusable layers (avoiding the monolithic engine’s duplication), but they execute in-process by default (avoiding the microservice engine’s mandatory operational overhead), with the optional `dagon.api` package available as an escape hatch when a deployment genuinely needs multi-process or multi-host coordination beyond what SSH-based remote execution already provides.

2.13 Reading the Architecture Document as a Living Artifact

A final, methodological point belongs in this chapter rather than in the developer guide (Appendix B), because it concerns how a *reader*, not only a contributor, should treat architec-

tural documentation. ?? opened by quoting the project’s own principle that documentation must remain consistent with the current implementation, and ?? reproduced, deliberately without softening, the project’s own list of open architectural debt. This is not incidental even-handedness; it reflects a specific, defensible position about what makes technical documentation trustworthy for a scientific audience.

A textbook — or any documentation — that described only an engine’s intended, idealized behavior, omitting known gaps such as incomplete shell quoting or partial test coverage, would actively mislead a researcher relying on it to assess whether a given workflow’s results are trustworthy. The constraints listed in ?? are not flaws unique to DAGonStar; comparable caveats (partial coverage of edge cases, ongoing hardening of command construction, uneven test coverage across optional integrations) are common to research software more broadly, precisely because research software is typically developed under different incentives than commercial infrastructure software. Treating these caveats as part of the architecture, rather than as a separate errata list, is this book’s attempt to model the kind of honest engineering communication the developer guide (Appendix B) explicitly asks contributors to maintain.

2.14 Chapter Summary

The architecture is a thin, layered stack: an authoring layer (`Workflow`, `DagonTask`), a dependency and scheduling layer built on threaded `Task` objects, a staging layer that abstracts data movement behind `DataMover`/`StagerMover`, a family of execution backends that share this common infrastructure while specializing only `on_execute()`, and an optional service layer for distributed registration and status propagation. This chapter additionally examined the factory pattern underlying `DagonTask` and its nested remote-variant dispatch, reasoned about the consequences of thread-based rather than process-based task isolation, compared the chosen layered-in-process design against monolithic and microservice alternatives, and argued for treating architectural documentation — including its documented limitations — as part of the scientific record a researcher relies on. The next chapter puts this architecture to work by walking through installation and a complete first workflow.

Exercise 2.1. Explain, in terms of ??, why a `NativeTask` performing a computationally intensive, pure-Python operation (for example, a nested loop over a large array with no NumPy vectorization) might *not* benefit from running concurrently with several sibling `Batch` tasks in the same workflow, even though the engine schedules all of them as threads. What change to the task’s implementation, or to its `executor` argument, would most directly address this?

Chapter 3

Installation and the First Workflow

This chapter takes a new user from an empty machine to a verified, running DAGonStar workflow, following the project’s own `getting_started.md` guide.

3.1 Requirements

- Python 3.8 or newer.
- A POSIX-like shell for the included command examples.
- Git (required only for the source-checkout installation path).

Optional integrations require additional, separately verified services:

- Docker, for Docker tasks (??);
- SSH access, for remote tasks;
- Slurm, for HPC scheduler tasks;
- cloud provider credentials, for cloud tasks;
- Globus or SKYCDS credentials, for specialized data movement.

3.2 Three Installation Paths

The master branch of DAGonStar provides three supported installation paths that map onto three user roles:

Role	Command	When to use
End-user / student	<code>pip install dagonstar</code>	PyPI stable release; no checkout needed
Latest features	<code>pip install git+...</code>	Tracks master, no checkout
Developer / examples	<code>pip install -e .</code>	Editable source checkout; required for lessons that reference <code>examples/</code>

Scientific Practice Note

The PyPI release (`pip install dagonstar`) is a new capability introduced in the master branch. Prior editions of this textbook were based on the `development-native-web` branch, which required a source checkout. For the tutorial lessons in this book, which reference files in the repository's `examples/` tree, the editable source checkout (Path 3) is the recommended installation.

3.3 Installing DAGonStar

The three installation paths and their recommended contexts are:

Path	Role	Command	When to use
1	End-user / student / notebook	<code>pip install dagonstar</code>	PyPI stable release; no checkout needed; Jupyter and Colab-ready
2	Latest repository version	<code>pip install git+...</code>	Tracks master; no local files needed
3	Developer / tutorial lessons	<code>pip install -e .</code>	Editable source checkout; required when lessons reference <code>examples/</code> or <code>docs/tutorial/</code>

Scientific Practice Note

The PyPI release (Path 1: `pip install dagonstar`) was introduced in the master branch and represents the simplest entry point for new users and notebook environments. Prior editions of this textbook required the editable source checkout (Path 3) because only the source tree provided the tutorial scripts and example files. Path 3 remains required for every tutorial lesson in this book that runs a script from the `examples/` or `docs/tutorial/` directory. For Jupyter Notebooks and Google Colab, Path 1 is sufficient when the notebook does not reference repository files directly; Path 3 is required for the unified tutorial notebook (`DAGonStar_tutorial.ipynb`) and the AI-HPC lesson scripts (`examples/tutorial/lesson_12_llm_tasks.py`, etc.).

```

1 git clone https://github.com/DagOnStar/dagonstar.git
2 cd dagonstar
3 python3 -m venv .venv
4 . .venv/bin/activate
5 python -m pip install --upgrade pip
6 python -m pip install -r requirements.txt
7 python -m pip install -e .

```

Listing 3.1: Path 3 (recommended for this textbook): editable development installation.

```

1 python3 -m venv .venv && . .venv/bin/activate
2 pip install --upgrade pip
3 pip install git+https://github.com/DagOnStar/dagonstar.git

```

Listing 3.2: Path 2: latest repository version, no checkout needed.

For users who only need the packaged release without the repository files:

```

1 python3 -m venv .venv && . .venv/bin/activate
2 pip install --upgrade pip && pip install dagonstar

```

Listing 3.3: Path 1: PyPI stable release.

For a smaller installation footprint, install only the core package and the optional integrations actually needed:

```

1 python -m pip install -e .
2 python -m pip install -e ".[docker]"      # Docker tasks
3 python -m pip install -e ".[cloud]"      # cloud tasks
4 python -m pip install -e ".[globus]"     # Globus staging
5 python -m pip install -e ".[api]"        # Flask workflow service

```

Listing 3.4: Minimal install with selective extras.

Scientific Practice Note

This extras-based packaging is itself a reproducibility decision: a published workflow's installation instructions should record *exactly* which extras were required, so a future reader does not need to guess whether Docker or cloud credentials are necessary to reproduce a given experiment.

3.4 Creating a Configuration File

```

1 cp dagon.ini.sample dagon.ini

```

For a first local workflow, the sample defaults suffice. Confirm the two keys that matter most for local execution:

```

1 [dagon_service]
2 use = False
3
4 [batch]
5 scratch_dir_base=/tmp/

```

The full configuration model is the subject of ?? and Appendix C.

3.5 Writing a First Workflow

```

1 from dagon import Workflow
2 from dagon.task import DagonTask, TaskType
3
4
5 if __name__ == "__main__":
6     workflow = Workflow("FirstWorkflow")
7
8     prepare = DagonTask(
9         TaskType.BATCH,
10        "prepare",
11        "mkdir -p data; echo 'temperature,depth' > data/profile.csv; "
12        "echo '18.5,12' >> data/profile.csv",
13    )
14
15    analyze = DagonTask(
16        TaskType.BATCH,
17        "analyze",

```

```

18     "wc -l workflow:///prepare/data/profile.csv > summary.txt",
19 )
20
21 workflow.add_task(prepare)
22 workflow.add_task(analyze)
23 workflow.make_dependencies()
24 workflow.run()

```

Listing 3.5: `first_workflow.py`: a two-task, data-oriented workflow.

```
1 python first_workflow.py
```

3.5.1 What Happens, Step by Step

1. `prepare` creates a small CSV file inside its own scratch directory.
2. `analyze` references that file with `workflow:///prepare/data/profile.csv`.
3. `workflow.make_dependencies()` detects, by parsing `analyze`'s command, that `analyze` depends on `prepare`.
4. DAGonStar stages the referenced file into the `analyze` task's working directory.
5. `summary.txt` is produced by the `analyze` task.

This five-step trace is a miniature instance of the fifteen-step lifecycle formalized in ??; the reader is encouraged to map each numbered step here onto the corresponding lifecycle phase.

3.6 Inspecting Generated Task Directories

By default, task directories are created under `/tmp/`, with names that include a millisecond timestamp and the task name, e.g.:

```

1 /tmp/1710000000000-prepare
2 /tmp/1710000000100-analyze

```

Each task directory contains:

- task output files;
- a `.dagon/` directory;
- generated launcher scripts;
- captured standard output.

?? examines the contents of `.dagon/` in detail, treating the generated launcher as a first-class provenance artifact.

3.7 Running the Unit Test Suite

```
1 python -m unittest discover -s tests -v
```

The test suite verifies behavior that the project commits to keeping stable while new features are developed; running it after installation is a useful sanity check that the local environment matches the documented one.

3.8 Understanding the Installation Footprint

The two installation paths given above — the full `requirements.txt` install and the minimal extras-based install — are not merely a convenience choice; they encode a deliberate packaging philosophy that is worth making explicit before proceeding further, since it recurs throughout Part II whenever a new execution backend is introduced.

The core package — `Workflow`, `DagonTask`, the `BATCH` backend, dependency resolution, and local data staging — has, by design, no dependency on Docker, any cloud SDK, Globus, or a web framework. A researcher whose entire pipeline consists of local shell commands and `workflow://` references can install DAGonStar with a minimal footprint and a correspondingly small attack surface and dependency tree. Every additional capability examined in ?? onward — container execution, cloud provisioning, Globus-mediated transfer, the optional HTTP service — is gated behind an explicit `pip` extra, and, as ?? noted, the corresponding Python modules use *lazy imports*, meaning the module is not even imported, let alone required to be installed, unless a workflow actually constructs a task of that type.

Scientific Practice Note

This has a direct, practical consequence for reproducibility reporting, foreshadowed already in this chapter’s earlier discussion of extras: a workflow’s installation instructions should state not just “install DAGonStar,” but the precise set of extras required, because that set is itself a statement about which execution backends — and therefore which external services and credentials — a reader will need to provision before they can run the workflow end to end. A workflow using only `BATCH` and `LLM` tasks against a mock provider, for instance, needs no extras at all; one that also schedules `DOCKER` tasks needs the `docker` extra and a running daemon.

3.9 The First Workflow as a Minimal Reproducible Experiment

It is worth pausing on `first_workflow.py` a second time, after the step-by-step trace already given, to read it specifically as an instance of what ?? called a scientific benefit rather than merely as a syntactic example.

Consider what would be required to reproduce this two-line “experiment” — the claim that a small CSV profile, once line-counted, yields a particular summary — using only shell history. A second researcher would need to know, without being told explicitly, that the `wc -l` invocation depended on the CSV having already been generated by an earlier, separately run command; nothing in the shell history itself records that dependency as a structural fact, only as an accident of the order in which commands happened to be typed. The DAGonStar version of the same experiment makes the dependency a first-class, parseable part of the program text: `workflow:///prepare/data/profile.csv` cannot be misread or silently decoupled from its producer, because `make_dependencies()` re-derives the edge from the reference every time the script runs, on any machine, in any order the engine chooses to schedule it. This is precisely the *traceability* benefit named in ??, made concrete in the smallest possible example before Part I’s later chapters scale it up to checkpointed, staged, multi-backend pipelines.

Exercise 3.1. Modify `first_workflow.py` so that a third task, `validate`, depends on `analyze`’s `summary.txt` output and exits with a non-zero status if the line count it contains is zero (indicating an empty or malformed profile). Run the modified workflow and verify, using the techniques of ??, that the `validate` task is correctly recorded as `FAILED` when you intentionally corrupt the `prepare` task’s output to produce an empty file.

3.10 Common Early Pitfalls

New users of any workflow engine tend to encounter a small, predictable set of early mistakes; naming them explicitly here, before Part I develops the underlying mechanisms in full, should save the reader some early debugging time.

Forgetting `make_dependencies()` while still inspecting the graph.

As ?? will show, `run()` and `launch()` both build missing dependencies automatically, so omitting the explicit call does not break execution — but it does mean that any code placed *between* task construction and `run()` that inspects `task.prevs` or `task.nexts` will see an empty, not-yet-resolved graph. Call `make_dependencies()` explicitly whenever the graph must be inspected or validated before execution.

Confusing the producer task’s working directory with the consumer’s.

A `workflow://` path is always relative to the *producer* task’s directory (??), never to the consumer’s; a common early error is writing a reference as though the path were relative to wherever the consumer happens to be running.

Assuming sibling tasks run in a particular order.

As ?? makes precise, two tasks with no dependency edge between them have no guaranteed relative order; code that implicitly relies on one finishing before the other without an edge connecting them is a latent bug that may not surface until the workflow is run on a machine with different thread-scheduling behavior.

Treating `/tmp` as durable storage.

The default `scratch_dir_base` of `/tmp/` is convenient for the tutorial-scale examples in this book, but `/tmp` is frequently cleared on system reboot or by periodic cleanup policies; any workflow relying on checkpoint reuse (??) across a span of time longer than a single working session should configure a more durable `scratch_dir_base`, as ?? discusses.

3.11 Where to Go Next

- ?? develops the complete configuration model.
- ?? formalizes the `workflow://` schema used implicitly in this chapter’s example.
- ?? onward follow the tutorial’s fourteen lessons incrementally.
- ?? explains how to wrap compiled, command-line scientific models such as WRF.
- ?? should be revisited before modifying engine internals.

Chapter 4

The workflow:// Schema: Data-Oriented Dependency Inference

The `workflow://` schema is DAGonStar’s mechanism for expressing file-level dependencies between tasks without requiring the workflow author to state them explicitly. This chapter gives the schema a complete, formal treatment, because it is the single mechanism that later unifies file-based scientific software (??), LLM tasks (??), Web tasks (??), and Native Python tasks (??) under one dependency model.

4.1 Purpose

The schema lets a command refer to a file produced by another task. DAGonStar infers the dependency from that reference and stages the file before the consumer task runs — turning a textual reference inside a shell command (or JSON request, or function-argument mapping) into a structural edge of the workflow DAG.

4.2 Formal Shape

Definition: `workflow://` URI Grammar

```
1 workflow://<workflow-name>/<task-name>/<path-inside-task-working-  
  -directory>
```

For references inside the *same* workflow, `<workflow-name>` may be omitted, yielding the common shorthand:

```
1 workflow:///<task-name>/<path-inside-task-working-directory>
```

```
1 workflow:///A/output/f1.txt  
2 workflow://Forecast/preprocess/out/forcing.nc  
3 workflow://Observations/download/data/profile.csv
```

Listing 4.1: Representative `workflow://` references.

4.2.1 Components

Scheme. The literal scheme `workflow://`, defined in the implementation as the class constant `Workflow.SCHEMA = "workflow://"`.

Workflow name. Identifies the *producer* workflow. Optional for references inside the current workflow (`workflow:///producer/output.dat`), mandatory for cross-workflow references (`workflow://OtherWorkflow/producer/output.dat`; see ?? for meta-workflow composition).

Task name. The producer task's `name` attribute, e.g. `prepare` in `workflow:///prepare/input.nc`.

Path. The remaining path segment, interpreted *relative to the producer task's working directory*. For example, `workflow:///prepare/output/result.nc` resolves to `<prepare working directory>/output/result.nc`.

4.3 Dependency Inference Algorithm

When `Workflow.make_dependencies()` is called, every task runs its `pre_run()` method. `pre_run()` scans the task's command string for `workflow://` occurrences, and, for each occurrence:

1. parses the workflow name, task name, and file path;
2. finds the producer task (locally, or via the optional DAGon service for cross-workflow references when no local match exists);
3. appends the consumer task to the producer's `nexts`;
4. appends the producer task to the consumer's `prevs`;
5. increments the producer's `reference_count`.

Consequently, the single-line command

```
1 DAGonTask(TaskType.BATCH, "B", "cat workflow:///A/output/f1.txt")
```

deterministically creates the graph edge $A \rightarrow B$, with no separate `add_dependency_to()` call required.

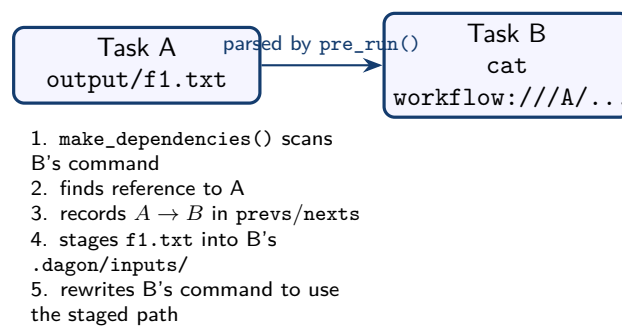


Figure 4.1: Dependency inference and command rewriting triggered by a single `workflow://` reference.

4.4 Command Rewriting

Before execution, `Task.pre_process_command()` stages the referenced file and rewrites the command body: the producer reference is replaced by a path inside the consumer task's working directory, under

```
1 .dagon/inputs/<workflow-name>/<task-name>/...
```

This lets the external program — which may be an arbitrary, unmodified scientific executable — read what it sees as an ordinary, local filesystem path, with no awareness that the file originated in a sibling task’s scratch directory.

4.5 Whitespace Boundary Rule

Caution

The current parser scans from `workflow://` until the next whitespace character or the end of the command string. Paths containing spaces are therefore *not* safely supported in `workflow://` references.

Recommended: `workflow:///A/output/result.nc`

Avoid: `workflow:///A/output/result with spaces.nc`

This constraint should be treated as a hard naming rule for any scientific data product that may ever be referenced through the schema: avoid spaces in output file and directory names produced by any task.

4.6 Same-Workflow and Cross-Workflow Examples

```
1 task_a = DagonTask(TaskType.BATCH, "A", "mkdir -p output; echo data > output
  /f1.txt")
2 task_b = DagonTask(TaskType.BATCH, "B", "cat workflow:///A/output/f1.txt >
  copy.txt")
```

Listing 4.2: Same-workflow producer/consumer pair.

```
1 task = DagonTask(
2     TaskType.BATCH,
3     "run_model",
4     "cat workflow://Observations/collect/out/forcing.txt > model_input.txt",
5 )
```

Listing 4.3: Cross-workflow reference, used by meta-workflows.

Cross-workflow behavior underlies the meta-workflow and transversal patterns developed in ??, where it may involve DAG_TPS or the optional DAGon service.

4.7 Interaction with Data Staging

The schema identifies *what* must be staged; *how* it is staged is controlled by the workflow’s or task’s data mover (??): COPY, LINK, SCP, GRIDFTP, or SKYCDS. For local workflows, COPY is the safest default; specialized movers require external services or remote configuration.

4.8 Extension to Non-File-Based Tasks

A defining property of the engine’s later AI-HPC integration is that the `workflow://` schema is *not* restricted to shell commands. `TaskType.LLM` parses `workflow://` values in both its JSON prompt and its `input_files` mapping, infers the producer dependency, and copies local UTF-8 text inputs into its `.dagon/inputs` folder before including their content in the API request body (??). `TaskType.WEB` similarly parses recursive `workflow://` values inside an HTTP

request specification (`??`), and `TaskType.NATIVE` accepts them as Python function-argument bindings (`??`). The schema is, in this precise sense, the engine’s *universal* dependency language.

4.9 Best Practices

- Use stable task names; the schema and checkpointing both key on them.
- Use simple path names without spaces (`??`).
- Write producer outputs under explicit, named directories such as `output/`.
- Reference exact files rather than broad directory assumptions where possible.
- Always call `workflow.make_dependencies()` before `workflow.run()` when the dependency graph should be inspected or validated ahead of execution — although, as `??` notes, `run()` and `launch()` build it automatically if it has not already been built.
- Keep cross-workflow references rare and well documented.

4.10 Common Errors

Missing producer task

If a referenced task is not found and the DAGon service is disabled, DAGonStar raises a connection/service-related error, because it cannot resolve the reference externally.

Dependency cycle

If `workflow://` references create a cycle, graph validation fails (`??`).

Wrong path inside producer task

The dependency may be structurally valid, yet staging or downstream execution fails if the producer did not actually create the referenced file — a reminder that the schema expresses *intended* data lineage, not a guarantee of file existence.

Path contains spaces

The parser uses whitespace as a token boundary; see `??`.

4.11 The Schema as a Naming Scheme: A Formal View

It is useful to read the `workflow://` grammar of `??` through the lens of *naming schemes* in distributed systems, a perspective that clarifies precisely what guarantee the schema does, and does not, provide.

A naming scheme maps names — here, `workflow://` URIs — to resources — here, files inside task scratch directories. Two properties are commonly demanded of a useful naming scheme: *location transparency*, meaning a name does not encode the physical location of the resource it refers to, only its logical identity; and *resolvability*, meaning the system can deterministically map a name to a current physical location. The `workflow://` schema satisfies location transparency by construction: a reference such as `workflow:///prepare/output/result.nc` says nothing about *where* `prepare`’s scratch directory physically resides — under `/tmp/`, under a configured alternate `scratch_dir_base` (`??`), or, for a remote task, on a different host entirely. The mapping from this logical name to a physical path is resolved at `pre_process_command()` time (`??`), using only information already present in the in-memory task graph: the producer task’s recorded `working_dir`.

This formal framing also clarifies why the schema’s resolvability is *intra-workflow-execution* only: a `workflow://` reference is meaningful only in the context of a graph that DAGonStar itself constructed and whose tasks’ working directories it itself tracked. A reference copied out of one workflow run’s source code and pasted, unmodified, into an unrelated script with no corresponding producer task would not resolve to anything — the name carries no inherent physical meaning outside the dependency graph that gives it one. This is, again, exactly the *Missing producer task* failure mode already catalogued in ??’s “Common Errors” section, now understood as a structural consequence of how the naming scheme is resolved rather than as an arbitrary implementation limitation.

4.12 Comparison with Alternative Dependency-Expression Mechanisms

Having developed the schema’s mechanics in full, it is worth comparing it explicitly against the two dependency-expression alternatives a workflow engine could instead have chosen, to clarify what specific trade-off DAGonStar’s designers accepted.

Configuration-file dependency declaration — in which a separate manifest (a YAML or JSON file, say) lists, for every task, the names of its predecessors — decouples the dependency declaration from the command string entirely. This has the advantage of being inspectable by tooling that does not parse arbitrary shell syntax, but it reintroduces precisely the synchronization risk ?? identified for explicit, asserted dependencies: a manifest entry and the command that actually consumes a file can silently drift apart, because nothing forces the two to agree. `workflow://` dependency inference avoids this drift structurally, by deriving the dependency from the very same string the consumer’s command will execute.

Implicit, timestamp-based dependency inference — the strategy classical build systems such as `make` use, inferring that a target is stale if any prerequisite file’s modification time is newer — requires no explicit reference syntax at all, but it depends on a shared, reliable filesystem clock and breaks down precisely in the heterogeneous, multi-host, multi-backend setting (??) DAGonStar is designed for: a remote task’s output file’s modification timestamp, as observed through SSH, may not be directly comparable to a local task’s, especially across hosts with imperfectly synchronized clocks. `workflow://` inference sidesteps the clock-synchronization problem entirely, because it never inspects timestamps; the dependency is structural, derived once at `make_dependencies()` time, and the engine’s own `Status` state machine (??), not the filesystem clock, is the sole authority on whether a producer has actually finished.

Scientific Practice Note

The schema’s chosen strategy — embedding the dependency reference directly inside the artifact (command, prompt, or request specification) that consumes it — trades a small amount of syntactic novelty (a workflow author must learn the `workflow://` grammar) for the elimination of an entire class of synchronization bugs that both configuration-file and timestamp-based alternatives are structurally prone to. This trade-off is visible again, in a different guise, every time ?????? show the same embedded-reference strategy extended into a JSON prompt, an HTTP request specification, and a Python argument-binding dictionary, respectively.

4.13 Provenance Beyond the Single Reference

A single `workflow://` reference establishes one edge of a provenance graph; it is worth being explicit about what larger structure emerges once a realistic workflow contains many such references, because this larger structure is the actual scientific deliverable a DAGOnStar workflow produces, alongside its numerical results.

Define the *provenance graph* of a completed workflow run as the directed graph whose nodes are the workflow’s tasks, annotated with their recorded `working_dir`, generated `.dagon/launcher.sh` (??), and final `Status`, and whose edges are exactly the dependencies `make_dependencies()` constructed — the same structure already developed as the dependency DAG, but now read *retrospectively*, after execution, as a historical record rather than a forward scheduling plan. Because every edge in this graph corresponds to an actual, staged file copy (under the chosen `DataMover`) and a rewritten command, the provenance graph is, in a precise sense, more complete than the kind of provenance a researcher typically reconstructs by hand from a shell history or a lab notebook: it records not just *that* one step followed another, but the *exact* file each step consumed, the exact path it was staged to, and the exact, fully-resolved command that ran.

Example 4.1. Consider a three-task chain `download` $\rightarrow$ `preprocess` $\rightarrow$ `simulate`, of the kind ?? discusses in the context of checkpointing. After a successful run, a researcher who receives only the three scratch directories — without the original Python source — can nonetheless reconstruct the full provenance chain by reading each task’s `.dagon/launcher.sh` in turn: `simulate`’s launcher will reference a path under `.dagon/inputs/<workflow>/preprocess/...`, which can be traced back to `preprocess`’s own output directory, and so on. This reconstructibility-from-artifacts-alone property is a direct, mechanical consequence of the command-rewriting behavior described in ??, and is part of why this book treats the generated launcher script, first introduced in ??, as a first-class scientific artifact in its own right, not merely an implementation detail.

4.14 Chapter Summary

The `workflow://` schema converts a textual file reference embedded in a task’s command, prompt, request specification, or argument binding into a structural DAG edge, a staged local file, and a rewritten, self-contained command. It is the single dependency language shared by every execution backend covered in this book, from a local `BATCH` task in ?? to an LLM task in ??. This chapter additionally framed the schema formally as a location-transparent naming scheme, compared it against configuration-file and timestamp-based dependency-inference alternatives, and showed how a workflow’s accumulated `workflow://` references constitute a reconstructible provenance graph that survives independently of the original source code.

Exercise 4.1. Using the provenance-graph notion of this chapter’s final section, describe precisely what information would be *lost*, and what would still be recoverable, if a workflow author replaced every `workflow://` reference in a five-task pipeline with explicit `add_dependency_to()` calls plus hard-coded absolute paths copied by hand from a previous run’s scratch directories. Relate your answer to ??.

Chapter 5

Configuration Management

5.1 Configuration Sources

DAGonStar reads configuration from an INI file, conventionally named `tagon.ini`; the sample file `tagon.ini.sample` documents every key recognized by the current implementation. Configuration is loaded by `tagon.config.read_config()`, which returns a nested dictionary of strings:

```
1 from dagon.config import read_config
2
3 cfg = read_config("tagon.ini")
4 scratch = cfg["batch"]["scratch_dir_base"]
```

Listing 5.1: Reading configuration programmatically.

A Workflow may also receive an in-memory configuration dictionary directly, bypassing the INI file entirely — useful for unit tests or programmatic workflow construction, and used extensively by the LLM and Web task lessons in Part II:

```
1 from dagon import Workflow
2
3 workflow = Workflow("Experiment", config={
4     "batch": {"scratch_dir_base": "/tmp/", "run_base": "", "threads": "1"},
5     "dagon_service": {"use": "False"},
6     "slurm": {"partition": ""},
7 })
```

Listing 5.2: In-memory configuration.

5.2 Minimal Configuration for Local Batch Workflows

```
1 [dagon_service]
2 route = http://localhost:57000
3 use = False
4
5 [ftp_pub]
6 ip = localhost
7
8 [batch]
9 scratch_dir_base=/tmp/
10 run_base=
11 remove_dir=False
12 threads=1
13
```

```
14 [slurm]
15 partition=
```

The current implementation provides safe defaults for missing optional `dagon_service` and `ftp_pub` values, but production workflows should keep these sections explicit, for clarity and for the sake of a future reader auditing the configuration.

5.3 Section-by-Section Reference

5.3.1 [dagon_service]

```
1 [dagon_service]
2 route = http://localhost:57000
3 use = False
```

When `use = True`, `Workflow` attempts to register itself with the DAGon service through `dagon.api.API`; the service then mediates workflow/task registration, status updates, and some transversal workflow lookups (??). When disabled, DAGonStar runs as a purely local, in-process engine — the recommended mode for introductory examples and unit tests, and the mode used throughout this book’s local exercises.

5.3.2 [ftp_pub]

```
1 [ftp_pub]
2 ip      = localhost
3 user    = anonymous
4 password =
```

Used by transversal workflow handling when task data must be downloaded from a host registered through the service. The three keys (`ip`, `user`, `password`) are required by the current in-memory configuration dictionary used in tutorial examples and notebook cells (see ?? and Appendix E); an incomplete `ftp_pub` section will raise a `KeyError` at runtime when the transversal data-movement path is exercised. For local-only workflows that never use transversal movement, the safe default values shown above (`ip = localhost`, `user = anonymous`, empty password) are sufficient.

Caution

Never store real FTP credentials in any committed configuration file. Use environment variables or a site-specific secret-management process for production deployments that require authenticated FTP access.

5.3.3 [dagon_ip]

```
1 [dagon_ip]
2 ip = localhost
```

Present in the sample configuration for service/node-related workflows; not required for basic local examples.

5.3.4 [batch]

```

1 [batch]
2 scratch_dir_base=/tmp/
3 run_base=
4 remove_dir=False
5 threads=1

```

scratch_dir_base

base directory used to create task scratch directories (??).

run_base

optional format string for grouping a workflow run under a timestamped subdirectory; supports strftime-style tokens and the literal token `__MILLIS__`.

remove_dir

documented as a configuration intent; in the current implementation, task garbage collection is controlled primarily by task reference counts and task flags rather than this key alone.

threads

used by staging command generation for parallel movement modes.

```

1 [batch]
2 scratch_dir_base=/tmp/dagon
3 run_base=%Y%m%d-%H%M%S-__MILLIS__
4 threads=4

```

Listing 5.3: Grouping a run under a timestamped subdirectory.

5.3.5 [slurm]

```

1 [slurm]
2 partition=

```

The configured `partition` value is used when generating Slurm staging commands. Individual SLURM task objects may also receive Slurm parameters directly at construction time (??):

```

1 task = DagonTask(
2     TaskType.SLURM, "model", "./run_model.sh",
3     partition="compute", ntasks=16, time="02:00:00",
4 )

```

5.3.6 LLM Provider Sections

`TaskType.LLM` reads an OpenAI-compatible provider configuration from a section named `[llm.<name>]`; an arbitrary number of provider sections may coexist:

```

1 [llm.example]
2 endpoint=https://api.example.org
3 api_key_env=DAGON_LLM_EXAMPLE_API_KEY
4 model=example-chat-model

```

`api_key_env` names a runtime environment variable holding the credential; `api_key` is available only for a local, ignored configuration file and must never be committed. `??` treats the full request/response model for this task type.

5.3.7 Web Task Authentication

`TaskType.WEB` authentication is supplied through *environment-variable names* embedded in the task specification, such as `token_env`, `username_env`, `password_env`, or `value_env`. As with LLM provider keys, literal HTTP tokens, passwords, or API keys must never be placed in configuration samples or workflow source files (??).

5.3.8 Cloud Provider Sections

```

1  [ec2]
2  key=<aws_access_key_id>
3  secret=<aws_secret_access_key>
4  region=
5
6  [digitalocean]
7  key=
8
9  [gce]
10 key=
11 secret=
12 project=

```

Cloud tasks use Apache Libcloud through `dagon.cloud`. All values are provider-specific and must be treated as secrets; the sample placeholders are to be replaced only in local, non-committed configuration files. Install cloud dependencies with:

```

1 python -m pip install "dagonstar[cloud]"

```

5.3.9 Logging Sections

The sample configuration includes a standard Python `logging.config.fileConfig` setup:

```

1  [loggers]
2  keys=root
3
4  [handlers]
5  keys=stream_handler
6
7  [formatters]
8  keys=formatter
9
10 [logger_root]
11 level=DEBUG
12 handlers=stream_handler
13
14 [handler_stream_handler]
15 class=StreamHandler
16 level=DEBUG
17 formatter=formatter
18 args=(sys.stderr,)
19
20 [formatter_formatter]
21 format=%(asctime)s %(name)-12s %(levelname)-8s %(message)s

```

When a `Workflow` is constructed without an explicit `config` dictionary, it calls `fileConfig(config_file)` internally; therefore, a file-based configuration *must* include valid logging sections for the engine to initialize correctly.

5.4 Runtime Environment Variables

SKYCDS and Globus credentials are intentionally not hard-coded anywhere in configuration; the current implementation reads them exclusively from the process environment:

```
1 export DAGON_SKYCDS_CLIENT_TOKEN=...
2 export DAGON_SKYCDS_CATALOG_TOKEN=...
3 export DAGON_SKYCDS_API_TOKEN=...
4 export DAGON_SKYCDS_IP=...
5
6 export DAGON_GLOBUS_TRANSFER_TOKEN=...
```

The current Globus transfer setup primarily uses the OAuth flow implemented in `GlobusManager.__init__`, which prompts the user to authorize a native application interactively.

5.5 Optional Integration Dependencies

```
1 python -m pip install "dagonstar[docker]"
2 python -m pip install "dagonstar[globus]"
3 python -m pip install "dagonstar[api]"
4 python -m pip install "dagonstar[all]"
```

5.6 Configuration as a Layered Override Mechanism

It is worth being explicit about the precedence relationship between the two configuration-loading paths ?? introduced — the file-based `config_file` path and the in-memory `config` dictionary — because conflating them is a frequent source of confusion when a workflow’s behavior unexpectedly diverges from what a committed `dagon.ini` appears to specify.

When a `Workflow` is constructed with an explicit `config` dictionary, that dictionary is used *in its entirety*, and `config_file` — even if also supplied, or left at its default value of `"dagon.ini"` — is not consulted at all for any section. This is not a partial override or a merge of the two sources; it is a complete substitution. The practical consequence, already exploited throughout the LLM, Web, and Native task lessons (??????), is that an in-memory `config` dictionary must itself be *complete* with respect to every section the workflow’s tasks will need — a runtime-constructed `llm.local_mock` section is of no help if the same in-memory dictionary omits the `[batch]` section the workflow’s `BATCH` producer task requires, since there is no fallback to the INI file’s corresponding section once the in-memory path has been chosen.

Caution

This all-or-nothing semantics also explains a subtlety in the logging behavior noted earlier in this chapter: `fileConfig(config_file)` is invoked only in the file-based path. A workflow constructed with an explicit `config` dictionary does not automatically configure Python’s logging subsystem; if a workflow author wants both an in-memory configuration and properly initialized logging, the logging setup must be performed separately, before workflow construction, using the standard library’s `logging` module directly.

5.7 Auditing a Configuration File for Reproducibility

?? closes, in its final section, with a list of configuration rules for reproducible science; it is useful to turn that list into an explicit auditing procedure a researcher can apply to an existing,

inherited `dragon.ini` before relying on it.

1. **Locate every secret-bearing key.** Search the file for `key`, `secret`, `api_key` (as opposed to `api_key_env`), and `password`; any non-empty, non-placeholder value found should be treated as a credential leak and rotated, not merely removed from the file.
2. **Confirm the service is disabled unless genuinely needed.** Check `[dragon_service] use`; for the overwhelming majority of single-researcher, single-machine workflows discussed in this book, this should read `False` (`??`).
3. **Verify the scratch base has adequate, durable storage.** As `??` cautioned, a default of `/tmp/` is convenient for short tutorial runs but unsuitable for any workflow relying on checkpoint reuse across sessions.
4. **Cross-check every `[llm.<name>]` and Web-task `*_env` reference against the deployment environment.** A provider section naming `api_key_env=DAGON_RESEARCH_LLM_KEY` is only as reproducible as the documentation, external to the repository, recording that this environment variable must be set before the workflow can run — confirm such documentation exists.
5. **Confirm the logging sections are present and valid** if the workflow does not supply an in-memory `config` dictionary, per the previous section’s discussion of `fileConfig` behavior.

Scientific Practice Note

This auditing procedure is deliberately framed as something a researcher does *once*, when first inheriting or reviewing a workflow’s configuration, and then records as part of the workflow’s own documentation — precisely the discipline `??`’s closing section asks for when it distinguishes operational secrets from reproducibility metadata. An audit that is performed but never written down provides no benefit to the next researcher who needs to run the same workflow.

5.8 Configuration Sections Not Yet Exercised in This Book

For completeness, and because Appendix C consolidates every section into a single reference document, it is worth noting explicitly which configuration sections this book’s worked examples do *not* exercise, so that a reader does not mistake their absence from the lesson chapters for an absence from the engine itself. The cloud provider sections (`[ec2]`, `[digitalocean]`, `[gce]`) and the `[dragon_ip]` section fall into this category: `CLOUD` tasks are described architecturally in `??` and referenced in the backend-selection guidance of `??`, but, consistent with this book’s local-first verification philosophy (`??`), no worked, runnable cloud example is given, because doing so responsibly would require a live, billable cloud account that cannot be assumed of every reader. A researcher who needs to configure a cloud provider section should treat the placeholders in Appendix C as a starting structural template, to be completed and verified against the target provider’s own current documentation and credential model rather than against this book, since provider-specific authentication mechanisms are, by their nature, outside the scope of what the DAGOnStar documentation itself specifies.

5.9 Configuration Rules for Reproducible Science

Scientific Practice Note

For scientific workflows, configuration is itself part of the scientific record and should be versioned with care:

- commit safe templates such as `dragon.ini.sample`;
- never commit real secrets;
- record the configuration values that affect scientific reproducibility — scratch layout, model parameters, container images, cloud instance types, and Slurm resource requests;
- keep operational secrets strictly separate from reproducibility metadata, e.g. by referencing environment-variable *names* in versioned configuration while the variable *values* live only in the runtime environment.

The full, consolidated configuration reference is reproduced in Appendix C for use as a standing lookup table while authoring workflows.

Chapter 6

Local Workflows and Explicit Dependencies

Running this chapter in Google Colab

All lessons in this chapter run in Google Colab with Option 3 installation (`git clone` then `pip install -e .`, see Appendix E). Use `local_config(workdir)` instead of `daemon.ini`, and use `Path(task.get_scratch_dir())` to read task outputs instead of shell `find/cat` commands (see ?? and Appendix E for the complete guide).

This chapter, and the two that follow it, present Lessons 1 through 7 of the project’s tutorial sequence in full, with the discursive expansion proper to a textbook. Each lesson retains its original pedagogical scaffold — objective, concept, code, expected behavior, verification, scientific practice note — inside a numbered *Lesson box*, and is followed by additional theoretical commentary connecting it to the architecture developed in Part I.

Lesson 1 — Create a Local Workflow

Objective. Create and run the smallest useful DAGonStar workflow.

Concept. A Workflow owns tasks. A DagonTask factory creates task instances. A local batch task executes a shell command on the same machine.

Code (`lesson_01.py`):

```
1 from dagon import Workflow
2 from dagon.task import DagonTask, TaskType
3
4
5 if __name__ == "__main__":
6     workflow = Workflow("Lesson01")
7     task = DagonTask(TaskType.BATCH, "hello", "echo 'hello from
8         DAGonStar' > hello.txt")
9
10    workflow.add_task(task)
11    workflow.make_dependencies()
12    workflow.run()
```

Run:

```
1 cp dagon.ini.sample dagon.ini
2 python lesson_01.py
```

Expected behavior. DAGonStar creates a scratch directory under the configured `scratch_dir_base`, executes the task, and writes `hello.txt` in that task’s directory.

Verify:

```
1 find /tmp -maxdepth 1 -type d -name '*-hello' | tail -n 1
2 ls /tmp/*-hello
```

Scientific practice note. Even a one-task workflow is useful when the task has scientific meaning, such as downloading observations or generating a reproducible input dataset.

6.1 Discussion: The Smallest Useful Workflow

Lesson 1 is deliberately trivial, but it already exercises the complete execution lifecycle of `Workflow`: a `Workflow` is constructed, a single `BATCH` task is added, `make_dependencies()` parses its (dependency-free) command, `Validate_WF()` trivially accepts the single-node graph, and `run()` starts, executes, and completes the one task thread. The pedagogical purpose of beginning here — rather than with a multi-task DAG — is to let the scratch-directory and `.dagon/` mechanism (formalized in `Workflow`) be observed in isolation, before dependency resolution is introduced as a second source of complexity.

Lesson 2 — Add Explicit Task Dependencies

Objective. Create a workflow where tasks execute in a controlled order.

Concept. Use `consumer.add_dependency_to(producer)` when one task must wait for another.

Code (`lesson_02.py`):

```
1 from dagon import Workflow
2 from dagon.task import DagonTask, TaskType
3
4
5 if __name__ == "__main__":
6     workflow = Workflow("Lesson02")
7
8     prepare = DagonTask(TaskType.BATCH, "prepare", "echo prepared >
9         prepared.txt")
10    simulate = DagonTask(TaskType.BATCH, "simulate", "echo simulated >
11        simulated.txt")
12    summarize = DagonTask(TaskType.BATCH, "summarize", "echo summarized
13        > summarized.txt")
14
15    workflow.add_task(prepare)
16    workflow.add_task(simulate)
17    workflow.add_task(summarize)
18
19    simulate.add_dependency_to(prepare)
20    summarize.add_dependency_to(simulate)
21
22    workflow.Validate_WF()
23    workflow.run()
```

Run:

```
1 python lesson_02.py
```

Expected behavior. The three tasks run in the order: `prepare` → `simulate` → `summarize`.

Verify:

```
1 find /tmp -maxdepth 1 -type d \( -name '*-prepare' -o -name '*-simulate'
   -o -name '*-summarize' \)
```

Scientific practice note. Explicit dependencies are best when the relation is conceptual or procedural, for example: “validate parameters before launching the model.”

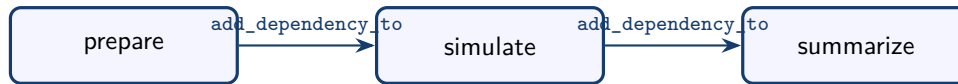


Figure 6.1: The strictly linear, explicitly-declared dependency chain of Lesson 2. No file is referenced between tasks; the ordering encodes a procedural, not a data, relationship.

6.2 Discussion: Explicit versus Inferred Ordering

Lesson 2 isolates the *explicit* half of the task-oriented/data-oriented distinction introduced in ???. Note carefully what this example does *not* do: none of the three tasks reads a file produced by another. The ordering `prepare` → `simulate` → `summarize` exists purely because the workflow author asserted it with `add_dependency_to()`. This is the correct modeling choice whenever the real-world constraint is procedural rather than a literal data dependency — the canonical example given in the documentation is “*validate parameters before launching the model*”: there may be no file the validation step hands to the model step, yet the ordering is still scientifically mandatory.

The call to `workflow.Validate_WF()` immediately after building the dependency graph, but before `run()`, is good practice independent of whether dependencies are explicit or inferred: it surfaces a cycle (??) as an explicit, early modeling error rather than as a runtime deadlock.

6.3 Mixing Explicit and Inferred Dependencies in One Workflow

Lessons 1 and 2 are presented separately for pedagogical clarity, but it is worth being explicit, before moving on, that nothing in the engine restricts a single workflow to using only one dependency-expression style. A realistic pipeline routinely mixes both: a `validate_parameters` task that gates `run_model` through an explicit `add_dependency_to()` call (because the relationship is procedural, as Lesson 2 demonstrates), and a `run_model` task that simultaneously consumes `prepare_input`’s output file through a `workflow://` reference (because that relationship is a genuine data dependency, as Lesson 3 will demonstrate in the next chapter).

Example 6.1. Consider a workflow with four tasks: `check_license` (verifies that a required model license file is present and current — a procedural precondition with no file product consumed by `run_model`); `prepare_input` (writes `forcing.nc`); `run_model` (consumes `forcing.nc` via `workflow://` and must wait for `check_license`, expressed via `add_dependency_to()`); and `archive` (consumes `run_model`’s output via `workflow://`). The resulting dependency graph has three edges of two different kinds: one explicit (`check_license` → `run_model`) and two inferred (`prepare_input` → `run_model`, `run_model` → `archive`). Both kinds are recorded identically in `prevs/nexts` (??) once `make_dependencies()` has run, and `Validate_WF()` treats the resulting four-node graph as a single, uniform DAG regardless of how each edge was created.

Scientific Practice Note

A useful authoring discipline, consistent with the modeling guidance of ??: when writing a new task, first ask whether the task *reads a file* that another task produces. If so, express that dependency through `workflow://` and let it be inferred — doing so automatically keeps the declared dependency synchronized with the actual data the command consumes, as ?? discusses at length. Reserve explicit `add_dependency_to()` calls for the residual cases where no file changes hands, but an ordering constraint is nonetheless real.

6.4 Validating the Graph Without Running It

Lesson 2’s use of `Validate_WF()` before `run()` previews a pattern this book will return to repeatedly in Part II, under the name *local structural verification* (??): separating the act of *constructing and validating* a dependency graph from the act of *executing* it. This separation has a concrete pedagogical and debugging benefit even at this early stage of the tutorial sequence. A workflow author uncertain whether a hand-edited set of `add_dependency_to()` calls has accidentally introduced a cycle can call `make_dependencies()` followed by `Validate_WF()` alone, inspect the raised exception (or its absence) and the resulting `prevs/nexts` lists on each task object, and only then decide whether to proceed to `run()` — without incurring the cost, or the side effects, of actually executing any task’s command. For workflows whose tasks have expensive or non-idempotent side effects (a remote job submission, a billable cloud operation, an API call with rate limits), this validate-before-run discipline is not merely convenient but essential.

Exercise 6.1. Rewrite Lesson 2 so that `simulate` actually consumes the file written by `prepare` (rename `prepared.txt` to a path inside an `output/` subdirectory), and replace the explicit `add_dependency_to()` calls with `workflow://` references. Confirm that `workflow.make_dependencies()` reconstructs the same linear order automatically. This exercise previews ??.

Exercise 6.2. Extend the four-task example of this section’s worked example into a runnable DAGonStar script. Verify, using `task.prevs/task.nexts` inspection as suggested above, that the engine records exactly three dependency edges, and confirm that `Validate_WF()` accepts the graph without raising an exception.

Chapter 7

Data Dependencies, Scratch Directories, and Generated Launchers

Lesson 3 — Use workflow:// Data Dependencies

Objective. Let DAGonStar infer dependencies from data references.

Concept. `workflow:///task/path` references a file inside another task's scratch directory. `Workflow.make_dependencies()` parses this reference and creates the dependency.

Code (`lesson_03.py`):

```
1 from dagon import Workflow
2 from dagon.task import DagonTask, TaskType
3
4
5 if __name__ == "__main__":
6     workflow = Workflow("Lesson03")
7
8     observations = DagonTask(
9         TaskType.BATCH,
10        "observations",
11        "mkdir -p data; printf 'time,value\n0,12.5\n1,13.1\n' > data/
12        series.csv",
13    )
14
15    statistics = DagonTask(
16        TaskType.BATCH,
17        "statistics",
18        "wc -l workflow:///observations/data/series.csv > line_count.
19        txt",
20    )
21
22    workflow.add_task(observations)
23    workflow.add_task(statistics)
24    workflow.make_dependencies()
25    workflow.run()
```

Run:

```
1 python lesson_03.py
```

Expected behavior. `statistics` waits for `observations`. DAGonStar stages `series.csv` into the consumer task and rewrites the command to use the local staged path.

Verify:

```
1 find /tmp -maxdepth 1 -type d -name '*-statistics' | tail -n 1
```

Inspect `line_count.txt` in the resulting directory.

Scientific practice note. Dataflow references make scientific provenance easier to read: an edge exists because a named data product is consumed.

7.1 Discussion: Inference versus Assertion

Lesson 3 is the data-oriented twin of Lesson 2, and the contrast is instructive when read side by side. In Lesson 2, a reader of the source code must already know, extrinsically, that “validate parameters” precedes “launch the model.” In Lesson 3, the dependency is *visible inside the command string itself* — `workflow:///observations/data/series.csv` simultaneously documents *and* enforces the relationship. This is precisely the property the documentation calls easier-to-read scientific provenance: the graph edge exists *because* a named data product is consumed, not by separate assertion.

Lesson 4 — Inspect Scratch Directories and Generated Launchers

Objective. Understand what DAGonStar writes during execution.

Concept. Each task has a working directory and an internal `.dagon` directory containing generated scripts and captured output.

Code (`lesson_04.py`, or reuse Lesson 3):

```
1 from dagon import Workflow
2 from dagon.task import DagonTask, TaskType
3
4
5 if __name__ == "__main__":
6     workflow = Workflow("Lesson04")
7     task = DagonTask(TaskType.BATCH, "inspect", "pwd; echo product >
8         product.txt")
9     workflow.add_task(task)
10    workflow.make_dependencies()
11    workflow.run()
```

Run:

```
1 python lesson_04.py
```

Verify:

```
1 DIR=$(find /tmp -maxdepth 1 -type d -name '*-inspect' | tail -n 1)
2 ls "$DIR"
3 ls "$DIR/.dagon"
4 sed -n '1,160p' "$DIR/.dagon/launcher.sh"
5 sed -n '1,160p' "$DIR/.dagon/stdout.txt"
```

Expected behavior. The reader should observe: task output files in the task directory; `.dagon/launcher.sh`; `.dagon/context.sh`; `.dagon/stdout.txt`.

Scientific practice note. Generated launchers are a practical provenance artifact. They show the actual command DAGonStar executed after preprocessing.

7.2 Anatomy of a Task Working Directory

?? formalizes the layout that Lesson 4 asks the reader to discover by inspection. This layout recurs, with backend-specific additions, across every execution backend treated in Part II.

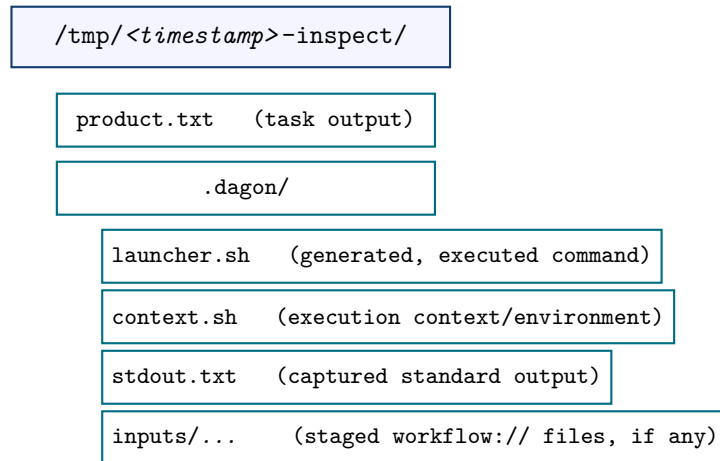


Figure 7.1: Canonical layout of a task’s scratch (working) directory, as discoverable through Lesson 4’s verification commands.

Scientific Practice Note

Treat `.dagon/launcher.sh` as the closest thing the engine offers to a laboratory notebook entry for a single task: it is the literal, fully resolved command that ran, *after* every `workflow://` reference has been rewritten to a concrete staged path. When debugging a failed scientific task, read this file before re-reading the Python source — it removes one layer of indirection between “what I wrote” and “what actually executed.”

7.3 From Inspection to Debugging Practice

Lesson 4’s verification procedure — listing the task directory, then reading `launcher.sh`, `context.sh`, and `stdout.txt` in sequence — is presented in the tutorial as a one-time exercise in familiarization. It is worth elevating it explicitly to a general-purpose debugging procedure, because the same three files, read in the same order, are the correct first response to *any* unexpected task failure encountered later in this book, including the heterogeneous backends of Part II.

1. **Read `launcher.sh` first.** This file contains the fully resolved command, after every `workflow://` reference has been rewritten to a concrete, staged path (??). The most common class of early failure — a referenced file that does not exist, or a path that resolved to somewhere unexpected — is usually visible immediately at this step, before any need to inspect output at all.
2. **Read `context.sh` second.** This file captures the execution context, including environment-related setup the launcher relies on; discrepancies between the environment a task actually saw and the environment a developer assumed are a frequent source of “works on my machine” failures, particularly once ??’s environment-modules and virtual-environment activation patterns are involved.

3. **Read `stdout.txt` last.** Only once the command and its execution context are confirmed correct does the captured standard output — which records what the external program itself reported — become the most informative artifact.

Scientific Practice Note

This ordered procedure exists because each file answers a strictly more specific question than the one before it: `launcher.sh` answers “what command, exactly, did DAGonStar decide to run?”; `context.sh` answers “in what environment did it decide to run that command?”; and `stdout.txt` answers “what did the command itself report once it ran?” Skipping directly to `stdout.txt` — the most tempting file to open first, since it most resembles ordinary terminal output — frequently wastes time chasing a symptom (a program’s own error message) whose root cause (a wrong path, a missing dependency, an unset environment variable) was already visible one file earlier.

7.4 Output Inspection in Notebook Environments

The inspection commands developed in this chapter (`find`, `cat`, `ls`) are the natural interface for a terminal-based tutorial. In Jupyter Notebook and Google Colab environments, however, shell commands in code cells are less idiomatic than Python-native `pathlib.Path` operations, and mixing shell and Python in the same notebook cell can make provenance harder to trace. DAGonStar provides a dedicated method, `task.get_scratch_dir()`, which returns the absolute path to a task’s scratch directory as a Python string, enabling fully native inspection:

```

1 from pathlib import Path
2
3 # After workflow.run() or workflow.wait():
4 task_dir = Path(task.get_scratch_dir())
5
6 # List all outputs produced by this task
7 print(list(task_dir.iterdir()))
8
9 # Read a specific output file
10 print((task_dir / "result.txt").read_text(encoding="utf-8"))
11
12 # Read the generated launcher (the provenance record)
13 print((task_dir / ".dagon" / "launcher.sh").read_text())
14
15 # Read captured standard output
16 print((task_dir / ".dagon" / "stdout.txt").read_text())

```

Listing 7.1: Python-native task output inspection in a Jupyter or Colab notebook cell.

This pattern is equivalent, in every meaningful respect, to the three-command shell procedure (`cat launcher.sh`; `cat context.sh`; `cat stdout.txt`) developed in ??: the same order of reading — `launcher.sh` first, `stdout.txt` last — applies, and the same principle governs the interpretation of results. The only difference is the interface: `pathlib.Path` instead of shell commands.

`task.get_scratch_dir()` is particularly important in Google Colab, where the runtime filesystem (`/content/`) does not expose shell globbing across sessions, and where the working directory may differ from what a terminal-based tutorial assumes. See ?? and Appendix E for the complete notebook integration context.

7.5 The Provenance Value of an Immutable Launcher

A subtlety worth making explicit, building on ??’s discussion of command rewriting: `launcher.sh` is generated *once*, before the task executes, and is not regenerated or modified afterward. This immutability is what makes it trustworthy as a provenance artifact in the sense already developed in ??’s discussion of the provenance graph. A researcher inspecting a scratch directory days or months after a workflow run can be confident that `launcher.sh` reflects exactly the command that executed at the time of the run — not a reconstruction, not an approximation, and not a file that some later process might have silently altered.

Contrast this with the alternative of relying solely on the original Python workflow source to reconstruct what ran. The Python source records the *template* a command was built from — including any unresolved `workflow://` references — but not the runtime values that template was eventually instantiated with, nor whether the workflow was later modified and rerun with different parameters. Two facts can only be recovered from `launcher.sh` itself: which specific staged path each reference resolved to at execution time, and the precise, final form of any command-line argument that depended on a runtime value. This is the concrete sense in which the generated launcher, not the source script, is the authoritative record of what a given scratch directory actually represents.

Exercise 7.1. Run Lesson 3’s workflow twice in succession, without deleting the resulting scratch directories between runs. Compare the two generated `statistics` task directories’ `launcher.sh` files. What differs between the two, and what does this tell you about which parts of a generated launcher are stable across repeated runs of an unmodified workflow, versus which parts (if any) vary?

7.6 Chapter Summary

Lessons 3 and 4 together complete the data-oriented half of dependency modeling and expose the concrete, on-disk artifacts the engine produces while enacting it: a staged input file under `.dagon/inputs/`, a rewritten `launcher.sh`, and a captured `stdout.txt`. This chapter additionally elevated the three-file inspection procedure of Lesson 4 into a general debugging discipline applicable throughout the rest of this book, and argued for the generated launcher’s immutability as the structural reason it functions as an authoritative provenance record, independent of and complementary to the original workflow source code. ?? turns to the failure and recovery mechanisms layered on top of this same on-disk model: cycle detection, checkpointing, and configurable data staging.

Chapter 8

Cycle Detection, Checkpointing, and Data Staging Modes

Lesson 5 — Detect and Fix Dependency Cycles

Objective. Learn how DAGonStar rejects invalid workflow graphs.

Concept. A DAG cannot contain cycles. If A depends on B and B depends on A, no valid execution order exists.

Code (lesson_05_cycle.py):

```
1 from dagon import Workflow
2 from dagon.task import DagonTask, TaskType
3
4
5 workflow = Workflow("Lesson05")
6
7 a = DagonTask(TaskType.BATCH, "A", "echo A")
8 b = DagonTask(TaskType.BATCH, "B", "echo B")
9
10 workflow.add_task(a)
11 workflow.add_task(b)
12
13 b.add_dependency_to(a)
14 a.add_dependency_to(b)
15
16 workflow.Validate_WF()
```

Run:

```
1 python lesson_05_cycle.py
```

Expected behavior. The script raises an exception reporting a cycle.

Fix. Remove one edge (keep only `b.add_dependency_to(a)`), then:

```
1 workflow.Validate_WF()
2 workflow.run()
```

Scientific practice note. Cycle detection is not only a software constraint. It also catches ambiguous scientific logic, such as attempting to calibrate a model using results that the same model has not produced yet.

8.1 Discussion: Cycles as Scientific Modeling Errors

?? gave the graph-theoretic definition of acyclicity; Lesson 5 demonstrates the engine enforcing it operationally, via `Validate_WF()`. The scientific practice note deserves emphasis because it reframes a software constraint as a *modeling* constraint: a cycle in a workflow DAG is frequently the symptom of a circular scientific argument — calibrating a model on its own not-yet-produced output, or post-processing a product that the post-processing step itself is meant to generate. Treat a cycle-validation failure as an invitation to re-examine the experimental design, not merely a bug to silence.

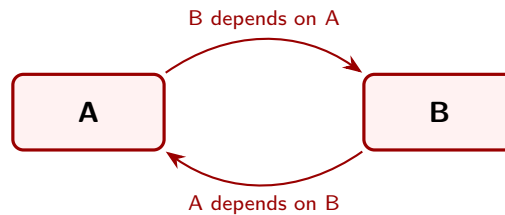


Figure 8.1: An invalid two-node cycle: neither task can legally run first. `Validate_WF()` rejects this graph before execution begins.

Lesson 6 — Use Checkpoint Files

Objective. Record task completion metadata and resume a workflow.

Concept. `Workflow(checkpoint_file=...)` writes checkpoint metadata after execution. `workflow.run(resume_checkpoint_file=...)` can reuse completed task directories when the checkpoint records success and the working directory still exists.

Code (`lesson_06.py`):

```

1  from dagon import Workflow
2  from dagon.task import DagonTask, TaskType
3
4
5  def build_workflow():
6      workflow = Workflow("Lesson06", checkpoint_file="
7          lesson_06_checkpoint.json")
8      slow = DagonTask(TaskType.BATCH, "slow", "echo computed > result.
9          txt")
10     workflow.add_task(slow)
11     workflow.make_dependencies()
12     return workflow
13
14 if __name__ == "__main__":
15     workflow = build_workflow()
16     workflow.run(resume_checkpoint_file="lesson_06_checkpoint.json")

```

Run twice:

```

1  python lesson_06.py
2  python lesson_06.py

```

Verify:

```

1  sed -n '1,200p' lesson_06_checkpoint.json

```

Expected behavior. After the first run, the checkpoint file contains the task working directory and exit code. On a later run, DAGOnStar can skip a task already marked

successful if the recorded working directory still exists.

Scientific practice note. Checkpointing is most valuable for expensive simulations. Preserve checkpoint files together with enough metadata to know which code and configuration produced them.

8.2 Discussion: Checkpoint Semantics and Their Limits

The checkpoint file written by Lesson 6 has the shape:

```

1 {
2   "_scratch_dir": "/tmp/",
3   "Experiment.A": {
4     "working_dir": "/tmp/1710000000000-A",
5     "workflow": "Experiment",
6     "name": "A",
7     "code": 0
8   }
9 }
```

Listing 8.1: Representative checkpoint JSON shape.

Checkpoint keys are built from `<workflow-name>.<task-name>`; stable naming is therefore essential, and renaming a task is, from the checkpoint system’s perspective, indistinguishable from introducing a new task. Checkpoint reuse additionally requires that the *recorded working directory still exist* on disk — a checkpoint entry alone cannot recover output that scratch-directory garbage collection or manual cleanup has already removed.

Caution

Checkpoints record task *success* and a working-directory path, not full semantic equivalence of code, inputs, or parameters. If a task’s command changes but its name and previous success remain recorded, checkpoint reuse may incorrectly skip recomputation. For rigorous scientific reproducibility, store code version, configuration, input checksums, and software environment *alongside* the checkpoint file, not only inside it.

A complete reference treatment of checkpoint semantics — including remote, Docker, and cloud checkpointing caveats — is reproduced in Appendix D.

8.2.1 Checkpointing in Jupyter Notebook and Google Colab

The persistence constraint that makes `/tmp/` unsuitable for checkpointed pipelines becomes significantly more pressing in notebook environments, and most acute in Google Colab, where the entire runtime filesystem (`/content/`) is discarded when the runtime is recycled or disconnected. A workflow that wrote its checkpoint to `/content/checkpoint.json` and its scratch directories to `/tmp/` cannot reuse either on resumption after a reset.

The recommended mitigation is to mount Google Drive and redirect both the `scratch_dir_base` and the checkpoint file to a durable Drive path:

```

1 from google.colab import drive
2 drive.mount('/content/drive')
3
4 WORKDIR = "/content/drive/MyDrive/dagonstar-experiments"
5
6 config = {
7     "batch": {
8         "scratch_dir_base": WORKDIR,
```

```

9         "run_base": "",
10        "remove_dir": "False",
11    },
12    "dagon_service": {"use": "False"},
13    "ftp_pub": {"ip": "localhost", "user": "anonymous", "password": ""},
14 }
15
16 workflow = Workflow("MyExperiment", config=config,
17                     checkpoint_file=WORKDIR + "/checkpoint.json")
18 workflow.run(resume_checkpoint_file=WORKDIR + "/checkpoint.json")

```

Listing 8.2: Checkpointing with Google Drive persistence in Colab.

In a local Jupyter environment, set `scratch_dir_base` to a project directory that survives kernel restarts (typically a path under the project root, not under `/tmp/`). The checkpoint semantics — key structure, reuse conditions, naming stability requirements — are identical to those described above for script-based workflows; only the storage target changes. See ?? and Appendix E for the complete notebook integration context.

Lesson 7 — Choose Data Staging Modes

Objective. Control how DAGOnStar stages files between tasks.

Concept. The workflow default data mover is set with `workflow.set_data_mover(...)`. Local workflows commonly use `DataMover.COPY` or `DataMover.LINK`.

Code (`lesson_07.py`):

```

1 from dagon import DataMover, Workflow
2 from dagon.task import DagonTask, TaskType
3
4
5 if __name__ == "__main__":
6     workflow = Workflow("Lesson07")
7     workflow.set_data_mover(DataMover.COPY)
8
9     source = DagonTask(TaskType.BATCH, "source", "mkdir -p out; echo
10        sample > out/data.txt")
11    sink = DagonTask(TaskType.BATCH, "sink", "cat workflow:///source/
12        out/data.txt > copied.txt")
13
14    workflow.add_task(source)
15    workflow.add_task(sink)
16    workflow.make_dependencies()
17    workflow.run()

```

Run:

```
1 python lesson_07.py
```

Verify:

```

1 DIR=$(find /tmp -maxdepth 1 -type d -name '*-sink' | tail -n 1)
2 sed -n '1,80p' "$DIR/copied.txt"

```

Expected behavior. The sink task receives a copy of the source file and writes `copied.txt`.

Experiment. Change to `workflow.set_data_mover(DataMover.LINK)`, run again, and inspect the generated launcher script to observe the staging command difference.

Scientific practice note. `COPY` is often better for archival reproducibility because downstream tasks get their own materialized inputs. `LINK` can reduce storage and I/O cost when intermediate files remain stable.

8.3 Discussion: Choosing a Staging Policy

The choice between `COPY` and `LINK` is a genuine engineering trade-off, not a stylistic preference, and it should be made deliberately per workflow (or even per task, since `set_data_mover` may also be invoked at task scope):

- **`COPY`** materializes an independent file in the consumer’s scratch directory. This maximizes archival reproducibility — a consumer task directory, once created, is self-contained even if the producer’s directory is later cleaned up — at the cost of doubled storage and I/O for large scientific products (e.g. multi-gigabyte NetCDF restart files).
- **`LINK`** creates a symbolic link rather than a copy. This is appropriate when intermediate files are large, numerous, and known to remain stable for the lifetime of the workflow run, but it *breaks* the self-containment property: deleting the producer’s scratch directory invalidates every consumer that linked to it.

Scientific Practice Note

A useful operational rule: use `COPY` as the default for any workflow whose outputs will be archived or published, and consider `LINK` only for large, purely intermediate products that are deleted at the end of a single run and never separately archived.

8.4 Cycle Detection as an Instance of a General Algorithmic Problem

?? already established the precise mathematical relationship between acyclicity and the existence of a valid execution order; it is worth, in this chapter, connecting that result to the concrete *algorithmic* problem `Validate_WF()` must solve, since the choice of algorithm has direct consequences for how the engine scales to large, realistic scientific workflows.

Cycle detection in a directed graph is classically solved by depth-first search, tracking, for each node, whether it is unvisited, currently on the recursion stack (“in progress”), or fully processed (“done”). A cycle is detected precisely when the search encounters an edge leading to a node already marked “in progress” — that is, a node reachable from itself through the edges currently being traversed. This algorithm runs in time linear in the number of nodes plus edges, $O(|V| + |E|)$, which matters in practice: a workflow with hundreds or thousands of tasks — not unrealistic for an ensemble of environmental simulations, each itself decomposed according to the granularity discussion of ?? — can still be validated in time proportional to its own size, rather than, say, quadratic time that would make validation itself a bottleneck on large DAGs.

Scientific Practice Note

The practical lesson for a workflow author is reassuring rather than cautionary: because cycle detection is cheap relative to the cost of running even a single scientific task, there is essentially no performance reason to ever omit a `Validate_WF()` call, or to avoid restructuring a large workflow’s dependencies during development for fear of expensive revalidation. The discipline recommended throughout this book — validate before running, especially for workflows with expensive or non-idempotent tasks (??) — is essentially free in computational terms.

8.5 Checkpointing in the Broader Context of Fault-Tolerant Computing

Checkpointing is a technique with a long history in fault-tolerant and high-performance computing, predating workflow engines entirely: long-running parallel simulations have, for decades, periodically written their internal state to disk precisely so that a hardware failure partway through a multi-day run need not force a complete restart from the beginning. It is useful to situate DAGonStar’s task-level checkpointing within this broader tradition, both to clarify what it does provide and to be precise about what it does not.

Classical *application-level* checkpointing, of the kind a simulation code implements internally, captures fine-grained numerical state — the full field values of a partially completed timestep, for instance — and typically requires the application itself to be written with checkpoint support in mind. DAGonStar’s checkpointing, by contrast, operates entirely at the *task* granularity: it records whether a task, treated as an opaque unit, completed successfully, and whether its output directory still exists. This is a coarser, but considerably more general, form of checkpointing — it requires no cooperation from the external program being wrapped, applies uniformly to a shell command, a Slurm job, a Docker container, or an LLM completion alike, and composes naturally with the granularity choices a workflow author makes when decomposing a pipeline into tasks (??).

This coarseness has a direct, quantifiable consequence for how a workflow should be decomposed if checkpoint-driven resumability is a priority for a given pipeline: the *finer* the task granularity, the more checkpoint boundaries exist, and the less recomputation a resumed run must perform after a failure. A single, coarse-grained task that internally runs an entire multi-hour simulation chain provides exactly one checkpoint boundary — success or failure of the whole chain — whereas the same chain decomposed into five sequential tasks, each individually checkpointed, allows a resumed run to skip however many of the five had already succeeded before the failure occurred. This is a second, independent argument — alongside the traceability argument already given in ?? — for preferring finer-grained task decomposition in workflows expected to run long enough that mid-run failure is a realistic possibility.

8.6 Staging Cost as a Function of Data Volume

??’s discussion of COPY versus LINK staging modes can be made more quantitative by considering explicitly how each mode’s cost scales with the size of the data product being staged.

COPY’s cost — in both wall-clock time and storage — scales linearly with file size: copying a ten-gigabyte NetCDF restart file takes meaningfully longer, and consumes meaningfully more disk space, than copying a ten-kilobyte CSV summary. LINK’s cost, by contrast, is essentially constant regardless of the referenced file’s size, since a symbolic link is a small filesystem object containing only a path string, independent of the size of the file it points to. For the small, text-oriented examples used throughout Lessons 1–7 of this book, this distinction is invisible — a kilobyte-scale CSV copies essentially instantly either way — but it becomes the dominant practical consideration the moment a workflow scales to the gigabyte-scale data products characteristic of the environmental modeling case study of ??: a multi-stage WRF pipeline that copies, rather than links, every intermediate NetCDF file between `geogrid`, `ungrib`, `metgrid`, and `real` could plausibly spend a non-trivial fraction of total wall-clock time on staging alone, an overhead LINK would largely eliminate at the cost of the archival self-containment ?? already discussed.

Exercise 8.1. For the three-stage `download` → `preprocess` → `simulate` checkpoint example discussed earlier in this chapter, sketch (in prose, no code required) how the choice between COPY and LINK for the `download` → `preprocess` edge would interact with checkpoint reuse

after a `simulate` failure, under the assumption that `download`'s output is a forty-gigabyte satellite-observation file. Which staging mode would you recommend for this specific edge, and why?

8.7 Chapter Summary

Part I closes with the three mechanisms that make DAGonStar workflows robust in practice, not merely correct on paper: `Validate_WF()` rejects structurally invalid (cyclic) graphs before execution, using a classical, linear-time graph algorithm that imposes essentially no practical performance cost; checkpoint files allow expensive workflows to resume past completed work, with explicitly documented limits on what “completed” guarantees, situated here within the broader, decades-old tradition of fault-tolerant checkpointing in high-performance computing; and the `DataMover` abstraction lets a workflow author trade storage/I-O cost against archival self-containment, per dependency edge if needed, a trade-off whose practical weight scales directly with the size of the scientific data products involved. Part II now extends this same dependency, staging, and lifecycle infrastructure across containers, remote HPC schedulers, meta-workflows, and — centrally for this book's stated subject — AI and web services.

Part II

Heterogeneous Execution Backends and AI–HPC Integration

Chapter 9

Containers, Remote Execution, and HPC Schedulers

Running this chapter in Google Colab

All lessons in this chapter run in Google Colab with Option 3 installation (`git clone` then `pip install -e .`, see Appendix E). Use `local_config(workdir)` instead of `dagon.ini`, and use `Path(task.get_scratch_dir())` to read task outputs instead of shell `find/cat` commands (see ?? and Appendix E for the complete guide).

Caution

Colab note for Docker and Slurm lessons. Docker tasks (`TaskType.DOCKER`) require a Docker daemon and cannot run in hosted Colab. Slurm tasks require a cluster connection. Both lessons remain **fully runnable in Colab in structural-verification mode**: construct the workflow, call `workflow.make_dependencies()`, and call `task.generate_command('launcher.sh')` or run with `workflow.set_dry(True)` to confirm the graph without contacting any infrastructure. See also `portable_emulation=True` (??).

Part II extends the foundations of Part I across the engine's heterogeneous execution backends. This chapter covers Lessons 8 and 9: Docker-backed execution, remote SSH execution, and HPC job-scheduler (Slurm) execution — the three backends most directly associated with classical HPC practice.

Lesson 8 — Run Docker-Backed Tasks

Objective. Run a task inside a Docker container.

Concept. `TaskType.DOCKER` creates a `DockerTask`. The task uses the Docker Python SDK, pulls or uses an image, starts a container, and runs the task command with `docker exec`.

Prerequisites. Docker installed; Docker daemon running; current user allowed to access Docker. Verify with `docker info`.

Code (`lesson_08.py`):

```
1 from dagon import Workflow
2 from dagon.task import DagonTask, TaskType
3
4
5 if __name__ == "__main__":
```

```

6     workflow = Workflow("Lesson08")
7
8     task = DagonTask(
9         TaskType.DOCKER,
10        "container_python",
11        "python --version > python_version.txt",
12        image="python:3.12-slim",
13    )
14
15    workflow.add_task(task)
16    workflow.make_dependencies()
17    workflow.run()

```

Verify:

```

1 DIR=$(find /tmp -maxdepth 1 -type d -name '*-container_python' | tail -
   n 1)
2 sed -n '1,80p' "$DIR/python_version.txt"

```

Expected behavior. The output file contains the Python version reported inside the container.

Scientific practice note. Containers help preserve software environments. For published workflows, record the exact image tag or digest used for scientific results.

9.1 Discussion: Containers as Environment Provenance

Lesson 8 illustrates that, architecturally, a `DockerTask` differs from a `Batch` task only in *where* the launcher executes — inside a container via `docker exec` rather than directly via `subprocess.Popen`. Every dependency, staging, and checkpoint mechanism developed in Part I applies unchanged. The scientific value is environment provenance: `image="python:3.12-slim"` is, in a published workflow, as much a part of the reproducible record as the command string itself, and should be pinned to a digest rather than a mutable tag whenever a result is meant to be exactly reproducible.

Lesson 9 — Prepare Remote and Slurm Workflows

Objective. Understand how to construct remote and Slurm tasks, and how to verify the environment before running them.

Concept. Remote task variants are selected by constructor arguments such as `ip`. Slurm tasks use `TaskType.SLURM` and generate `sbatch` commands.

Remote batch task:

```

1 from dagon import Workflow
2 from dagon.task import DagonTask, TaskType
3
4
5 workflow = Workflow("RemoteExample")
6
7 remote = DagonTask(
8     TaskType.BATCH,
9     "remote_hostname",
10    "hostname > host.txt",
11    ip="example.org",
12    ssh_username="researcher",
13    keypath="/home/researcher/.ssh/id_rsa",
14 )

```

```

15
16 workflow.add_task(remote)
17 workflow.make_dependencies()
18 workflow.run()

```

Before running, verify SSH manually:

```

1 ssh -i /home/researcher/.ssh/id_rsa researcher@example.org hostname

```

Slurm task:

```

1 from dagon import Workflow
2 from dagon.task import DagonTask, TaskType
3
4
5 workflow = Workflow("SlurmExample")
6
7 task = DagonTask(
8     TaskType.SLURM,
9     "slurm_hostname",
10    "hostname > host.txt",
11    partition="compute",
12    ntasks=1,
13    time="00:05:00",
14 )
15
16 workflow.add_task(task)
17 workflow.make_dependencies()
18 workflow.run()

```

Before running, verify Slurm:

```

1 which sbatch
2 sinfo

```

Local structural verification (no scheduler required):

```

1 from dagon.task import DagonTask, TaskType
2
3 task = DagonTask(TaskType.SLURM, "check", "echo check", partition="
4     debug", ntasks=1, working_dir="scratch")
5 print(type(task).__name__)
6 print(task.generate_command("launcher.sh"))

```

Scientific practice note. Remote and HPC workflows are reproducible only when resource requests, environment modules, host configuration, and input data locations are recorded.

9.2 Discussion: The Factory Pattern for Remote Variants

Lesson 9 makes explicit a design decision already anticipated in ??: *remote* execution is not a separate `TaskType`, but a keyword-argument-triggered specialization of an existing one. Passing `ip`, `ssh_username`, and optionally `keypath` to a `TaskType.BATCH` construction causes `Batch.__new__` to return a `RemoteBatch` instance instead (??); the same pattern extends to remote Slurm and remote Docker. This keeps the DAG-authoring surface uniform — a workflow author reasons about *tasks and their dependencies*, not about a proliferating taxonomy of remote/local task classes.

Table 9.1: Backend selection by scientific software need (after the project’s own guidance).

Need	Typical backend
Small local utility	BATCH
Compiled HPC model	SLURM
Dependency-heavy tool	DOCKER
Remote instrument/service host	remote BATCH
Elastic compute	CLOUD

9.3 The Local-Structural-Verification Pattern

A recurring theme across the remote, Slurm, and (later) meta-workflow lessons is that a workshop laptop or CI environment frequently lacks the external service (a reachable SSH host, a live Slurm controller) needed for full execution. Lesson 9’s structural-verification snippet — constructing the task object and calling `generate_command()` without ever calling `workflow.run()` — is the project’s recommended substitute: it exercises the *command-generation* logic exhaustively while requiring no external service at all, and is the pattern this book recommends whenever a reader wants to validate a Slurm or remote task specification before deploying it to a real cluster.

9.4 Containers and the Reproducibility Problem, Revisited

??’s discussion of Lesson 8 already noted that container image identifiers are part of a workflow’s scientific record. This deserves expansion, because the distinction between a mutable *tag* and an immutable *digest* is one of the more consequential, and most frequently overlooked, reproducibility decisions a workflow author using DOCKER tasks will make.

A tag such as `python:3.12-slim` is a mutable pointer: the maintainers of that image may, and routinely do, push a new image under the same tag — to apply a security patch, update a base operating system, or bump a minor library version — without any change to the tag string itself. A workflow that pins only the tag is therefore not fully reproducible in the strict sense this book has used throughout: rerunning the identical workflow source code six months later may silently execute a different, updated image, with no error, warning, or visible change to the workflow definition. A *digest*, by contrast — a cryptographic hash of the image’s content, typically written as `image@sha256:<hex-digest>` — refers to one, and only ever one, immutable image content. Pinning to a digest sacrifices automatic access to upstream patches but gains the property that the phrase “the same container image” becomes mathematically, not merely conventionally, true across arbitrarily long timespans.

Scientific Practice Note

For workflows whose results are destined for publication, archival, or long-term operational reuse — the environmental forecasting deployments discussed in ?? being the clearest example — this book recommends recording, alongside the workflow source, both the human-readable tag (for legibility) and the resolved digest (for strict reproducibility) at the time a result was produced, even though the `DockerTask` constructor itself accepts only the `image` string and does not distinguish the two forms syntactically.

9.5 The Economics of Remote and Scheduled Execution

Lesson 9’s two backends — remote SSH execution and Slurm scheduling — share a structural property worth making explicit: both introduce a *queueing* or *latency* cost that purely local BATCH execution does not incur, and a workflow author reasoning about overall pipeline turnaround time should account for this cost as a first-class design consideration, not an incidental detail.

A local BATCH task’s total wall-clock cost is, to a first approximation, just the time the underlying command itself takes to run, plus the comparatively small overhead of scratch-directory creation and staging (??). A SLURM task’s total wall-clock cost, by contrast, is the sum of the command’s own runtime *and* the time the submitted job spends queued, waiting for the scheduler to allocate the requested resources — a quantity that depends on cluster-wide contention entirely outside the workflow’s own control, and that can, on a busy shared HPC system, dwarf the actual computational time by a large factor. A remote BATCH task incurs a smaller, but still non-zero, analogous cost: SSH connection establishment and, depending on network conditions, command round-trip latency.

Scientific Practice Note

This has a direct consequence for how the asynchronous launch mechanism developed in ?? should be read in anticipation: a workflow mixing local and Slurm-scheduled tasks benefits disproportionately from `launch()/wait()` non-blocking execution precisely *because* Slurm queueing time is unpredictable and potentially long — a controller blocked synchronously on `run()` during a multi-hour queue wait cannot do anything else useful in the meantime, whereas a controller using `launch()` remains free to handle other requests, report progress, or even submit additional, independent workflows, while the Slurm-scheduled portion of the DAG waits its turn.

9.6 A Worked Comparison Across the Backend Taxonomy

To consolidate ??’s qualitative guidance into a more concrete worked illustration, consider a single post-processing operation — regriding a model’s native output onto a regular latitude-longitude grid — and how the choice of backend would change the shape of the corresponding `DagonTask` construction, without changing the regriding logic itself.

As a local utility, assuming the regriding tool is already installed on the machine running the workflow controller, the task is simply:

```
DagonTask(TaskType.BATCH, "regrid", "regrid_tool --in workflow:///model/out.nc --out regridded.nc")
```

As a dependency-heavy tool requiring a specific, hard-to-install geospatial library stack, the same logical operation becomes a DOCKER task, trading installation complexity for container-image maintenance:

```
DagonTask(TaskType.DOCKER, "regrid", "regrid_tool --in workflow:///model/out.nc --out regridded.nc", image="lab/regrid-tools:2.3")
```

As a compute-intensive operation on a very large grid, justifying scheduler- managed resource allocation, it becomes a SLURM task, trading local immediacy for queued, resource-guaranteed execution:

```
DagonTask(TaskType.SLURM, "regrid", "regrid_tool --in workflow:///model/out.nc --out regridded.nc", partition="compute", ntasks=8, time="00:30:00")
```

Scientific Practice Note

Note what does *not* change across all three variants: the `workflow:///model/out.nc` dependency reference, and therefore the dependency edge it induces, the staging behavior that places the file in the task's working directory, and the task's name within the DAG. This invariance is the architectural payoff promised in ??’s discussion of the factory pattern (??): changing *where* a logical operation executes is, in the common case, a matter of changing the `TaskType` and a small number of backend- specific keyword arguments, not of redesigning the workflow’s dependency structure.

9.7 Chapter Summary

Docker, remote-batch, and Slurm execution share one architectural lesson: heterogeneity of *where* a command runs is isolated entirely within the concrete task subclass’s execution hook, while dependency resolution, staging, checkpointing, and the `workflow://` schema are identical to the purely local workflows of Part I. This chapter additionally examined the tag-versus-digest reproducibility distinction for container images, reasoned about the queueing and latency costs that distinguish remote and scheduler-backed execution from purely local execution, and worked through a single logical operation expressed across three different backends to make concrete the claim that backend choice is, architecturally, orthogonal to dependency-graph structure. ?? now turns to composing multiple workflows together and to controlling *when*, rather than merely *where*, execution happens.

Exercise 9.1. Using the worked regridding example above as a template, write the `TaskType.BATCH` *remote* variant of the same task (executing on a named remote host via SSH, as in Lesson 9), and explain, referencing ??, exactly which class `DagonTask` would instantiate for this fourth variant, and how that class relates to the plain local `Batch` class used in the first variant above.

Chapter 10

Meta-Workflows and Asynchronous Execution

Lesson 10 — Compose Meta-Workflows

Objective. Coordinate more than one workflow and use transversal references.

Concept. DAG_TPS coordinates a set of workflows. A task in one workflow can reference a task in another workflow with `workflow://OtherWorkflow/task/path`.

Code (lesson_10.py):

```
1 from dagon import Workflow
2 from dagon.dag_tps import DAG_TPS
3 from dagon.task import DagonTask, TaskType
4
5
6 if __name__ == "__main__":
7     observations = Workflow("Observations")
8     model = Workflow("Model")
9
10    collect = DagonTask(
11        TaskType.BATCH,
12        "collect",
13        "mkdir -p out; echo 'forcing' > out/forcing.txt",
14    )
15
16    run_model = DagonTask(
17        TaskType.BATCH,
18        "run_model",
19        "cat workflow://Observations/collect/out/forcing.txt >
20            model_output.txt",
21    )
22
23    observations.add_task(collect)
24    model.add_task(run_model)
25
26    meta = DAG_TPS("Lesson10")
27    meta.add_workflow(observations)
28    meta.add_workflow(model)
29    meta.make_dependencies()
30    meta.run()
```

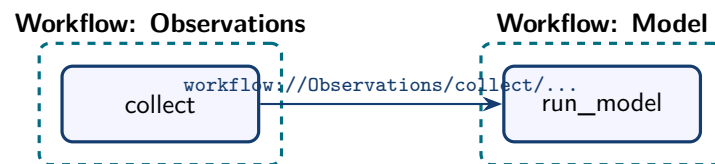
Expected behavior. The model workflow waits for the observations workflow task that produces forcing.txt.

Verify:

```
1 DIR=$(find /tmp -maxdepth 1 -type d -name '*-run_model' | tail -n 1)
2 sed -n '1,80p' "$DIR/model_output.txt"
```

If this legacy path behaves differently in a given environment, first verify construction without execution (`print(meta.as_json()["name"])`) and then inspect the dependency graph with `task.prevs` and `task.nexts`.

Scientific practice note. Meta-workflows are useful when a research system has separable but connected pipelines, such as observations, model execution, validation, and dissemination.



DAG_TPS("Lesson10") coordinates both workflows

Figure 10.1: A meta-workflow (DAG_TPS) coordinating two independently defined Workflow objects, connected by a cross-workflow `workflow://` reference.

10.1 Discussion: Why Compose Workflows Rather Than Tasks?

A natural question is why DAG_TPS exists at all, given that every example so far has placed all tasks inside a single Workflow. The answer is organizational, not merely technical: real research systems frequently have *separable* pipelines maintained by different people or teams — an observations-ingestion pipeline, a model-execution pipeline, a validation pipeline, a dissemination pipeline — each independently versioned and independently runnable, yet occasionally needing to consume each other's products. DAG_TPS lets each pipeline remain a self-contained Workflow, while a thin coordinating layer expresses the few cross-pipeline data dependencies that connect them.

Caution

The documentation is explicit that meta-workflow code is *legacy-sensitive* and less covered by automated tests than local single-workflow behavior. For production scientific use, verify the exact transversal scenario with representative workflows, and preserve the verification command alongside the experiment record.

Lesson 11 — Launch a Workflow Asynchronously

Objective. Start a workflow without blocking the main Python thread, observe its lifecycle, and wait for completion only when needed.

Concept. `Workflow.launch()` runs the same execution path as `run()`, but in a background thread. `Workflow.wait()` joins that thread. Event hooks let an application report task and workflow progress without polling task status.

Code:

```
1 from dagon import Workflow
```

```

2 from dagon.task import DagonTask, TaskType
3
4
5 workflow = Workflow("Lesson11")
6 workflow.add_task(DagonTask(TaskType.BATCH, "hello", "echo hello"))
7
8 workflow.on_task_start += lambda task: print("starting", task.name)
9 workflow.on_task_end += lambda task: print("ended", task.name, task.
    status.name)
10 workflow.on_workflow_end += lambda wf: print("workflow complete")
11
12 workflow.launch()
13 print("The caller is free to do other work.")
14 workflow.wait()

```

Run:

```

1 cp dagon.ini.sample dagon.ini
2 python3 lesson_11.py

```

Expected behavior. The caller's print statement can execute while the workflow is running; `wait()` then blocks until the task- and workflow-end callbacks have run. Dependencies are discovered automatically by `launch()`; calling `make_dependencies()` explicitly remains useful when the graph should be inspected or validated before launch.

Verify:

```

1 python3 examples/async/asynchronous_launch.py

```

Scientific practice note. Asynchronous launch is useful for notebook interfaces, web services, and experiment controllers that need to remain responsive while workflows run. Event callbacks should report progress or update state quickly; expensive work in a callback delays the workflow thread.

10.2 The Workflow Lifecycle Event Model

`launch()` returns the background `threading.Thread`; calling `launch()` while the same workflow is already running raises `RuntimeError`. `wait(timeout=...)` returns `True` when the workflow completed and `False` when the timeout expired; calling `wait()` *before* `launch()` is a documented no-op that returns `True`.

Listeners are registered with `+=` (or `workflow.add_listener(event_name, callback)`) and removed with `-=` (or `remove_listener`). Workflow-level callbacks receive the workflow object; task-level callbacks receive the task object. Listener exceptions are logged and do not stop the workflow — an important robustness property for long-running, unattended operational deployments.

Scientific Practice Note

DAGOnStar currently generates staging-in commands as part of a task launcher and has no separate data staging-out *transfer* phase; the corresponding hooks accurately delimit the existing preparation and cleanup phases rather than implying a transfer step that does not yet exist. Treat the table above as a description of current behavior, in keeping with the project's documentation-fidelity principle.

Table 10.1: Workflow and task lifecycle events.

Event	Argument	Fires when
<code>on_workflow_start</code>	workflow	Workflow execution begins.
<code>on_workflow_end</code>	workflow	Workflow execution ends.
<code>on_task_start</code>	task	A task thread begins.
<code>on_task_wait</code>	task	The task enters dependency waiting.
<code>on_task_staging_in_start</code>	task	Staging-in launcher preparation begins.
<code>on_task_staging_in_end</code>	task	Staging-in launcher preparation completes.
<code>on_task_execute_start</code>	task	The task begins its executor phase.
<code>on_task_staging_out_start</code>	task	Post-execution output/reference cleanup begins.
<code>on_task_staging_out_end</code>	task	Post-execution output/reference cleanup completes.
<code>on_task_end</code>	task	A task thread ends, whether finished or failed.

10.3 Operational Significance for Forecasting Chains

Asynchronous launch is the mechanism that allows DAGonStar to sit behind a notebook interface, a web service, or an automated experiment controller without blocking it — exactly the operational shape required by the recurring forecasting-chain theme of this book (developed fully in ???): a controller can `launch()` a daily forecast workflow, remain responsive to status queries or new requests while it runs, and react to `on_workflow_end` to trigger downstream publication once the chain completes.

10.4 Meta-Workflows and the Modularity Argument

It is worth strengthening, with a software-engineering argument, the organizational rationale ?? already gave for DAG_TPS. Software-engineering practice has long recognized a general principle, sometimes called the *single-responsibility principle*: a unit of code (here, a **Workflow**) should have one reason to change. An **Observations** workflow’s internal structure should change only when the observation-ingestion process itself changes — a new data source, a revised quality-control step — and should be insulated from changes to, say, the internal structure of a **Model** workflow’s simulation stages, even though the two workflows are connected by a data dependency.

Folding both pipelines into a single, undifferentiated **Workflow** object would violate this principle: a researcher modifying the model’s internal task decomposition would need to navigate the observation pipeline’s tasks to find the model’s, and a change to either pipeline’s internal task names could silently break `workflow://` references authored against the old names, with no namespace boundary to localize the blast radius of the change. DAG_TPS restores the namespace boundary the cross-workflow form `workflow://<workflow-name>/...` already implies syntactically (??): each constituent **Workflow** can be developed, version-controlled, and reasoned about largely independently, with the meta-workflow layer responsible only for the comparatively small number of edges that genuinely cross the boundary.

Scientific Practice Note

A practical corollary: when a researcher notices that a single `Workflow` object has grown to encompass two or more conceptually distinct pipelines — observation ingestion and model execution being the canonical example used throughout this book — that growth is itself a signal worth acting on. Splitting the monolithic workflow into separate `Workflow` objects, connected through `DAG_TPS` and explicit cross-workflow `workflow://` references, restores the modularity single-responsibility design favors, at the documented cost (??) of relying on a less heavily tested code path than single-workflow execution.

10.5 Event-Driven Design and the Observer Pattern

The lifecycle event model of Lesson 11 (??) is a direct application of what object-oriented design literature calls the *observer pattern*: a subject (here, the `Workflow` or an individual `Task`) maintains a list of interested observers (callbacks registered via `+=` or `add_listener`) and notifies each of them, without itself needing to know anything about what those observers do, whenever a relevant state transition occurs. This decoupling is what allows the same workflow engine core to serve radically different operational contexts — a one-off script that simply prints progress to a terminal, a notebook cell that updates a progress bar widget, or a production web service that updates a database row and triggers a downstream notification — without any of those contexts requiring a modification to the engine itself; each is simply a different set of registered observers attached to the same, unmodified event-emitting `Workflow`.

Example 10.1. Consider a minimal operational controller built entirely from the lifecycle events of ??, with no polling loop anywhere in its implementation:

```

1  status = {"state": "idle"}
2
3  def on_start(wf):
4      status["state"] = "running"
5
6  def on_end(wf):
7      status["state"] = "complete"
8      notify_downstream_service(wf.name)
9
10 workflow.on_workflow_start += on_start
11 workflow.on_workflow_end += on_end
12 workflow.launch()
```

This controller’s `status` dictionary — which might back, for instance, an HTTP status endpoint in a small operational service — stays correctly synchronized with the workflow’s actual state purely through event notification, with no thread of the controller’s own ever needing to ask “is it done yet?”

10.6 Failure Isolation in the Listener Model

?? noted, in passing, that listener exceptions are logged and do not stop the workflow. This design choice deserves a fuller discussion, because it represents a deliberate trade-off between two competing goals — *robustness* of the core workflow execution, and *visibility* of bugs in observer code — and a workflow author should understand which side of that trade-off the engine has chosen before relying on lifecycle callbacks for anything operationally critical.

The chosen design favors robustness: a buggy or exception-raising callback — for instance, a progress-reporting function that crashes on an unexpected character encoding — cannot, by design, cause an otherwise healthy scientific workflow to fail. This is generally the right default

for a workflow engine, since the scientific computation a workflow exists to perform should not be held hostage to the correctness of incidental monitoring or logging code layered on top of it.

Caution

The cost of this robustness is that a callback’s silent failure is, by construction, not visible through the workflow’s own `Status` (??) or return value — only through whatever logging configuration (??) is in place to surface logged exceptions. A workflow author who depends on a lifecycle callback for an operationally important side effect — triggering a downstream publication step, say, as the synthesis chapter’s forecasting-chain pattern (??) will do — should verify, separately from the workflow’s own success/failure status, that the relevant callback actually executed without error, rather than assuming that workflow success implies callback success.

10.7 Chapter Summary

DAG_TPS composes independently defined workflows through the same `workflow://` schema used within a single workflow, at the cost of reduced automated-test coverage that should be respected operationally. `launch()/wait()` and the lifecycle event table give a workflow author non-blocking execution and fine-grained progress reporting without polling. This chapter additionally grounded the meta-workflow design in the single-responsibility principle of software engineering, framed the lifecycle event model as an instance of the classical observer pattern with a worked, polling-free controller example, and examined the deliberate robustness-versus-visibility trade-off behind the engine’s choice to isolate listener exceptions from workflow execution. Together, these two lessons complete the engine’s *composition and orchestration* layer; ?? through ?? now turn to the engine’s most distinctive contribution: first-class AI and web-service tasks.

Exercise 10.1. Design, in prose and pseudocode, a lifecycle-event-based mechanism that would let a workflow author detect, *without* consulting external logs, whether an `on_workflow_end` callback raised an exception during a `launch()/wait()` execution. (Hint: consider what a callback can do, within itself, to communicate failure back to the code that called `wait()`, given that the callback’s own exceptions are swallowed by the engine.)

Chapter 11

Large Language Model Tasks

Running this chapter in Google Colab

All lessons in this chapter run in Google Colab with Option 3 installation (`git clone` then `pip install -e .`, see Appendix E). Use `local_config(workdir)` instead of `dagon.ini`, and use `Path(task.get_scratch_dir())` to read task outputs instead of shell `find/cat` commands (see ?? and Appendix E for the complete guide).

This chapter is the book’s first sustained treatment of the engine’s AI-HPC integration layer, and the conceptual core of the workshop abstract that motivates this textbook. `TaskType.LLM` invokes an OpenAI-compatible Chat Completions API *as a workflow task*: a Large Language Model inference becomes a DAG node with the same dependency, staging, and provenance guarantees as a numerical simulation.

11.1 Design Goals

`TaskType.LLM` is intended for JSON request construction, local workflow data inputs, and response capture; notably, **it does not require the OpenAI Python package**. The task speaks the OpenAI-compatible Chat Completions HTTP surface directly, which means any provider exposing that surface — not only OpenAI itself — can be configured as a target.

11.2 Provider Configuration

Each provider has an INI section named `llm.<provider-name>`; any number of these sections may be configured simultaneously (??). `endpoint` is the provider base URL (or a full `/chat/completions` URL); `model` is the default model; credentials come from either `api_key_env` or `api_key` — the environment-variable form is preferred:

```
1 [llm.research]
2 endpoint=https://api.example.org
3 api_key_env=DAGON_RESEARCH_LLM_KEY
4 model=research-chat
```

Caution

Do not commit real API keys. The endpoint receives a `POST /v1/chat/completions` request unless its configured URL already ends in `/chat/completions`.

11.3 Creating an LLM Task

The prompt is a JSON-compatible Chat Completions request. `params` supplies named values for `{placeholder}` strings anywhere in that JSON. The `model` value, if present in the request, overrides the provider default.

```

1 ask = DagonTask(
2     TaskType.LLM,
3     "summarize",
4     {"messages": [{"role": "user", "content": "Summarize {subject}."}], "
      temperature": 0.2},
5     provider="research",
6     params={"subject": "the experiment"},
7 )

```

Listing 11.1: Minimal LLM task construction.

The prompt may instead be supplied as a JSON *string*, which is useful when a script assembles the request dynamically with `json.dumps`. When a workflow is serialized, LLM tasks use `"type": "llm"` and retain the provider, parameters, input-file mapping, and output filename; loading that JSON elsewhere requires the named provider to be configured locally on the loading machine.

11.4 Workflow Input Files: Connecting Simulation Output to a Prompt

`input_files` attaches UTF-8 text produced by another task. Keys become prompt parameters; values use the ordinary `workflow://` form developed in ??:

```

1 input_files={"report": "workflow:///prepare/output/report.txt"}

```

This single line creates the `prepare` → `review` dependency, exactly as in any file-based task. Before the request is sent, the file is copied under `.dagon/inputs/<workflow>/<task>/...` inside the LLM task's directory, and its UTF-8 content replaces `{report}` in the prompt. An inline `workflow:///prepare/output/report.txt` reference written directly inside a prompt string is also parsed, staged, and replaced; using `input_files` is recommended chiefly because it keeps longer prompts readable.

Caution

Input paths must be relative to the producer task directory and cannot contain ... Binary files and remote-only producer files are **not** supported by this task type: the LLM task's file integration is deliberately restricted to local, UTF-8 text.

The full response JSON is written to `response.json` in the LLM task's working directory (or to the relative `output_file` supplied to the task), so a downstream task can consume it via `workflow://` exactly as it would consume any other task's file output.

11.5 Worked Example: A Locally Verifiable LLM Workflow

The repository ships a complete, locally verifiable, credential-free example that this book reproduces in full, because it is the cleanest illustration of how an AI inference step is woven into an ordinary DAG.

```

1 import json
2 import tempfile
3 import threading
4 from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
5 from pathlib import Path
6
7 from dagon import Workflow
8 from dagon.task import DagonTask, TaskType
9
10
11 class MockChatHandler(BaseHTTPRequestHandler):
12     def do_POST(self):
13         if self.path != "/v1/chat/completions":
14             self.send_error(404)
15             return
16         request = json.loads(self.rfile.read(int(self.headers["Content-
17             Length"]))).decode("utf-8"))
18         content = request["messages"][-1]["content"]
19         payload = json.dumps({"choices": [{"message": {"role": "assistant",
20             "content": "Mock summary: " + content}}]}).encode("utf-8")
21         self.send_response(200)
22         self.send_header("Content-Type", "application/json")
23         self.send_header("Content-Length", str(len(payload)))
24         self.end_headers()
25         self.wfile.write(payload)
26
27     def log_message(self, _format, *_args):
28         pass
29
30 def main():
31     server = ThreadingHTTPServer(("127.0.0.1", 0), MockChatHandler)
32     threading.Thread(target=server.serve_forever, daemon=True).start()
33     try:
34         with tempfile.TemporaryDirectory(prefix="dagon-llm-lesson-") as
35             scratch:
36                 config = {
37                     "batch": {"scratch_dir_base": scratch, "run_base": "", "
38                         threads": "1"},
39                     "dagon_service": {"use": "False", "route": "http://localhost
40                         :57000"},
41                     "ftp_pub": {"ip": "localhost", "user": "anonymous", "
42                         password": ""}, "slurm": {"partition": ""},
43                     "llm.local_mock": {"endpoint": "http://127.0.0.1:%s" %
44                         server.server_port,
45                         "api_key": "example-key-not-a-secret", "
46                         model": "mock-chat"},
47                 }
48                 workflow = Workflow("Lesson12", config=config)
49                 produce = DagonTask(TaskType.BATCH, "prepare_report", "mkdir -p
50                     output; printf 'temperature increased by 1.2 C' > output/
51                     report.txt")
52                 summarize = DagonTask(TaskType.LLM, "summarize_report",
53                     {"messages": [{"role": "user", "content": "Summarize this
54                         report: {report}"}]}),
55                     provider="local_mock", input_files={"report": "workflow:///
56                         prepare_report/output/report.txt"})
57                 workflow.add_task(produce)
58                 workflow.add_task(summarize)
59                 workflow.run()
60                 print(Path(summarize.working_dir, "response.json").read_text(
61                     encoding="utf-8"))

```

```

50     finally:
51         server.shutdown()
52         server.server_close()
53
54
55 if __name__ == "__main__":
56     main()

```

Listing 11.2: `lesson_12_llm_tasks.py`: a self-contained, mock-backed LLM workflow.

```

1 python3 examples/tutorial/lesson_12_llm_tasks.py

```

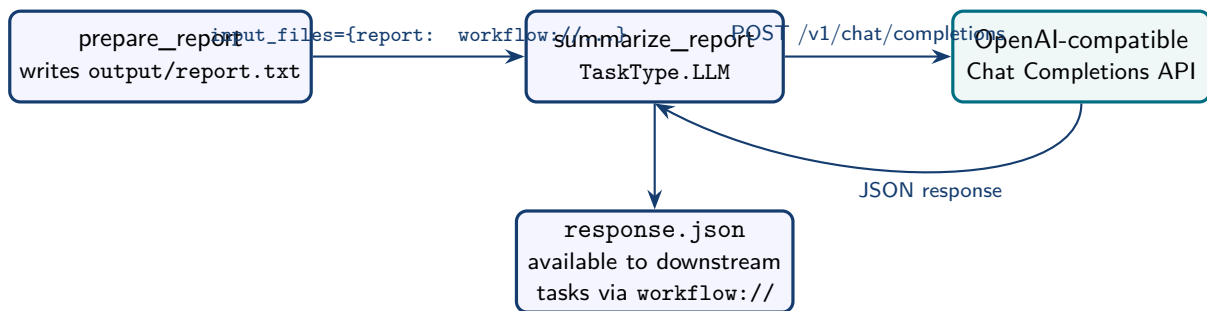


Figure 11.1: An LLM task as a first-class DAG node: it consumes a producer’s text output through the ordinary `workflow://` mechanism and exposes its JSON response as an ordinary, dependency-trackable task output.

11.6 Walkthrough

11.6.1 Step 1: Execute and Observe

The mock server implements only the minimal OpenAI-compatible `/v1/chat/completions` surface needed for the lesson, bound to `127.0.0.1` on an unused port, and is shut down automatically when the workflow completes. Expected output is a JSON completion of the shape:

```

1 {"choices": [{"message": {"content": "Mock summary: Summarize this report:
   temperature increased by 1.2 C"}}]}

```

11.6.2 Step 2: The Producer Task

```

1 produce = DagonTask(
2     TaskType.BATCH,
3     "prepare_report",
4     "mkdir -p output; printf 'temperature increased by 1.2 C' > output/
   report.txt",
5 )

```

Its eventual file location is conceptually `<prepare_report task directory>/output/report.txt` — an absolute path generated only at runtime, and therefore never written directly into the LLM task’s prompt, which would make the workflow non-portable.

11.6.3 Step 3: The JSON Request and File Attachment

```

1 summarize = DagonTask(
2     TaskType.LLM,
3     "summarize_report",
4     {"messages": [{"role": "user", "content": "Summarize this report: {
5         report}"}]},
6     provider="local_mock",
7     input_files={"report": "workflow:///prepare_report/output/report.txt"},
8 )

```

{report} is a prompt parameter; `input_files` supplies its value via the standard `workflow:///<producer-name>` form. Building dependencies recognizes the reference and creates `prepare_report` → `summarize_report`, exactly as ?? predicts for any other task type.

11.6.4 Step 4: Provider Configuration in Code

Because the mock server selects a free port at runtime, the example builds its provider configuration in Python rather than in a static INI file:

```

1 "llm.local_mock": {
2     "endpoint": "http://127.0.0.1:%s" % server.server_port,
3     "api_key": "example-key-not-a-secret",
4     "model": "mock-chat",
5 }

```

Provider section names always use the `llm.<provider-name>` pattern; the task's `provider` argument selects the matching suffix.

11.6.5 Step 5: Run and Inspect

```

1 workflow.add_task(produce)
2 workflow.add_task(summarize)
3 workflow.run()

```

`run()` builds missing dependencies automatically. On completion, `summarize.working_dir` identifies the LLM task's directory, and the full response is available at `Path(summarize.working_dir, "response.json")`. The *complete* JSON response is deliberately retained, rather than only the assistant's text, so a downstream task can extract provider-specific metadata or choices with its own deterministic program — a design choice directly in service of provenance.

11.6.6 Step 6: Switching to a Real Provider

For a real OpenAI-compatible service, the workflow request is unchanged; only configuration changes, in a local, ignored `dagon.ini`:

```

1 [llm.research]
2 endpoint=https://api.example.org
3 api_key_env=DAGON_RESEARCH_LLM_KEY
4 model=research-chat

```

```

1 export DAGON_RESEARCH_LLM_KEY=...

```

with `provider="research"` substituted into the task constructor.

Caution

Never commit the key, and never replace the safe sample configuration with a real one in version control.

11.7 Verification

```
python3 -m unittest tests.test_llm -v
```

Scientific Practice Note

Record the provider name, model, prompt template, and source input files in your workflow source, and preserve `response.json` as a workflow output. This makes AI-assisted interpretation inspectable — a downstream reader can audit exactly what was asked and what was returned — while keeping secret credentials entirely out of the workflow and the repository.

11.8 Why a Provider-Agnostic Surface Matters

?? already noted that `TaskType.LLM` requires no provider-specific SDK, speaking the OpenAI-compatible Chat Completions surface directly over HTTP. This design decision is worth examining for its broader methodological implications, because it bears directly on the scientific reproducibility properties this book has emphasized throughout.

A workflow engine that depended on a specific vendor’s Python SDK would couple every LLM-using workflow to that SDK’s release cadence, versioning behavior, and continued maintenance — a dependency risk qualitatively different from, and arguably larger than, the dependency risk already accepted for, say, the Docker SDK (??), since the generative-AI provider landscape has historically changed faster than container runtimes have. By speaking the request/response *protocol* directly, rather than a vendor SDK, `TaskType.LLM` decouples a workflow’s AI-assisted step from any single vendor’s continued existence: the `endpoint` configuration value (??) is, in principle, swappable to any provider implementing the same `/chat/completions` surface, with no change to the workflow’s Python source.

Scientific Practice Note

This protocol-level portability is the same architectural strategy already examined, in a different guise, for remote execution (??): just as a `BATCH` task’s command string is portable across any POSIX-like remote host reachable by SSH, an LLM task’s JSON request is portable across any provider speaking the OpenAI-compatible surface, with the concrete provider selected entirely by configuration (??), not by code.

11.9 The Epistemics of AI-Assisted Scientific Interpretation

Embedding an LLM call inside a scientific DAG, as Lesson 12 demonstrates, raises a methodological question that is worth addressing directly, rather than leaving implicit: what kind of scientific claim, if any, does the output of an LLM task support, and how should a researcher’s confidence in that output be calibrated?

It is important to be precise about what the mock example of ?? is, and is not, demonstrating. The mechanism — a prompt assembled from a producer task’s UTF-8 output, sent to a configured endpoint, with the full response preserved as a dependency-trackable file

— is provider-agnostic and equally applicable whether the configured `provider` ultimately resolves to a deterministic mock, a well-characterized classical text-processing service, or a state-of-the-art generative language model. The engine itself makes no claim about, and has no mechanism for assessing, the *factual reliability* of whatever text a given provider returns; that assessment is and remains entirely the responsibility of the researcher consuming the response.

Caution

A workflow that uses an LLM task to *summarize* or *describe* a model’s numerical output (as in the worked example of this chapter) is making a categorically different kind of use than a workflow that uses an LLM task’s output as a direct, unreviewed input to a further numerical or decision-making step. The former is analogous to using a research assistant’s draft summary — valuable as a starting point, properly understood as requiring expert review before being relied upon; the latter would require a substantially higher standard of verification, calibration, and, in most scientific contexts, human review in the loop. This book’s worked examples are deliberately confined to the former, lower-stakes category, and the full-response-retention design discussed below is precisely what makes the necessary expert review possible after the fact.

This is, in fact, the deeper reason ?? emphasized that the *complete* response JSON, not merely the assistant’s extracted text, is preserved in `response.json`: an unreviewed text summary, divorced from the full request that produced it (including the exact prompt, provider, and model), is far harder for a later reader to critically assess than a complete, inspectable request/response pair. The provenance argument already developed in ?? for file-based dependencies applies with, if anything, greater force here: the epistemic status of an AI-generated scientific summary depends critically on knowing exactly what was asked and exactly what model produced the answer, information that a bare text string alone does not carry.

11.10 Cost, Latency, and Rate Limits as Operational Constraints

A final consideration, absent from the purely mechanical treatment of Lesson 12 but essential for any researcher moving from the mock example to a real provider, concerns the operational constraints that distinguish LLM tasks from the local, cost-free `BATCH` tasks used throughout most of this book’s tutorial examples.

Unlike a local shell command, a real-provider LLM completion typically incurs a per-request monetary cost, a network round-trip latency significantly larger than a local subprocess call, and is frequently subject to rate limits imposed by the provider. These constraints interact directly with the workflow-design questions already raised elsewhere in this book. The task-granularity discussion of ??, for instance, takes on a sharper edge for LLM tasks specifically: an overly fine-grained decomposition that issues many small, separate LLM requests where one consolidated request would suffice multiplies both cost and the risk of rate-limit-induced failures, with no corresponding gain in traceability beyond what a single, well-structured request and response already provide.

Scientific Practice Note

The `timeout` parameter exposed on the `LLMTask` constructor (Appendix A, default 120 seconds) is the engine’s primary mechanism for bounding the operational risk of

an unresponsive or rate-limited provider; a workflow author moving from the mock provider used in this book’s local examples to a production LLM provider should explicitly reconsider this default against the provider’s documented typical latency and the workflow’s own tolerance for a stalled task, rather than relying on the default value chosen for local, low-latency testing.

11.11 Chapter Summary

TaskType.LLM demonstrates the engine’s central architectural claim at its most consequential: an AI inference call, expressed as a JSON Chat Completions request, participates in the same dependency graph, the same `workflow://` staging mechanism, and the same provenance discipline as a shell command or an HPC job submission. This chapter additionally argued that the task type’s provider-agnostic, protocol-level design is itself a reproducibility decision rather than a mere implementation convenience, addressed directly the epistemic status of AI-generated scientific text embedded in a DAG and the corresponding case for full-response retention, and discussed the cost, latency, and rate-limit constraints that distinguish LLM tasks operationally from the local tasks examined in Part I. ?? extends the same architectural principle to arbitrary HTTP/HTTPS services, and ?? extends it to native Python functions.

Exercise 11.1. A researcher proposes replacing a workflow’s **NATIVE**-task statistical quality-control step (??) with an LLM task that asks a language model to “check whether this data looks reasonable.” Using the epistemic-status discussion of this chapter, write a short methodological critique of this proposal, and suggest a revised design that uses an LLM task appropriately alongside, rather than instead of, a deterministic quality-control mechanism.

Chapter 12

Web Service Tasks

Running this chapter in Google Colab

All lessons in this chapter run in Google Colab with Option 3 installation (`git clone` then `pip install -e .`, see Appendix E). Use `local_config(workdir)` instead of `tagon.ini`, and use `Path(task.get_scratch_dir())` to read task outputs instead of shell `find/cat` commands (see ?? and Appendix E for the complete guide).

12.1 Design Goals

`TaskType.WEB` accepts a JSON-serializable HTTP/HTTPS request specification *rather than a shell command*. The workflow driver writes that specification under `.tagon/`, stages any referenced files below `inputs/`, and invokes the web runner inside the task directory. Consequently, the HTTP request occurs *where the task executes*: a Slurm-scheduled web task (`executor="slurm"`) issues its request from the compute node, not from the controlling process, without changing the dataflow model in any way.

Scientific Practice Note

The lesson's stated objective captures the design intent precisely: a web request is a first-class DAGOnStar task — its dependency is inferred from `workflow://`, its input is staged into a private scratch directory, and its response is a declared workflow output, exactly like any other task type covered in this book.

Lesson 13 — Execute a Web Task Locally

Objective. Construct a reproducible workflow in which a local HTTP service receives a file produced by an upstream task.

Maintained source. `examples/tutorial/lesson_13_web_tasks.py`.

```
1 python3 examples/tutorial/lesson_13_web_tasks.py
```

It starts a local server bound only to 127.0.0.1; no network service, account, or credential is required.

Code (standalone, copy-pasteable version):

```
1 import json
2 import tempfile
3 import threading
4 from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
```

```

5  from pathlib import Path
6
7  from dagon import Workflow
8  from dagon.task import DagonTask, TaskType
9
10
11 class EchoHandler(BaseHTTPRequestHandler):
12     def do_POST(self):
13         body = self.rfile.read(int(self.headers["Content-Length"]))
14         reply = json.dumps({"received": "sample" in body.decode("utf-8")
15                             }).encode("utf-8")
16         self.send_response(200)
17         self.send_header("Content-Type", "application/json")
18         self.send_header("Content-Length", str(len(reply)))
19         self.end_headers()
20         self.wfile.write(reply)
21
22     def log_message(self, _format, *_args):
23         pass
24
25 def config(scratch):
26     return {
27         "batch": {"scratch_dir_base": scratch, "run_base": "", "threads": "1"},
28         "dagon_service": {"use": "False", "route": "http://localhost:57000"},
29         "ftp_pub": {"ip": "localhost", "user": "anonymous", "password": ""},
30         "slurm": {"partition": ""},
31     }
32
33
34 def main():
35     server = ThreadingHTTPServer(("127.0.0.1", 0), EchoHandler)
36     threading.Thread(target=server.serve_forever, daemon=True).start()
37     try:
38         with tempfile.TemporaryDirectory(prefix="dagon-web-lesson-") as scratch:
39             workflow = Workflow("Lesson13", config=config(scratch))
40             prepare = DagonTask(
41                 TaskType.BATCH, "prepare", "mkdir -p output; printf sample > output/data.txt"
42             )
43             upload = DagonTask(
44                 TaskType.WEB,
45                 "upload",
46                 {
47                     "method": "POST",
48                     "url": "http://127.0.0.1:%s/upload" % server.server_port,
49                     "multipart": {
50                         "dataset": {
51                             "file": "workflow:///prepare/output/data.txt",
52                             "content_type": "text/plain",
53                         }
54                     },
55                     "expected_status": 200,
56                     "outputs": {"body": "reply.json", "metadata": "request.json"},

```

```

57         },
58     )
59     consume = DagonTask(
60         TaskType.BATCH, "consume", "cat workflow:///upload/
           outputs/reply.json"
61     )
62     for task in (prepare, upload, consume):
63         workflow.add_task(task)
64     workflow.run()
65     print(Path(upload.working_dir, "outputs", "reply.json").
           read_text())
66     print(Path(upload.working_dir, ".dagon", "web_result.json")
           .read_text())
67 finally:
68     server.shutdown()
69     server.server_close()
70
71
72 if __name__ == "__main__":
73     main()

```

Scientific practice note. A web request is, in this model, exactly as auditable as a shell command: its specification, staged inputs, and declared outputs all live inside the task's scratch directory, alongside the launcher-based provenance developed in ??.

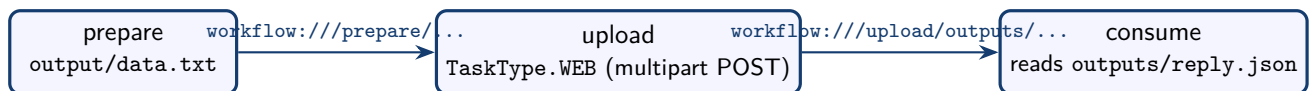


Figure 12.1: The three-task Web-task pipeline of Lesson 13: a file producer, an HTTP upload step whose multipart payload is a staged `workflow://` reference, and a consumer of the declared HTTP response output.

12.2 Interpretation and Inspection

The multipart wrapper

```

1  "multipart": {
2      "dataset": {
3          "file": "workflow:///prepare/output/data.txt",
4          "content_type": "text/plain",
5      }
6  }

```

explicitly distinguishes a file upload from an ordinary string field. DAGonStar creates the `prepare` → `upload` dependency from the reference and copies the file into `upload/inputs/Lesson13/prepare/output/data.txt`, mirroring exactly the staging behavior already established for shell-command tasks (??). The response body is written to `upload/outputs/reply.json`; response metadata — including status code, elapsed time, content hash, and a *non-secret* authentication description — is written to `.dagon/web_result.json` and to the declared `outputs/request.json`.

12.3 Authentication

For authenticated endpoints, the specification uses environment-variable *names*, never literal secrets:

```
1 {"type": "bearer", "token_env": "SERVICE_TOKEN"}
```

Basic and API-key authentication modes follow the same principle (see `username_env`, `password_env`, `value_env` in ??).

Caution

Never place secret values in a workflow, a generated request file, a task output, or a tutorial. The environment-variable indirection is not optional boilerplate; it is the load-bearing mechanism that keeps credentials out of version-controlled workflow definitions and out of checkpoint or serialization JSON.

12.4 Specification Surface

Beyond `multipart`, the full WebTask specification supports `query`, `headers`, `json`, `data`, `auth`, `timeout`, `retry` settings, `expected_status` codes, and `outputs`. Recursive `workflow://` values anywhere in the specification infer dependencies; referenced files are copied below `inputs/`. Body, headers, and metadata outputs are written to safe relative paths below `outputs/`. `executor="slurm"` accepts the same scheduler options, in `resources`, used by Slurm batch tasks (??) — allowing a web request to be issued from a scheduled compute node, for instance to publish a forecast product from the same node that produced it.

12.5 Verification

```
1 python3 -m unittest tests.test_web -v
```

12.6 Scientific Application: Web Tasks as a Publication Stage

In the operational forecasting-chain pattern developed fully in ??, the Web task type is most naturally the *publication* stage of a pipeline: a model’s post-processed product is uploaded to a dissemination API, a dashboard, or a data catalog as the final DAG node, with the same dependency-graph guarantees — it cannot run before post-processing completes, and its outcome is recorded as an ordinary, inspectable task output — as every preceding stage.

12.7 Idempotency and the Risks of Retrying a Web Task

A subtlety that the mechanical treatment of Lesson 13 does not surface, but that any researcher moving beyond the tutorial’s local echo-server example must reason about explicitly, concerns *idempotency* — whether repeating a request has the same effect as issuing it once.

An HTTP `GET` request is, by the protocol’s own design convention, expected to be idempotent and free of side effects: issuing it twice should leave the world in the same state as issuing it once, which makes automatic retry behavior (referenced among the specification surface in ?? as “retry settings”) generally safe to enable. An HTTP `POST` request, by contrast, frequently *is not* idempotent: a publication endpoint that appends a new forecast record to a database, or a notification endpoint that sends an email, may have a materially different effect if invoked twice than if invoked once, regardless of whether the second invocation was a deliberate workflow rerun or an automatic retry triggered by a transient network failure.

Caution

A workflow author configuring a **WEB** task’s retry behavior for a non-idempotent endpoint — the publication stage of the forecasting-chain pattern developed in ?? being the clearest example in this book — should verify, against the target service’s own documentation, whether the endpoint itself provides idempotency guarantees (for instance, via a client-supplied idempotency key included in the request body or headers) before relying on the engine’s retry mechanism. Where no such guarantee exists, automatic retries risk silently duplicating a published scientific product, an error considerably harder to detect after the fact than an outright request failure would be.

12.8 Comparing Web Tasks and LLM Tasks as Sibling Mechanisms

Having now examined both `TaskType.LLM` (??) and `TaskType.WEB` in full, it is worth making explicit the structural relationship between them, since a first reading might reasonably ask why two separate task types exist at all, given that an LLM completion request is, after all, simply a particular kind of HTTP POST.

The answer is that `TaskType.LLM` can be understood as a *specialization* of the general HTTP-request capability `TaskType.WEB` provides, pre-configured with the specific conventions — the `/chat/completions` path convention, the `messages`-based JSON body shape, the `{parameter}` prompt-templating mechanism, and the `provider`-section-based credential lookup of ?? — that make authoring an AI-completion task substantially more convenient than assembling the equivalent raw **WEB** task specification by hand would be. ?? in ?? will complete this comparison with the third sibling, `TaskType.NATIVE`; the present comparison can already be made concrete:

Table 12.1: LLM as a convenience specialization of **WEB**.

Aspect	<code>TaskType.WEB</code>	<code>TaskType.LLM</code>
Request shape	Fully general (<code>method</code> , <code>url</code> , <code>headers</code> , <code>json</code> , <code>multipart</code> , ...)	Fixed to a Chat Completions JSON body with <code>messages</code>
Endpoint configuration	Literal <code>url</code> per task	Resolved from <code>[llm.<provider>]</code> section
Templating	None built in; values must be assembled by the caller	<code>{parameter}</code> substitution from <code>params/input_files</code>
Typical role	Publication, ingestion, arbitrary third-party APIs	AI-assisted interpretation, summarization

Scientific Practice Note

A useful diagnostic question for a workflow author unsure which task type to reach for: if the target endpoint is an OpenAI-compatible Chat Completions API, `TaskType.LLM`’s built-in conventions remove enough boilerplate to be worth using even though the underlying mechanism — an HTTP request with a JSON body — is the same one `TaskType.WEB` would also support; for every other HTTP/HTTPS service, `TaskType.WEB`’s full generality is the correct, and only, choice.

12.9 Designing a Robust Publication Specification

Building on ??’s observation that Web tasks are naturally suited to the publication stage of a forecasting chain, it is worth working through, in more depth than the tutorial’s locally verifiable example requires, what a production-quality publication task specification should additionally account for.

1. **Explicit `expected_status` checking.** As shown in Lesson 13’s specification (`"expected_status": 200`), a publication task should never treat an HTTP-level success (a response received at all) as equivalent to an application-level success (the endpoint accepted and processed the published product); checking the expected status code is the mechanism that distinguishes the two.
2. **Bounded `timeout`.** Publication endpoints, like LLM providers (??), may become slow or unresponsive under load; an unbounded wait on a stalled publication step blocks the `publish` task and, transitively, anything downstream of it, indefinitely.
3. **Declared outputs sufficient for audit.** Retaining both `body` and `metadata` outputs, as Lesson 13 does, ensures that a later audit of a published product can confirm not only *what* was published but *when* and with what response the publishing endpoint acknowledged it.
4. **Authentication via environment-variable indirection.** As ?? emphasized, and as this chapter’s own discussion of multipart uploads illustrated, no literal credential should ever appear in the specification itself.

12.10 Chapter Summary

`TaskType.WEB` generalizes file-based scientific software integration (??) to network services: any HTTP/HTTPS endpoint becomes a DAG node whose inputs are staged via `workflow://` and whose outputs are ordinary, dependency-trackable files. This chapter additionally examined the idempotency risks of automatic retries against non-idempotent publication endpoints, situated `TaskType.LLM` explicitly as a convenience specialization of the more general `TaskType.WEB` mechanism, and worked through a four-point checklist for designing a production-quality publication task specification. ?? completes the AI-HPC integration triad by bringing native, importable Python functions into the same model.

Exercise 12.1. Using the four-point checklist above, critique the minimal Lesson 13 specification reproduced earlier in this chapter, identifying which of the four points it already satisfies and which would need strengthening before the same specification could be safely used against a real, non-idempotent, internet-facing publication endpoint rather than the tutorial’s local echo server.

Chapter 13

Native Python Tasks

Running this chapter in Google Colab

All lessons in this chapter run in Google Colab with Option 3 installation (`git clone` then `pip install -e .`, see Appendix E). Use `local_config(workdir)` instead of `dagon.ini`, and use `Path(task.get_scratch_dir())` to read task outputs instead of shell `find/cat` commands (see ?? and Appendix E for the complete guide).

13.1 Design Goals

`TaskType.NATIVE` lets an *importable Python function* act as a workflow task while preserving DAGOnStar dependencies, scratch isolation, staged files, declared outputs, and checkpoint metadata. Native tasks are appropriate when scientific logic is naturally Python code — for instance, an array-processing or statistical routine built on NumPy or xarray — and should not be artificially encoded as a shell command merely to fit the BATCH task model.

Caution

`NativeTask` records a `module:function` reference and JSON-compatible parameter bindings. The runner *imports* the function in the task environment, so functions must be module-level and importable. Lambdas and nested functions are deliberately rejected, because they are not portable across executors (in particular, a remote Slurm executor must be able to import the same function from the same module on a different machine).

13.2 Worked Example

```
1 def transform(input_file: str, scale: float, output_file: str) -> dict:
2     with open(input_file, encoding="utf-8") as source:
3         values = [float(line.strip()) for line in source if line.strip()]
4     with open(output_file, "w", encoding="utf-8") as destination:
5         for value in values:
6             destination.write(f"{value * scale}\n")
7     return {"count": len(values), "scale": scale}
```

Listing 13.1: `native_functions.py`: the importable native function.

```
1 import json
2 import tempfile
3 from pathlib import Path
4
```

```

5 from dagon import Workflow
6 from dagon.task import DagonTask, TaskType
7
8
9 def config(scratch):
10     return {
11         "batch": {"scratch_dir_base": scratch, "run_base": "", "threads": "1"},
12         "dagon_service": {"use": "False", "route": "http://localhost:57000"},
13         "ftp_pub": {"ip": "localhost", "user": "anonymous", "password": ""},
14         "slurm": {"partition": ""},
15     }
16
17
18 with tempfile.TemporaryDirectory(prefix="dagon-native-lesson-") as scratch:
19     workflow = Workflow("Lesson14", config=config(scratch))
20     produce = DagonTask(
21         TaskType.BATCH, "produce", "mkdir -p data; printf '2\n4\n' > data/
22         input.txt"
23     )
24     transform = DagonTask(
25         TaskType.NATIVE,
26         "transform",
27         "native_functions:transform",
28         inputs={"input_file": "workflow:///produce/data/input.txt", "scale": 1.5},
29         outputs={"output_file": "scaled.txt"},
30     )
31     consume = DagonTask(
32         TaskType.BATCH, "consume", "cat workflow:///transform/outputs/scaled
33         .txt"
34     )
35     for task in (produce, transform, consume):
36         workflow.add_task(task)
37     workflow.run()
38     print(Path(transform.working_dir, "outputs", "scaled.txt").read_text())
39     print(json.loads(Path(transform.working_dir, ".dagon", "native_result.
40     json").read_text()))

```

Listing 13.2: lesson_14_native_tasks.py: the workflow that calls it.

Scientific Practice Note

This small two-file layout (`native_functions.py` alongside the workflow script) is essential, not incidental: it is precisely what makes the function importable by the separate runner process. A native function defined inline inside the workflow script's `__main__` block would not be importable, and would fail at the same architectural boundary that excludes lambdas.

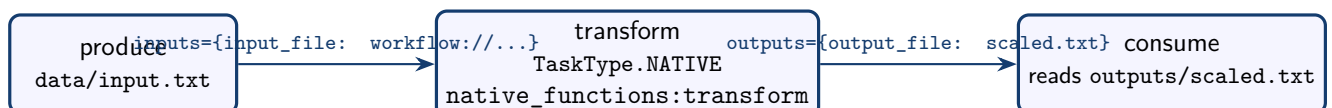


Figure 13.1: The Native task pipeline: a staged input file becomes a Python function argument, and a declared output parameter becomes an ordinary, dependency-trackable file for the downstream consumer.

13.3 Interpretation

The reference creates **produce** $\rightarrow$ **transform**; the runner passes a scratch-local file path to the **input_file** parameter and a scratch-local output path to the **output_file** parameter. The declared output is therefore available to the downstream task through **workflow:///transform/outputs/scaled**, exactly as any other task’s file output would be. The Python function’s *return value* is retained as JSON metadata in **.dagon/native_result.json** — analogous in spirit to the full-response retention strategy used by LLM tasks (??): both preserve a richer, machine-readable record than the minimum a human reader strictly needs.

13.4 Constraints and Extension

- Scalars and return values must be JSON-serializable.
- Output paths must be relative and cannot escape the task’s scratch directory.
- For a configured cluster, **executor="slurm"** may be used with scheduler settings supplied in **resources**; the function’s module and DAGonStar itself must be installed where the compute node can import them.

13.5 Verification

```
python3 -m unittest tests.test_native -v
```

See also **examples/native/README.md** in the repository for further, runnable verification material.

13.6 Native Tasks in a Scientific Pipeline

Native tasks complete a triad with LLM and Web tasks that, taken together, constitute the engine’s full AI–HPC integration surface:

Table 13.1: The three non-shell task types and their characteristic role.

Task type	What it executes	Characteristic role
LLM	OpenAI-compatible chat request	AI-assisted interpretation, summarization
WEB	Arbitrary HTTP/HTTPS request	Publication, ingestion, third-party API calls
NATIVE	Importable Python function	In-process scientific computation (NumPy/xarray-style)

In a typical environmental forecasting chain (??), a **NATIVE** task is often the most natural choice for a post-processing step that is already implemented as a Python function — for instance, a unit conversion, a regridding routine, or a statistical quality-control check — avoiding the indirection of wrapping that function in a throwaway command-line script merely to satisfy a **BATCH** task interface.

13.7 Why Importability, Not Mere Callability, Is Required

?? stated, as a constraint, that native functions must be module-level and importable, and that lambdas and nested functions are rejected. It is worth explaining *why* this constraint is

structurally necessary, rather than an arbitrary restriction, since understanding the reason clarifies what a workflow author can and cannot work around.

A `NativeTask` does not execute the supplied function inside the same Python process, or even necessarily the same thread, that constructed the `DagOnStar` object. Recall from ?? that DAGOnStar tasks are threads, not processes, for backends whose computation happens in a spawned subprocess or over a network connection; a `NativeTask` configured with `executor="local"` does still ultimately need to make the target function available to whatever runner process or thread actually performs the call, and a `NativeTask` configured with `executor="slurm"` must additionally make the function available on a *different machine entirely* — the compute node a Slurm job lands on, which may share no process memory whatsoever with the controller that submitted the job.

A lambda or a function nested inside another function exists only as an in-memory object inside the defining process; Python provides no general mechanism to serialize such an object's code and ship it, intact, to another process or machine (the standard `pickle` mechanism, which many readers may expect to handle this, explicitly does not support pickling lambdas or local functions, for exactly this reason). A module-level function, by contrast, can always be *re-imported by name* — the runner simply executes the equivalent of `from native_functions import transform` — which works identically whether the runner is in the same process, a different local process, or a remote compute node, provided only that the same module is installed and importable wherever the runner executes.

Scientific Practice Note

This is the same architectural principle already encountered, in a different guise, in this book's discussion of remote Batch and Slurm execution (??): just as a `BATCH` task's portability across hosts depends on the target executable being installed wherever the command runs, a `NATIVE` task's portability across executors depends on the target Python module being installed wherever the runner executes. The module-level-function constraint is simply this same requirement, restated for Python's own import mechanism rather than for a shell's `PATH`.

13.8 Native Tasks and the Scientific Python Ecosystem

?? already suggested that Native tasks are a natural fit for NumPy- or xarray-style computation; it is worth developing this suggestion into a fuller discussion of where, exactly, the boundary should fall between a `NATIVE` task and a `BATCH` task that happens to invoke a Python script.

Consider a regridding or unit-conversion routine already implemented as a well-tested, importable function in a research group's existing Python codebase. Wrapping it as a `BATCH` task would require writing a small command-line entry point — parsing arguments, reading and writing files explicitly — purely to satisfy the shell-command interface, even though the underlying logic never needed a command-line interface in the first place. Wrapping the same function as a `NATIVE` task, by contrast, requires no such adapter: the function's existing signature, provided its arguments are JSON-compatible scalars or file paths, can be bound directly through the `inputs/outputs` mapping mechanism of Lesson 14, with DAGOnStar itself handling the file staging and result capture that a hand-written command-line wrapper would otherwise need to implement redundantly.

Example 13.1. Contrast the two wrapping strategies for a hypothetical `convert_units(value, from_unit, to_unit)` function already present in a research codebase. As a `BATCH` task, a researcher would need to write and maintain a separate `convert_units_cli.py` script with `argparse`-based argument handling, purely to make the function invocable from a shell com-

mand string. As a **NATIVE** task, the existing function can be called directly — provided it is moved to, or already lives in, an importable module — via:

```
1 convert = DagonTask(  
2     TaskType.NATIVE, "convert", "units_module:convert_units",  
3     inputs={"value": 18.5, "from_unit": "celsius", "to_unit": "kelvin"},  
4     outputs={},  
5 )
```

eliminating the redundant command-line adapter entirely, at the cost of the importability constraint discussed above.

Caution

This convenience is not unconditional. A function that internally launches a separate, long-running compiled simulation via its own subprocess calls gains little from **NATIVE**-task wrapping over a direct **BATCH** or **SLURM** task targeting the simulation executable itself; the JSON-serializability requirement on inputs and outputs (??) is also a genuine constraint that purely in-process, array-oriented scientific computation satisfies easily, but that functions exchanging large binary scientific data structures — raw NumPy arrays, for instance, as opposed to file paths to NetCDF files — do not satisfy directly without an explicit serialization step the function itself must perform.

13.9 Chapter Summary

TaskType.NATIVE closes Part II's progression through heterogeneous execution backends by bringing ordinary, importable Python functions into the same dependency, staging, and provenance model already established for shell commands (Part I), containers and HPC schedulers (??), and AI/web services (????). This chapter additionally explained, in terms of Python's own serialization limitations, why the module-level-importability constraint is structurally necessary rather than arbitrary, and examined where the boundary should fall between wrapping existing scientific Python code as a **NATIVE** task versus as a **BATCH** task invoking a hand-written command-line adapter. ?? now turns from the engine's task-type taxonomy to the practical discipline of wrapping real, compiled scientific software — the discipline that Part III's environmental-science case study depends on directly.

Exercise 13.1. A colleague proposes converting a research group's existing `archive2plot.py` rendering script (??) from a **BATCH** task, invoked as a standalone command-line program, into a **NATIVE** task calling its core rendering function directly. List the specific, concrete checks you would perform on the existing script before agreeing this conversion is straightforward, referencing both the importability constraint and the JSON-serializability constraint discussed in this chapter.

Chapter 14

Running External Scientific Software

Running this chapter in Google Colab

All lessons in this chapter run in Google Colab with Option 3 installation (`git clone` then `pip install -e .`, see Appendix E). Use `local_config(workdir)` instead of `tagon.ini`, and use `Path(task.get_scratch_dir())` to read task outputs instead of shell `find/cat` commands (see ?? and Appendix E for the complete guide).

Scientific workflows rarely consist only of Python functions. They typically wrap large external programs: compiled models, command-line preprocessors, domain-specific converters, visualization tools, and publication utilities. DAGonStar’s task model, developed across the preceding chapters, is designed precisely for this style of file-oriented scientific computing. This chapter collects the engineering discipline needed to wrap such software reliably, in preparation for Part III’s environmental-science case study.

14.1 External Software Prerequisites

Before placing a program inside a DAGonStar task, it must be verified *outside* DAGonStar. For each external tool, record:

- executable path;
- version;
- build/compiler information, if compiled;
- required environment variables;
- required dynamic libraries;
- required input files;
- expected output files;
- working-directory assumptions;
- license or access restrictions;
- expected runtime resources.

```

1 which wrf.exe
2 wrf.exe --help || true
3 ldd wrf.exe 2>/dev/null || otool -L wrf.exe 2>/dev/null

```

Listing 14.1: Verifying a compiled executable before integration.

```

1 python -V
2 python -m pip freeze
3 python script.py --help

```

Listing 14.2: Verifying a Python tool.

```

1 docker run --rm image:tag command --help

```

Listing 14.3: Verifying a containerized tool.

14.2 Choosing an Execution Environment

The first design decision for any new task is *which execution environment best matches the stage’s resource profile and dependency requirements*. The table below consolidates this decision, aligning the available task types with their primary scientific use case and the keyword argument that activates each backend:

Scientific Need	TaskType	Key Parameter(s)
Small, I/O-bound utility	BATCH	command string
Remote instrument / service host	BATCH + ip=	ip=, ssh_username=, keypath=
Compiled HPC model (hours-long)	SLURM	partition=, ntasks=, time=
Dependency-heavy tool (GDAL, Proj)	DOCKER	image= (pin to digest for publication)
Cloud elasticity / burst compute	CLOUD	provider name, machine flavor
AI-assisted interpretation	LLM	provider=, input_files=
REST publication / ingestion	WEB	specification dictionary
Python post-processing function	NATIVE	module:function, inputs=, outputs=

Scientific Practice Note

Assign backends *per stage* based on that stage’s actual resource and dependency profile. A common mistake is to default every stage of a pipeline to **BATCH**, even when some stages are computationally intensive enough to warrant **SLURM**, or dependency-heavy enough to warrant **DOCKER**. The staging and dependency mechanisms — and therefore the scientific guarantees — are identical across all backends; only the execution surface differs.

14.3 File-Based Data Exchange

DAGonStar’s most robust integration pattern is file exchange. Each task:

1. reads files from its working directory or staged inputs;
2. executes external software;
3. writes named output files;
4. is consumed by downstream tasks through `workflow://` references (??).

This pattern is robust precisely because most scientific software already communicates through files: NetCDF, GRIB, CSV, GeoTIFF, JSON, shapefiles, binary restart files, and logs are the native exchange formats of environmental and earth-system modeling, and DAGonStar requires no special support for any of them — only a file path.

```

1 post = DagonTask(
2     TaskType.BATCH,
3     "postprocess",
4     "python post.py --input workflow:///simulation/output/model.nc --output
      product.nc",
5 )

```

Listing 14.4: A post-processing step referencing simulation output.

14.4 Preparing Launcher Scripts

For complex software, a long command should not be embedded directly into the task string. Instead, create a small wrapper script and call it from DAGonStar.

```

1 #!/usr/bin/env bash
2 set -euo pipefail
3
4 input_file="$1"
5 output_dir="$2"
6
7 mkdir -p "$output_dir"
8
9 module purge 2>/dev/null || true
10 module load netcdf 2>/dev/null || true
11
12 export OMP_NUM_THREADS="${OMP_NUM_THREADS:-1}"
13
14 ./model.exe --input "$input_file" --output "$output_dir/result.nc"
15
16 test -s "$output_dir/result.nc"

```

Listing 14.5: `run_model.sh`: a defensive wrapper around a compiled model.

```

1 model = DagonTask(
2     TaskType.BATCH,
3     "model",
4     "bash run_model.sh workflow:///prepare/input/forcing.nc output",
5 )

```

Listing 14.6: The corresponding DAGonStar task.

14.4.1 Wrapper Script Checklist

A good wrapper script should:

- fail fast (`set -euo pipefail`);

- print useful diagnostic information;
- create required output directories;
- validate required inputs exist;
- run the external program;
- validate expected outputs exist and are non-empty;
- avoid embedding secrets;
- write logs to predictable files.

```

1 if [ ! -s "$input_file" ]; then
2     echo "Missing input file: $input_file" >&2
3     exit 2
4 fi

```

Listing 14.7: A representative input-validation block.

14.5 Working Directories

DAGOnStar changes into the task working directory before executing the command; external software should therefore use paths relative to the task directory when possible. A recommended internal layout:

```

1 task scratch/
2   input/ or .dagon/inputs/
3   output/
4   logs/
5   config/

```

14.6 Environment Modules and Activation

HPC systems commonly use environment modules:

```

1 module purge
2 module load gcc/12.2 netcdf openmpi

```

Python tools often use virtual environments:

```

1 . /path/to/venv/bin/activate
2 python tool.py

```

or Conda:

```

1 source /path/to/miniconda/etc/profile.d/conda.sh
2 conda activate environment-name

```

Scientific Practice Note

Put this setup *inside the wrapper script*, not in the DAGOnStar task command string, so that the task remains reproducible independent of the calling shell's pre-existing environment.

14.7 Slurm and Docker Wrapper Patterns

```

1 task = DagonTask(
2     TaskType.SLURM,
3     "wrf",
4     "bash run_wrf.sh workflow:///real/wrfinput_d01 output",
5     partition="compute",
6     ntasks=64,
7     time="06:00:00",
8 )

```

Listing 14.8: Slurm wrapper pattern: command focus stays on the model, resources are declared at construction time.

The generated `sbatch` command wraps DAGOnStar’s launcher script; the wrapper script itself should still validate inputs and outputs.

```

1 task = DagonTask(
2     TaskType.DOCKER,
3     "map_render",
4     "python map2png.py workflow:///post/product.nc output/map.png",
5     image="my-lab/map-render:1.0",
6 )

```

Listing 14.9: Docker wrapper pattern.

The container image must contain the executable, required system libraries, Python/R packages, fonts or geospatial data needed by renderers, and any shell tools used by wrapper scripts.

14.8 Logs and Provenance

External software should write logs that remain inside the task scratch directory:

```

1 ./model.exe "$input" "$output" > logs/model.stdout 2> logs/model.stderr

```

A complete provenance record additionally captures: command line; software version; input checksums when possible; output checksums for critical products; and the runtime environment.

14.9 Error Behavior

DAGOnStar marks a task as failed when the executed command returns an error code, or when the launcher script raises an exception. Wrapper scripts should return non-zero exit codes when *scientific* validation fails, even if the external program itself exits with status zero:

```

1 if ! ncdump -h output/result.nc >/dev/null; then
2     echo "Invalid NetCDF output" >&2
3     exit 3
4 fi

```

14.10 The Five-Stage Practical Design Pattern

For each external software component, the documentation recommends a canonical five-task decomposition:

1. `prepare_*`: creates configuration and input files.
2. `run_*`: executes the external program.
3. `validate_*`: checks scientific and structural validity.
4. `postprocess_*`: converts outputs into analysis products.
5. `publish_*`: exports final artifacts.

`workflow://` references connect each stage, so that data lineage remains explicit end to end — this five-stage pattern is precisely the decomposition Part III’s environmental application case study (??) will apply to a real numerical weather pipeline.



Figure 14.1: The canonical five-stage wrapping pattern for external scientific software, connected end to end by `workflow://` references.

14.11 Defensive Scripting as a Research-Software-Engineering Discipline

The wrapper-script checklist of ?? draws on a body of practice often called *defensive programming*, and it is worth making the underlying rationale explicit, because the checklist’s individual items can otherwise read as an arbitrary list of best practices rather than as a coherent response to a specific category of risk.

The risk that defensive wrapper scripting addresses is the gap between *a program exiting with status zero* and *a program having done what was scientifically intended*. Most compiled scientific models, including the WRF components discussed in ??, were not originally designed with workflow-engine integration in mind, and their exit-code conventions are frequently inherited from decades-old Fortran or C codebases that may return a zero exit status even after writing a truncated, corrupted, or scientifically meaningless output file, simply because the specific failure mode encountered was never anticipated by the original authors as something the calling environment would need to detect programmatically. The `set -euo pipefail` preamble, input existence validation, and output non-emptiness validation enumerated in ??’s checklist are not generic shell-scripting hygiene; each addresses a specific, named failure mode in this gap:

set -e

ensures that a failure in any *intermediate* command within a multi-step wrapper script propagates to the script’s own exit status, rather than being silently masked by a later command’s success.

set -u

converts a reference to an unset shell variable — frequently the symptom of a missing or misnamed environment-module load — from silent, wrong behavior into an immediate, loud failure.

set -o pipefail

ensures that a failure anywhere within a shell pipeline (`a | b | c`) is detected, rather than only the exit status of the pipeline’s final command, which would otherwise mask an upstream failure.

Input existence/non-emptiness validation

catches the case where an upstream `workflow://` dependency resolved structurally (??) but the referenced file is empty or malformed — a failure mode the dependency graph itself cannot detect, since it tracks task completion, not output content quality.

Output existence/non-emptiness validation

catches exactly the “zero exit status, garbage output” failure mode this section opened with, converting a silent scientific error into an explicit task failure DAGonStar’s `Status` model (??) can record and a downstream researcher can act on.

Scientific Practice Note

A useful framing for a research group adopting this discipline: every line in the wrapper-script checklist exists to convert one specific, otherwise-silent failure mode into an explicit, recorded `FAILED` task status. A wrapper script that omits a checklist item has not merely skipped a stylistic nicety; it has left one corresponding failure mode invisible to the workflow engine, and therefore invisible to whatever automated checkpointing, retry, or notification logic a researcher might build on top of task status.

14.12 The Cost of Skipping External Verification

?? opened with a prerequisites checklist — record the executable path, version, required libraries, and so on — to be completed *before* a program is placed inside a DAGonStar task. It is worth being explicit about why this ordering matters, since the temptation to skip directly to workflow integration is understandable and the consequence of doing so is not always immediately visible.

When an external program is verified manually first — run once at a terminal, with its output inspected by a human — any configuration error (a missing library, an unset environment variable, an incompatible input format) surfaces in the simplest possible debugging context: a single command, a single terminal, full visibility into standard output and standard error as they are produced. When the same program is instead integrated directly into a multi-task DAGonStar workflow without prior manual verification, an identical configuration error instead surfaces inside a generated launcher script, inside a scratch directory, captured to a file rather than a terminal, and potentially obscured by whatever upstream staging or downstream dependency behavior the workflow also exercises. The debugging procedure of ?? — read `launcher.sh`, then `context.sh`, then `stdout.txt` — remains available and effective in this second scenario, but it is strictly more work than simply having ruled the configuration error out beforehand.

Scientific Practice Note

This is the underlying rationale for the local-structural-verification pattern already introduced for Slurm and remote tasks (??), now generalized: verify the smallest possible unit — a single external program invocation, a single task’s command, a single task’s dependency resolution — in the simplest available context before composing it into a larger, harder-to-debug structure. This is not specific to DAGonStar; it is a general discipline for working with compiled scientific software in any orchestration context, and ?? will apply it explicitly to the multi-stage WRF pipeline.

14.13 When the Five-Stage Pattern Should Be Adapted

The prepare/run/validate/postprocess/publish decomposition given in ?? is presented as canonical, and it serves well for the great majority of file-oriented scientific pipelines this book addresses. It is worth being equally honest about when a researcher should deviate from it, since presenting any pattern as universally applicable without qualification would understate the judgment genuine workflow design requires.

For a pipeline whose “run” stage is itself an ensemble of many structurally identical simulations — differing only in initial conditions or parameter values, a common pattern in environmental forecasting and uncertainty quantification — the five-stage pattern is best applied *per ensemble member*, with an additional, ensemble-level aggregation stage following all individual “postprocess” tasks, rather than attempting to force the entire ensemble through a single “run” task. For a pipeline whose validation logic is genuinely inexpensive and tightly coupled to the run stage’s own internal consistency checks, collapsing “run” and “validate” into a single task may be preferable to the traceability cost of a separate validation task whose only content is a near-trivial check already effectively performed by the run stage itself — an instance of the granularity trade-off already developed in ??.

14.14 Chapter Summary

Wrapping external scientific software in DAGonStar is, in essence, a discipline of defensive shell scripting (fail fast, validate inputs and outputs, log everything) combined with the engine’s existing dependency and staging mechanisms. This chapter additionally grounded each item of the wrapper-script checklist in a specific, named failure mode it exists to convert from silent to explicit, argued for manual external verification as a strictly cheaper debugging context than in-workflow verification, and discussed concrete cases — ensemble simulations, tightly coupled run/validate logic — in which the canonical five-stage prepare/run/validate/postprocess/publish pattern should be deliberately adapted rather than applied mechanically. The five-stage pattern, and its documented exceptions, recur throughout operational scientific pipelines and form the organizing template Part III now applies to a real numerical-weather-and-ocean forecasting case study.

Exercise 14.1. Using the failure-mode framing of this chapter’s first section, identify which specific, named failure mode each of the following omissions from a wrapper script would leave undetected: (a) omitting `set -u`; (b) omitting output non-emptiness validation; (c) omitting `pipefail` when the wrapper pipes the external program’s output through a post-processing filter before writing it to disk.

Part III

Computational Environmental Science Applications and Interactive Deployment

Chapter 15

Case Study: The Environmental Application

Running this chapter in Google Colab

All lessons in this chapter run in Google Colab with Option 3 installation (`git clone` then `pip install -e .`, see Appendix E). Use `local_config(workdir)` instead of `dacon.ini`, and use `Path(task.get_scratch_dir())` to read task outputs instead of shell `find/cat` commands (see ?? and Appendix E for the complete guide).

This chapter applies the entire theoretical and mechanical apparatus developed in Parts I and II to the repository’s environmental application example — the closest the documentation comes to a complete, realistic computational environmental science workflow, and the natural capstone for a textbook “*Orchestrating AI-HPC Workflows for Science and Engineering*.”

15.1 Source and Scope

Source directory: `examples/envapp`

```
1 examples/envapp/  
2   app.py  
3   README.md  
4   models/  
5   models/wrf/  
6   models/wrf/scripts/  
7   models/wrf/utills/  
8   archive2plot/  
9   publishWrf/
```

Listing 15.1: Layout of the environmental application example.

Caution

The repository describes this material explicitly as a *mockup/sanitized demonstration* of a larger, real environmental forecasting system. It is not the production code running at the University of Naples “Parthenope” Center for Monitoring and Modeling Marine and Atmosphere applications; treat it as a structurally faithful template, not as a deployable forecasting system.

15.2 Scientific Purpose

The environmental application examples are research-oriented demonstration material inspired by real forecasting workflows involving three families of numerical models:

- **WRF** — the Weather Research and Forecasting model, an atmospheric numerical weather prediction system;
- **ROMS** — the Regional Ocean Modeling System, a free-surface, terrain-following ocean circulation model;
- **WaComM++** — a marine pollution transport-and-diffusion model used for contaminant and Lagrangian particle tracking.

15.3 Concepts Exercised

The example exercises, in combination, every architectural concept developed in this book:

- wrapping external scientific model executables (??);
- preparing model input files;
- executing model stages, potentially across BATCH, SLURM, and DOCKER backends (??);
- post-processing outputs;
- rendering archived products to plots;
- publishing products, naturally suited to a WEB task (??).

15.4 The WRF Pipeline as a Canonical Numerical-Weather Workflow

The `models/wrf/scripts/` directory includes shell-oriented wrappers representing the standard structure of a numerical weather workflow:

Script family	Role
<code>geogrid</code>	Define and prepare the model's geographical domain.
<code>ungrib</code>	Decode gridded meteorological input (e.g. GRIB) data.
<code>metgrid</code>	Horizontally interpolate meteorological data onto the domain.
<code>real</code>	Vertically interpolate and generate initial/boundary conditions.
<code>wrf</code>	Execute the core atmospheric simulation.
<code>publishWrfOutput</code>	Export and disseminate simulation products.
<code>namelist generators</code>	Produce the model's run-time configuration files.

This script family corresponds to the canonical six-stage structure of a WRF-based forecast:

1. configure domain and static geographical data;
2. prepare meteorological input;
3. generate model-ready files;
4. run initialization;

5. run simulation;
6. post-process and publish outputs.



Figure 15.1: A DAGOnStar-style DAG induced by the canonical WRF pipeline. `geogrid` and `ungrib` are independent producers consumed by `metgrid`, after which the chain becomes linear through simulation, rendering, and publication. Backend annotations indicate a representative (not prescriptive) execution-environment assignment, following ??.

15.5 Post-Processing: `archive2plot`

`archive2plot` contains a utility for rendering post-processed model output as image plots:

```

1 archive2plot/
2   archive2plot.py
3   Render.py
4   maps.json
5   archive2plot.conf
6   requirements.txt
7   (shapefile documentation)

```

In DAGOnStar terms, `archive2plot` is naturally either a **BATCH** task (if invoked as a standalone Python script with its own environment already on the host) or a **NATIVE** task (??), if its rendering logic is refactored into an importable, module-level function consuming a staged NetCDF product and producing a declared image output.

15.6 Prerequisites and Operational Honesty

The documentation is explicit that these examples may require substantial external software:

- compiled WRF and related preprocessing tools;
- geospatial Python libraries;
- NetCDF/HDF libraries;
- model input datasets;
- shapefiles or map assets;
- HPC or containerized execution environments.

Caution

These are not generic unit tests. They should be treated as scientific application *templates* — structurally complete, but requiring site-specific software, data, and credentials before they can be executed end to end. This is consistent with the project’s own examples-catalog guidance (??), which distinguishes locally verifiable examples from those requiring external software or services.

15.7 Adapting the Pattern: A Five-Step Method

The documentation’s own adaptation checklist, already previewed structurally in ??’s five-stage pattern, applies directly here:

1. identify each model executable as a task;
2. define all file products exchanged between tasks;
3. wrap each executable in a reproducible launcher script;
4. encode cross-task data products with `workflow://`;
5. verify each model stage independently before running the full workflow.

Example 15.1. Applying step 5 concretely to ??: before composing the full seven-task DAG, a researcher should first verify `geogrid` alone (does it produce the expected static geographical files?), then `ungrib` alone (does it correctly decode the chosen meteorological input source?), and only then connect them through `metgrid` via `workflow://` references. This staged verification strategy mirrors the local-structural-verification pattern already introduced for Slurm and remote tasks in ??: verify each component in isolation before trusting the composed graph.

15.8 Real-World Anchoring

The environmental application generalizes patterns from real, funded deployments that the project’s own acknowledgments document (reproduced in full in the Bibliography):

- **MytilX** — a research agreement on modelling mytilus (mussel) farming at scale, in which DAGonStar orchestrates the production workflow delivering study-zone-specific weather, marine, pollutant, and farmed-mussel contamination forecasts and predictions.
- **MytilEx** — a research contract on mussel farming with HPC and AI, in which DAGonStar orchestrates the daily production of 168 forecast hours of weather, marine, pollutant, and contamination predictions.
- **ADMIRE** (EuroHPC H2020, Work Package 7) — DAGonStar orchestrates an environmental application delivering on-demand weather, marine, and pollutant simulations and forecasts for the Campania Region, Italy.

These deployments are the operational referent for the synthesis chapter that follows: real systems in which DAGonStar coordinates exactly the kind of numerical-model, post-processing, and publication chain sketched in ??, at production cadence.

15.9 Reading the WRF Pipeline as a Layered Dependency Structure

?? presented the WRF pipeline as a DAG; it is worth examining more closely *why* the pipeline has the particular dependency shape it does, since the shape directly reflects the scientific structure of numerical weather prediction, not an arbitrary engineering choice.

`geogrid` and `ungrib` are independent of one another because they consume entirely disjoint kinds of input: `geogrid` processes static geographical data (terrain elevation, land use, soil type) that does not depend on any particular forecast initialization time, while `ungrib`

decodes time-varying meteorological fields (temperature, pressure, wind) from an external data source. Because neither task’s output depends on the other’s, the engine is free, per the topological-ordering discussion of ??, to execute them concurrently — a genuine, not merely theoretical, opportunity for parallelism that a researcher decomposing this pipeline by hand should explicitly preserve by *not* introducing an artificial explicit dependency between the two merely because they happen to be related model-preparation steps.

`metgrid` depends on *both*, because horizontal interpolation onto the model’s domain genuinely requires both the static geography `geogrid` produced and the decoded meteorological fields `ungrib` produced; this is a genuine data dependency in the sense of ??, not a procedural one, and is correctly expressed, in a DAGonStar implementation of this pipeline, through two separate `workflow://` references inside `metgrid`’s command rather than through a single combined reference. From `metgrid` onward, the chain becomes strictly linear — `real`, then `wrf`, then post-processing and publication — because each subsequent stage genuinely requires the immediately preceding stage’s specific numerical output and nothing else.

Scientific Practice Note

This analysis generalizes into a useful design heuristic for decomposing any new, unfamiliar scientific pipeline into a DAGonStar workflow: before writing any `DagonTask` code, first identify, for each candidate stage, the *minimal* set of upstream products it genuinely requires, and resist the temptation to introduce additional, unnecessary dependency edges purely because two stages are conceptually related or were historically run in a fixed order out of habit rather than necessity. Every unnecessary edge removes a genuine opportunity for the engine to exploit concurrency, exactly as an artificial `geogrid` → `ungrib` edge would needlessly serialize two genuinely independent stages of the pipeline examined here.

15.10 Resource Heterogeneity Within a Single Pipeline

A second dimension worth making explicit, beyond the dependency structure already analyzed, is the heterogeneity of *computational resource requirements* across the WRF pipeline’s stages — a heterogeneity that directly motivates the backend-selection guidance of ?? and that a researcher decomposing a similar pipeline should reason about explicitly rather than defaulting every stage to the same backend out of convenience.

`geogrid` and `ungrib` are typically I/O-bound, data- preparation steps whose resource demands are modest relative to the core simulation; running them as local `BATCH` tasks, or lightweight remote `BATCH` tasks, is usually entirely adequate. `wrf` itself, the core atmospheric simulation, is typically the pipeline’s most computationally demanding stage by a wide margin, routinely requiring many-core, multi-node parallel execution over a span of hours — precisely the profile `TaskType.SLURM` exists to address, with its explicit `ntasks`, `nodes`, and `time` resource-request parameters (??) mapping directly onto the simulation’s actual computational needs. `archive2plot`’s rendering step sits between these two extremes: more demanding than simple data preparation, since it processes the simulation’s full output volume, but typically far less demanding than the simulation itself, and frequently dependent on a specific geospatial software stack (the shapefile and mapping libraries noted in ??’s file listing) that argues for `TaskType.DOCKER` on dependency-management grounds, independent of its computational intensity.

Scientific Practice Note

The general principle this illustrates: a realistic scientific pipeline’s backend assignment should follow the *resource and dependency profile of each individual stage*, not a single, pipeline-wide default. The environmental application example’s implicit backend diversity — light local or remote tasks for preparation, Slurm for the simulation core, Docker for a specialized rendering toolchain — is not an arbitrary illustrative choice in ??; it is the natural consequence of applying ??’s selection criteria stage by stage to a pipeline whose stages genuinely differ in their computational and dependency-management needs.

15.11 From Demonstration Pipeline to Operational Deployment

?? was careful to distinguish the sanitized, demonstration `examples/envapp` material from the production deployments it generalizes (MytilX, MytilEx, ADMIRE), and it is worth examining, in more depth than the original chapter’s brief mention, what additional concerns typically separate a demonstration pipeline from an operational one running at production cadence — concerns this book’s earlier chapters have each already introduced individually, but that converge specifically at the point of operational deployment.

An operational, daily-cadence forecasting chain must additionally address: **scheduling** — something external to DAGonStar itself (a cron job, a system-level scheduler, or a controller using the asynchronous-launch pattern of ??) must trigger a new workflow run at the appropriate cadence; **failure recovery across runs**, not merely within a single run — checkpointing (??) addresses resuming a single failed run, but an operational deployment must also decide what happens if an entire day’s forecast cycle fails outright, a question outside the scope of any single workflow’s own configuration; **monitoring and alerting** — the lifecycle event hooks of ?? provide the mechanism, but an operational deployment must build the actual alerting logic on top of them; and **data retention policy** — ??’s checkpoint-persistence discussion already noted that checkpoint reuse depends on scratch directories remaining on disk, which has direct, compounding storage-cost implications once a pipeline runs daily indefinitely rather than once, for a tutorial exercise.

Caution

None of these four operational concerns is addressed by DAGonStar’s own documented feature set in isolation; each requires deliberate engineering on top of the mechanisms this book has developed. A researcher planning to move from a demonstration pipeline, of the kind examined in this chapter, to genuine operational deployment should treat this list as a minimum checklist of additional design work required, not as a list of problems DAGonStar itself solves automatically.

15.12 Chapter Summary

The environmental application example is the richest demonstration of DAGonStar’s intended domain: orchestrating heterogeneous model components and data products across scientific software stacks. This chapter additionally analyzed the WRF pipeline’s dependency structure as a direct reflection of the underlying numerical weather prediction science rather than an arbitrary engineering choice, examined the resource heterogeneity across pipeline stages that motivates differentiated backend selection, and enumerated the additional operational concerns — scheduling, cross-run failure recovery, monitoring, and data retention — that separate a

demonstration pipeline from a genuine, production-cadence operational deployment of the kind the MytilX, MytilEx, and ADMIRE acknowledgments describe. ?? now surveys the remainder of the project's example catalog, and ?? closes the book by assembling these pieces into the shape of a complete, AI-augmented, operational forecasting chain.

Exercise 15.1. Using the dependency-structure analysis of this chapter's first section, identify one further pair of stages in ??, beyond `geogrid/ungrib`, that could in principle run concurrently if the pipeline were extended to include a second, independent post-processing path (for instance, a statistical verification step computed directly from `wrf`'s output, alongside `archive2plot`'s rendering step). Sketch the resulting DAG.

Chapter 16

The Examples Catalog and Verification Strategy

The repository documents its runnable and research-oriented examples (`examples/`) as a guided catalog rather than a replacement for the source files themselves: each page explains scientific/computational purpose, the DAGonStar mechanisms exercised, prerequisites, execution model, and expected verification strategy. This chapter surveys that catalog as a reference map for readers extending this book’s lessons toward their own research problems.

16.1 Example Families

Family	Scope
Taskflow examples	Explicit dependency graphs (cf. Lesson 2, ??).
Asynchronous launch	Local background execution and lifecycle callbacks; source: <code>examples/async/asynchronous_launch.py</code> (??).
Batch dataflow examples	Local and remote dataflow, checkpointing, and <code>workflow://</code> file references (????).
Docker dataflow examples	Container-backed tasks (??).
Slurm dataflow examples	HPC scheduler-backed tasks (??).
Cloud dataflow examples	Apache Libcloud-backed virtual-machine tasks.
Globus dataflow example	Endpoint-based scientific data movement.
Transversal and meta-workflow examples	Cross-workflow dependencies and legacy transversal scripts (??).
HiPES tutorial examples	A staged scientific tutorial from configuration to map rendering.
Environmental application examples	WRF/ROMS/WaComM++-oriented demonstration material (??).
LLM task example	A local OpenAI-compatible mock and a real-provider skeleton (??).
Native Python task example	Staged files, scalar parameters, and declared outputs (??).
Web task example	Local HTTP execution, staged uploads, and response outputs (??).
Colab examples (new)	<code>examples/colab/hello_colab.py</code> and <code>workflow_files_colab.py</code> with paired <code>.ipynb</code> notebooks; credential-free, Colab- and Jupyter-runnable (??).
Tutorial notebook (new)	<code>docs/tutorial/DAGonStar_tutorial.ipynb</code> : 32 cells covering all fourteen lessons, designed for Google Colab and local Jupyter (??).
Tutorial source programs	Runnable local sources for the advanced LLM and web lessons, under <code>examples/tutorial/</code> .

16.2 Verification Levels

The documentation distinguishes four verification levels that the teaching progressions in this book follow, from the simplest to the most infrastructure-demanding:

Level 1 — Fully local, no external service

All computation and data exchange happen on the developer's own machine using only `/tmp/` storage, the Python standard library, and DAGonStar itself. Every Lesson 01–07 example, both Colab example scripts, and the unified tutorial notebook (`DAGonStar_tutorial.ipynb`) fall into this category. Running requires only the repository checkout and a Python virtual environment.

Level 2 — Local with optional infrastructure

The example requires a service that may or may not be present on a developer machine, but has a structural-verification fallback that does not require the live service. Lesson 08 (Docker daemon required; structural-only fallback available) and Lesson 09 (Slurm cluster required; `generate_command()` structural verification available) are the canonical Level 2 examples.

Level 3 — Live service integration

The example requires a reachable external service, credential, or API key to execute fully. Lesson 12 on a real LLM provider (as opposed to the bundled mock), Lesson 13 against a real HTTP endpoint, remote SSH tasks, and cloud examples are Level 3. Each has a Level 1 or Level 2 fallback (mock provider, structural verification) for learners without the relevant credential.

Level 4 — Site-specific infrastructure and data

The example requires a specific HPC cluster, dataset, or software stack that cannot be generalized across sites. The environmental application (`examples/envapp/`) is Level 4: WRF, ROMS, and WaComM++ require site-installed model executables and domain data that no mock can replace. These examples are presented as case studies, not as runnable tutorials.

Scientific Practice Note

The workshop progression in this book deliberately follows Level 1 → Level 2 → Level 3 → Level 4, so that each step builds on fully verified infrastructure. When developing a *new* DAGOnStar pipeline in practice, the same progression is recommended: build and verify the local-only skeleton first — using mock services where the LLM and Web lessons demonstrate the pattern (????) — before introducing the site-specific services (real Slurm cluster, real cloud account, real LLM provider) that the production version will ultimately require.

16.3 The Consistency Rule

The examples directory includes legacy research material alongside actively maintained tutorials. The four verification levels above — locally verifiable; optional infrastructure; live service or credential; site-specific data or software stack — constitute this book’s explicit classification, derived directly from the four categories the documentation asks contributors to declare at the point of contribution.

Caution

A reader extending these examples toward production use should explicitly classify any adapted example into one of the four verification levels and record that classification alongside the adapted workflow. This is precisely the kind of operational metadata that keeps a scientific pipeline auditable as it evolves away from its tutorial origin and toward production deployment. Failing to do so produces the well-known failure mode of research software documentation: a directory of scripts of wildly varying reliability, with no way to tell, without reading each script in full, which examples will run unmodified on a fresh checkout and which silently assume a specific cluster, credential, or dataset the original author had access to but never documented.

16.4 The Pedagogical Function of an Examples Catalog

It is worth situating the examples catalog’s organizational philosophy within a broader observation about how research software documentation tends to fail, and how a catalog of the kind ?? surveys is designed specifically to avoid that failure mode.

A common pattern in research software projects is for examples to accumulate organically, over years, as individual researchers contribute scripts addressing their own immediate needs,

with no consistent classification of what each example assumes, requires, or guarantees. The result, familiar to anyone who has inherited a research codebase, is a directory of scripts of wildly varying reliability, with no way to tell, short of reading each one in full, which examples will run unmodified on a fresh checkout and which silently assume a specific cluster, credential set, or dataset the original author had access to but never documented. The four-category consistency rule ?? reproduced — locally verifiable; requiring external software; requiring remote services or credentials; historical, retained for context — exists precisely to prevent this accumulation from becoming opaque: every example is required to declare, explicitly, which category it belongs to, so that a reader can decide, before investing time, whether a given example is even attemptable in their current environment.

Scientific Practice Note

This four-category discipline is worth adopting independently of DAGonStar, as a general practice for any research group maintaining a shared example or script collection: require every contributed example to declare its verification category at the point of contribution, rather than leaving later readers to discover the category empirically, through failed attempts to run code whose external requirements were never documented.

16.5 Mock Services as a Verification Strategy

The staged verification strategy of ?? — local examples first, site-dependent examples second — depends critically on a technique this book has used repeatedly without naming it explicitly until now: the *mock service*. Lessons 12 and 13’s locally verifiable LLM and Web examples (????) both achieve local verifiability by substituting a small, purpose-built HTTP server — bound only to 127.0.0.1, requiring no credential, implementing only the minimal subset of a real provider’s protocol surface needed for the example — for the real, external service a production workflow would ultimately target.

This technique deserves recognition as a general-purpose strategy, not merely an implementation convenience specific to the tutorial. A mock service verifies the *mechanism* — dependency inference, staging, request construction, response capture — entirely independently of the *content* a real provider would return, which is exactly the separation of concerns a researcher needs when developing or debugging a new LLM- or WEB-task-based workflow stage: mechanism bugs (a malformed request, a missing dependency, an incorrectly staged file) can be found and fixed against the mock, with zero cost and zero external dependency, before ever spending a real API quota or risking a real, possibly non-idempotent, side effect against a production endpoint.

Example 16.1. A researcher developing a new WEB-task-based integration with an unfamiliar third-party API can profitably begin by writing a mock server that returns a fixed, known-good response matching the target API’s documented response schema, developing and debugging the DAGonStar task specification entirely against this mock, and only substituting the real endpoint URL — with no other change to the task specification, by the provider-agnostic design principle already discussed in ?? — once the mechanism is confirmed correct. This mirrors exactly the relationship between Lesson 12’s `local_mock` provider and Lesson 12’s documented path to a real provider (??, Step 6).

16.6 Reading an Unfamiliar Example Critically

Building on the recommended reading order of ??, it is worth giving a concrete procedure for how a reader should approach any *individual* example in the catalog, beyond simply running

it, since the goal of reading an example is generally to extract a reusable pattern for one’s own workflow, not merely to confirm that the example itself executes successfully.

1. **Identify the verification category first** (??), to calibrate what can be assumed about the example’s reliability and portability before reading further.
2. **Identify which task types and staging modes the example exercises**, using the vocabulary developed across Parts I and II of this book, to situate the example relative to material already covered.
3. **Identify the dependency structure**, ideally by sketching the DAG the example constructs, before reading the task bodies in detail — the structural skeleton is usually more transferable to a reader’s own problem than the specific commands inside any one task.
4. **Identify what the example deliberately omits**, particularly for examples in the “requiring external software” or “requiring remote services” categories, where production-readiness concerns (??’s operational-deployment discussion is directly relevant here) are frequently out of scope by design, for the sake of pedagogical focus.

16.7 Chapter Summary

The examples catalog is organized around the same staged, local-first verification philosophy this book has followed since ??: prove a mechanism with no external dependency, then extend it toward the real service it is meant to integrate with. This chapter additionally situated the catalog’s four-category consistency rule within a broader critique of how research-software examples typically accumulate without documentation, named and generalized the mock-service verification technique already used implicitly by the LLM and Web task lessons, and gave a concrete, four-step procedure for critically reading any unfamiliar example in the catalog for transferable patterns rather than merely confirming its successful execution. ?? now closes the book by assembling every mechanism surveyed across Parts I–III into a coherent picture of an operational, reproducible, AI-augmented forecasting chain.

Exercise 16.1. Apply the four-step reading procedure of this chapter to the HiPES tutorial examples family (??’s example-family table), based only on its one-line catalog description (“a staged scientific tutorial from configuration to map rendering”). What predictions can you make about its likely task-type composition and dependency structure before consulting the source material directly, and what aspects of the four-step procedure necessarily require reading the actual source rather than the catalog description alone?

Chapter 17

Toward Operational AI–HPC Forecasting Chains

Running this chapter in Google Colab

All lessons in this chapter run in Google Colab with Option 3 installation (`git clone` then `pip install -e .`, see Appendix E). Use `local_config(workdir)` instead of `daemon.ini`, and use `Path(task.get_scratch_dir())` to read task outputs instead of shell `find/cat` commands (see ?? and Appendix E for the complete guide).

This closing chapter does not introduce new DAGonStar mechanisms; it assembles the mechanisms developed across Parts I–III into a single, coherent architectural picture, and uses that picture to revisit, explicitly, the workshop abstract that motivates this book: *“orchestrating AI-HPC workflows for computational environmental science.”*

17.1 The Argument of the Book, Restated

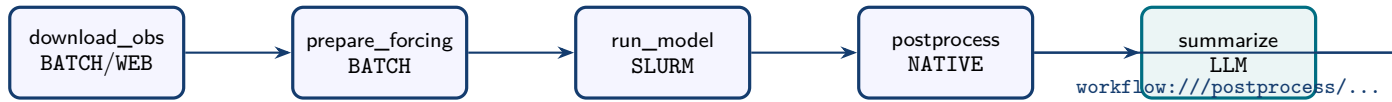
Three claims have been developed, in order, across the preceding sixteen chapters:

1. **A workflow is a DAG of tasks, expressed either explicitly or inferred from data references (???)**. This single abstraction — realized mechanically by the architecture of ?? and exercised incrementally through Lessons 1–7 (?????) — already delivers reproducibility, traceability, portability, parallelism, and recoverability for purely local, purely numerical pipelines.
2. **Heterogeneous execution environments share this same dependency model**. Containers, remote SSH hosts, and HPC schedulers (??) plug into the identical **Task/Stager** infrastructure; meta-workflows and asynchronous launch (??) compose and operationalize multiple such DAGs without altering that infrastructure.
3. **AI and web services are first-class DAG nodes, not external add-ons**. LLM, WEB, and NATIVE tasks (?????) consume and produce `workflow://`-addressed files with exactly the same dependency, staging, and provenance discipline as a Slurm-submitted numerical model.

The environmental application case study (??) is the point at which all three claims operate simultaneously on a single, realistic pipeline.

17.2 Anatomy of an Operational, AI-Augmented Forecasting Chain

?? synthesizes the book’s running examples into the shape of a complete operational chain: observation ingestion, numerical modeling, post-processing, AI-assisted interpretation, and publication, each stage realized by the task type best suited to it, all connected by `workflow://` references, and the whole chain launched asynchronously so that a controlling service remains responsive while it runs.



Launched with `workflow.launch() / wait() (??)`; `on_workflow_end` triggers downstream notification once publish completes.

Figure 17.1: A complete, AI-augmented operational forecasting chain, synthesizing every task type covered in this book: data ingestion, simulation, post-processing, AI-assisted summarization, and publication, all connected through `workflow://` and launched asynchronously.

17.3 Why This Architecture Suits Science and Engineering Broadly

Three properties of computational environmental science specifically motivate the DAGonStar design choices recapitulated above:

Heterogeneity is intrinsic, not incidental.

A real forecasting chain genuinely spans local preprocessing, HPC-scheduled simulation, containerized rendering tools, and — increasingly — AI services for summarization, anomaly detection, or decision support. A workflow engine that privileged only one of these environments would force the scientist to script the boundary between environments by hand, exactly the implicit, unauditable practice ?? opened by rejecting.

File-based exchange is the scientific software lingua franca.

NetCDF, GRIB, CSV, and shapefiles already mediate between WRF, ROMS, and WaComM++-class models (??); the `workflow://` schema (??) requires no new exchange format, only a uniform way to *name* a file produced by another stage.

Operational cadence demands recoverability and non-blocking control.

A daily or sub-daily forecast cycle, of the kind documented for the MytilEx and ADMIRE deployments (??), cannot afford to recompute an entire chain after a transient failure in one stage, nor can its controlling service block while the chain executes. Checkpointing (??, Appendix D) and asynchronous launch (??) directly address these two operational constraints.

17.4 A Methodological Checklist for New AI-HPC Workflows

Synthesizing the scientific-practice notes distributed across every preceding chapter into a single checklist:

1. Decompose the pipeline into named stages following the prepare/run/validate/postprocess/publish pattern (??).
2. Choose, per stage, the narrowest task type that matches its operational need (????): **BATCH** for small utilities, **SLURM** for compiled HPC models, **DOCKER** for dependency-heavy tools, **NATIVE** for in-process Python computation, **LLM** for AI-assisted interpretation, **WEB** for publication and external APIs.
3. Prefer `workflow://` data dependencies over explicit ones whenever a real file product is exchanged (??); use explicit `add_dependency_to()` only for genuinely procedural ordering.
4. Verify each stage in isolation before composing the full DAG, following the local-structural-verification pattern (????).
5. Configure secrets exclusively through environment-variable indirection — `api_key_env` for LLM providers, `token_env/username_env/password_env` for Web tasks — and never commit literal credentials (??????).
6. Enable checkpointing with stable workflow and task names for any stage expensive enough to warrant resumability (??).
7. Launch operational chains asynchronously and react to lifecycle events rather than polling (??).
8. Record, alongside the workflow source, the configuration values, container image digests, model/provider names, and software versions that affect scientific reproducibility (??).

17.5 Four Operational Gaps Beyond DAGonStar

The synthesis architecture in ?? represents what DAGonStar provides directly. Any production deployment of a daily forecasting chain built on this architecture must additionally address four operational concerns that the engine itself does not resolve:

Scheduling.

Something external must trigger each daily workflow execution. DAGonStar provides the orchestration mechanism; it does not provide the trigger. Common approaches include `cron` or a system-level scheduler for time-driven chains, an event-driven trigger (a data-arrival notification from an observation system), and a workflow management platform (Apache Airflow, Luigi, or similar) that wraps DAGonStar as an execution step. The `launch()` / `wait()` pattern of ?? is designed precisely for integration as a non-blocking step inside a larger scheduling framework.

Cross-run failure recovery.

Checkpointing (??) handles within-run recovery: a failed task on a given day can be rerun from the last successful checkpoint without recomputing preceding stages. Cross-run failure recovery — what happens if an entire day's run fails, or if a partial failure leaves a published product in an inconsistent state — requires application-level logic that the workflow source alone cannot provide: a retry policy, a dead-letter queue, a consistency check on previously published products, or a compensating transaction.

Monitoring and alerting.

The `on_workflow_end` lifecycle hook (??) provides the integration point for monitoring: a callback registered on this event can log completion metadata, report to a monitoring service, or send an alert if the chain finished in the **FAILED** state. However, the DAGonStar

lifecycle event system itself does not persist logs, expose metrics, or connect to a monitoring backend; these integrations require application code in the callbacks.

Data retention policy.

Daily operational runs accumulate scratch directories at a rate proportional to the per-run scratch volume multiplied by the number of tasks. Without an explicit retention policy, this accumulation can exhaust available storage within days on an active production system. The `remove_dir = True` configuration option (??) instructs the engine to remove scratch directories after the workflow completes, at the cost of losing the on-disk provenance record that ?? described as one of DAGonStar’s most valuable scientific properties. A well-designed operational system archives the provenance-relevant files (specifically `launcher.sh`, `context.sh`, and `stdout.txt`) to durable storage before setting `remove_dir = True`, so that the storage cost is bounded while the scientific record is preserved.

None of these four gaps undermines the value of the DAGonStar-based architecture; each is a natural consequence of the division of responsibility between an orchestration engine and the broader operational system that embeds it. A mature scientific workflow deployment addresses all four explicitly.

17.6 Revisiting the Book’s Three Claims Against Their Documented Limits

Academic honesty requires that a synthesis chapter not merely restate its argument’s strongest form, but also revisit, explicitly, the documented limitations that qualify it — several of which this book has reproduced, without softening, at the point each was originally relevant. It is worth gathering them here, in one place, alongside the three claims they qualify, so that the picture ?? presents is read with appropriate calibration rather than as an unqualified endorsement.

The first claim — that a workflow is a DAG of tasks, with dependencies explicit or inferred — is qualified by the documented constraints of ??: some command strings are still assembled by concatenation, with shell-quoting hardening an ongoing, not yet complete, priority; and the project’s own test coverage, while including checkpoint reuse, staging, and command-failure propagation, is explicitly described as a starting point rather than a complete quality gate.

The second claim — that heterogeneous execution environments share one dependency model — is qualified by the meta-workflow caveats of ??: the `DAG_TPS` cross-workflow composition mechanism central to ??’s natural extension to observation-ingestion and model-execution as separate workflows is explicitly flagged as legacy-sensitive and less thoroughly tested than single-workflow execution, and production use is recommended only after workflow-specific verification.

The third claim — that AI and web services are first-class DAG nodes — is qualified most directly by ??’s epistemic-status discussion: the engine provides no mechanism for assessing the factual reliability of an LLM provider’s output, and the full-response-retention design exists precisely because that assessment remains, irreducibly, a human responsibility.

Caution

None of these qualifications undermines the book’s central architectural argument; each is a documented, specific limitation of the current implementation or of a particular task type’s epistemic guarantees, not a flaw in the underlying design principle of uniform, DAG-structured task composition. Presenting the argument without these qualifications,

however, would constitute exactly the kind of selectively favorable account ??’s discussion of architectural honesty (??) explicitly argued against.

17.7 What Generalizes Beyond DAGonStar Specifically

A final, deliberately broader question is worth addressing before this book’s closing remarks: which of the methodological lessons developed across seventeen chapters are specific to DAGonStar’s particular implementation choices, and which would transfer to a reader evaluating, or even designing, a different workflow orchestration system entirely?

The following generalize, in this book’s assessment, well beyond any single engine’s implementation: the topological-ordering argument for why acyclicity is the precise precondition for any valid schedule (??); the task-granularity trade-off between traceability/recoverability and staging/management overhead (??); the defensive-scripting discipline for wrapping external scientific software reliably (??); the distinction between location-transparent, embedded-reference dependency expression and configuration-file or timestamp-based alternatives (??); and the epistemic-status discussion of embedding AI inference inside a scientific pipeline (??), which applies to any workflow engine’s AI-integration feature, present or future, not only to DAGonStar’s specific `TaskType.LLM`.

The following are comparatively more specific to DAGonStar’s particular implementation choices, and a reader evaluating an alternative engine should expect analogous but not identical mechanisms: the precise `workflow://` URI grammar (??), though the underlying naming-scheme concept it implements is general; the thread-based, rather than process-based, task-isolation model (??), which is a specific engineering trade-off other engines have resolved differently; and the specific factory-pattern dispatch mechanics of `DagonTask` (??), though the general principle of a uniform authoring surface over heterogeneous backends is, again, broadly transferable.

Scientific Practice Note

A reader who has worked through this book primarily to evaluate DAGonStar for a specific research project, rather than to learn it for its own sake, may find this distinction directly useful: the generalizable lessons above are worth retaining regardless of which engine is ultimately adopted, while the DAGonStar-specific mechanics are worth treating as one well-documented, thoroughly worked example of how those general lessons can be realized in a concrete, lightweight, Python-native implementation.

17.8 Closing Remarks

DAGonStar’s central methodological contribution, as this book has aimed to demonstrate, is not any single execution backend but the *uniformity* with which it treats them: a shell command, an `sbatch` submission, a Docker container, an HTTP request, an LLM completion, and a native Python function call are, from the standpoint of the dependency graph, the staging mechanism, and the checkpoint system, the same kind of thing — a task with inputs, an execution, and outputs, situated inside a DAG that can be inspected, validated, resumed, and audited. For computational environmental science, where a single forecast cycle may genuinely require all of these execution environments in sequence, this uniformity is precisely what turns a collection of separately maintained scripts into a single, reproducible, operational, AI-augmented scientific workflow.

Chapter 18

Interactive Workflow Development: Jupyter Notebooks and Google Colab

A notebook is an interactive front end for DAGonStar; backend requirements still apply to Docker, SSH, Slurm, cloud, and staging integrations.

*DAGonStar documentation,
docs/jupyter_notebook.md*

The master branch of DAGonStar introduces two dedicated integration guides and a set of accompanying examples that lower the barrier to entry for interactive and educational use: a Jupyter Notebook guide (`docs/jupyter_notebook.md`), a Google Colab guide (`docs/colab.md`), two ready-to-run Colab example scripts with paired `.ipynb` notebooks (`examples/colab/`), and a comprehensive 32-cell tutorial notebook (`docs/tutorial/DAGonStar_tutorial.ipynb`) that covers all fourteen lessons in a single, self-contained, Colab-ready document. This chapter develops the concepts, practical setup, and limitations associated with this new interactive layer.

18.1 Motivation: Notebooks as a Workflow Authoring Environment

The existing tutorial sequence (Lessons 01–14) was designed as a collection of standalone Python scripts, each run from a terminal. This model is entirely appropriate for production workflows, but it creates a friction point in two pedagogical contexts where DAGonStar is increasingly used: classroom teaching, where an instructor benefits from being able to walk through lessons in a single, ordered, executable document without managing multiple files; and interactive research development, where a researcher wants to inspect a workflow’s intermediate outputs in-line, alongside prose commentary, rather than switching between a script editor and a terminal.

Jupyter and Colab address both contexts, and DAGonStar’s kernel-independent Python API slots into them naturally: every workflow construction, dependency resolution, execution, and output inspection operation is a Python function call that works identically inside a notebook cell and inside a script. The new documentation formalizes this compatibility and identifies its limits, rather than treating notebooks as an unofficial use mode that simply happens to work.

18.2 Installing DAGonStar in a Notebook Environment

18.2.1 Local Jupyter Notebook Setup

The docs/jupyter_notebook.md guide specifies the following installation path for a local JupyterLab or Jupyter Notebook deployment:

```

1 python3 -m venv .venv
2 source .venv/bin/activate           # Linux / macOS
3 python -m pip install --upgrade pip
4
5 # Choose one installation path:
6 pip install dagonstar               # PyPI stable release
7 # OR
8 pip install git+https://github.com/DagOnStar/dagonstar.git
9 # OR (for development / examples that need repo files)
10 git clone https://github.com/DagOnStar/dagonstar.git
11 cd dagonstar && pip install -e .
12
13 pip install notebook jupyterlab ipykernel
14 python -m ipykernel install --user \
15     --name dagonstar \
16     --display-name "Python (DAGonStar)"
17 jupyter lab

```

Listing 18.1: Creating a dedicated kernel for DAGonStar.

The dedicated kernel isolates DAGonStar’s dependencies from the system Python and from other project kernels. After launching JupyterLab, select **Python (DAGonStar)** and verify the installation in a cell:

```

1 import dagon
2 print("DAGonStar import OK")

```

Listing 18.2: Kernel verification cell.

Scientific Practice Note

Note that the documentation now documents a **PyPI release** (`pip install dagonstar`) as the primary installation path for end-users who do not need the full repository tree. This is a new capability relative to the `development-native-web` branch, which required a source checkout. The PyPI release is appropriate for notebook experimentation; the editable source checkout (`pip install -e .`) remains recommended for development or for lessons whose code depends on files from the `examples/` directory.

18.2.2 Google Colab Setup

docs/colab.md provides three installation patterns for a hosted Colab runtime:

```

1 # Option 1: PyPI stable release
2 !pip install -U pip
3 !pip install dagonstar
4
5 # Option 2: latest repository version (no local files)
6 !pip install git+https://github.com/DagOnStar/dagonstar.git
7
8 # Option 3: full checkout (when examples/ or docs/ are needed)
9 !git clone https://github.com/DagOnStar/dagonstar.git
10 %cd dagonstar

```

```
11 !pip install -e .
```

Listing 18.3: Colab installation options.

Option 3 is required for lessons that reference files within the repository tree, such as the LLM and Web tutorial scripts:

```
1 !python examples/tutorial/lesson_12_llm_tasks.py
2 !python examples/tutorial/lesson_13_web_tasks.py
```

Listing 18.4: Running tutorial scripts from a Colab cell.

18.3 Persistence: Scratch Directories and Checkpoints in a Notebook

A fundamental constraint of both local notebooks and hosted Colab runtimes is the lifetime of scratch directories. By default, DAGonStar creates task directories under the `scratch_dir_base` configured in `dagon.ini` (or in the in-memory configuration dictionary) — which defaults to `/tmp/`. This is entirely safe for scripts, where the process exits and the user inspects results immediately. In a notebook, however, two additional scenarios must be considered:

- **Long-running interactive sessions:** a kernel that stays alive for hours may accumulate many scratch directories in `/tmp/`, which the OS may clean between notebook runs without any DAGonStar action. Checkpoint files that reference these directories become invalid.
- **Colab runtime resets:** a hosted Colab runtime is ephemeral by design; everything in `/content/` except mounted Google Drive is discarded when the runtime resets or disconnects.

The documentation's recommended mitigation is to use an explicit, project-specific `scratch_dir_base` that survives runtime lifetime:

```
1 from google.colab import drive
2 drive.mount('/content/drive')
3 WORKDIR = "/content/drive/MyDrive/dagonstar-experiments"
4
5 config = {
6     "batch": {
7         "scratch_dir_base": WORKDIR,
8         "run_base": "",
9         "remove_dir": "False",
10    },
11    "dagon_service": {"use": "False"},
12    "ftp_pub": {"ip": "localhost"},
13 }
```

Listing 18.5: Directing scratch to a durable location in a Colab notebook.

In a local Jupyter notebook, the equivalent is to set `scratch_dir_base` to a project directory that persists across kernel restarts, rather than using the default `/tmp/`.

18.4 The Colab Example Scripts

The new `examples/colab/` directory provides two self-contained, credential-free example workflows specifically designed for teaching and Colab demonstration:

18.4.1 hello_colab.py / hello_colab.ipynb

A three-task chain (`write_hello` → `decorate_hello` → `validate`) that demonstrates:

- In-memory configuration (no `dagon.ini` required);
- a `workflow://` reference from `decorate_hello` to `write_hello`'s output;
- accessing a task's scratch directory path from the calling code with `task.get_scratch_dir()`;
- reading a task output file via `pathlib.Path`.

```

1 def build_workflow(workdir: Path) -> Workflow:
2     workflow = Workflow("ColabHello", config=workflow_config(workdir))
3     workflow.add_task(DagonTask(
4         TaskType.BATCH, "write_hello",
5         "printf 'hello from DAGonStar\n' > hello.txt"))
6     workflow.add_task(DagonTask(
7         TaskType.BATCH, "decorate_hello",
8         "sed 's/$/ from a dependent task/' "
9         "workflow:///write_hello/hello.txt > result.txt",
10    ))
11    workflow.add_task(DagonTask(
12        TaskType.BATCH, "validate",
13        "grep -q 'dependent task' "
14        "workflow:///decorate_hello/result.txt",
15    ))
16    workflow.make_dependencies()
17    return workflow
18
19 def main() -> int:
20     workdir = Path(tempfile.mkdtemp(prefix="dagonstar-colab-"))
21     workflow = build_workflow(workdir)
22     workflow.run()
23     result = Path(workflow.tasks[1].get_scratch_dir()) / "result.txt"
24     print(result.read_text(encoding="utf-8").strip())
25     return 0

```

Listing 18.6: `hello_colab.py`: key structure.

Note the use of `workflow.tasks[1].get_scratch_dir()` to obtain the scratch directory path programmatically from the calling code after execution. This is the standard pattern for output inspection in notebook contexts, replacing the manual `find /tmp ...` command used in terminal-based lessons.

18.4.2 workflow_files_colab.py / workflow_files_colab.ipynb

A three-stage CSV pipeline (`prepare_data` → `transform_data` → `summarize_data`) that demonstrates the data-oriented dependency style on realistic, multi-step scientific data processing:

- `prepare_data`: writes a two-row CSV with `printf`;
- `transform_data`: consumes it via `workflow:///prepare_data/input.csv` and doubles the value column with `awk`;
- `summarize_data`: consumes the transformed CSV via a second `workflow://` reference and computes the column total.

This example is more representative of typical scientific data-processing workflows than the `hello_colab` greeting example, and better demonstrates the three-node linear chain that characterizes most preprocessing → model → postprocessing pipelines.

18.5 The Tutorial Notebook

`docs/tutorial/DAGonStar_tutorial.ipynb` is a 32-cell notebook that covers all fourteen lessons in order. Each lesson consists of one markdown cell (objective and concept) and one or more code cells. The notebook is designed for a fresh Colab runtime but also works in a local Jupyter kernel. Several lessons require special treatment in the notebook context:

Lesson	Notebook-specific notes
L04 (Scratch dirs)	Uses <code>task.get_scratch_dir()</code> and <code>pathlib</code> for output reading rather than <code>find</code> and <code>cat</code> shell commands.
L06 (Checkpointing)	Checkpoint file written to the <code>tempfile</code> -provided <code>workdir</code> ; use Google Drive for persistence across resets.
L08 (Docker)	Not supported in hosted Colab (no Docker daemon). The notebook markdown cell says so explicitly; run on a local workstation instead.
L09 (Remote/Slurm)	Client-only in Colab. The code cell performs local structural verification only: <code>task.generate_command()</code> is called but <code>workflow.run()</code> is not.
L12–L14 (LLM, Web, Native)	All three run fully in Colab via the repository tutorial scripts or equivalent inline cells.

18.5.1 The `local_config()` Pattern

The tutorial notebook introduces and standardizes a helper function `local_config(workdir)` that encapsulates the in-memory configuration dictionary used throughout all notebook cells:

```

1 def local_config(workdir):
2     return {
3         "batch": {
4             "scratch_dir_base": str(workdir),
5             "run_base": "",
6             "remove_dir": "False",
7             "threads": "1",
8         },
9         "dagon_service": {"use": "False"},
10        "ftp_pub": {
11            "ip": "localhost",
12            "user": "anonymous",
13            "password": "",
14        },
15    }
```

Listing 18.7: Standard in-memory configuration helper used throughout the tutorial notebook.

This is a direct formalization of the in-memory configuration pattern already used by the LLM, Web, and Native lesson examples (??????), now extended with a `workspace()` helper that creates a fresh subdirectory for each lesson, preventing cross-lesson scratch-directory interference when cells are re-run:

```

1 def workspace(name: str) -> Path:
2     base = Path(tempfile.mkdtemp(prefix="dagonstar-nb-"))
3     (base / name).mkdir(parents=True, exist_ok=True)
4     return base / name
```

Listing 18.8: `workspace()` helper used in the tutorial notebook.

18.6 Capability Matrix: What Works in Each Environment

The `docs/colab.md` guide provides an explicit capability matrix, reproduced and extended here to cover both Colab and local notebook contexts:

Table 18.1: DAGonStar capability support in notebook execution environments.

Capability	Colab (hosted)	Local Jupyter
Local BATCH and BASH tasks	Fully supported	Fully supported
<code>workflow://</code> references	Fully supported	Fully supported
LLM tasks (mock provider)	Fully supported	Fully supported
Web tasks (local echo server)	Fully supported	Fully supported
Native Python tasks	Fully supported	Fully supported
Checkpointing	Supported with care*	Fully supported
Asynchronous <code>launch()</code>	Supported	Supported
Docker tasks	Not supported	Supported (if daemon available)
Remote SSH execution	Limited (needs reachable host)	Supported (if SSH available)
Slurm scheduler	Client-only	Supported (if cluster accessible)
Cloud tasks	Possible (needs credentials)	Supported
Long production workflows	Not recommended	Supported

*Use Google Drive for checkpoint and scratch persistence across runtime resets.

Caution

Hosted Colab runtimes are ephemeral. A workflow that runs across multiple Colab sessions without persistent storage (Google Drive) cannot rely on checkpoint reuse, because the scratch directories referenced in the checkpoint JSON are deleted when the runtime resets. The documentation's explicit recommendation is: *for any workflow with expensive, resumable work, either use Google Drive for scratch storage, or run on a local Jupyter kernel connected to a workstation with durable storage.*

18.7 Security Considerations in Notebook Contexts

The new documentation addresses security explicitly, because notebooks introduce specific credential-exposure risks that are absent from purely script-based workflows. The documentation states the rule clearly and without exception: *never commit secrets or paste permanent private keys into a notebook.*

In the context of DAGonStar's task types, this means:

- **LLM tasks:** use `api_key_env` in the configuration, and set the named environment variable at the kernel level (or via Colab Secrets) rather than pasting the key into a cell.
- **Web tasks:** use `token_env`, `username_env`, or `password_env` rather than hard-coding credentials in the request specification.

- **SSH and Slurm:** use short-lived credentials and verify connectivity outside the notebook before wiring tasks; private key paths go in the configuration, not in cell source.
- **Cloud tasks:** use provider credentials from environment variables or credential files outside the repository, never in the notebook source.

Scientific Practice Note

The Colab Secrets feature (Tools → Secrets in the Colab menu) provides an environment-variable-like mechanism for storing sensitive values across sessions without embedding them in cell source. For LLM tasks, the workflow: set `api_key_env=MY_LLM_KEY` in the configuration dictionary, then in Colab: add `MY_LLM_KEY` as a secret, and retrieve it with `from google.colab import userdata; os.environ["MY_LLM_KEY"] = userdata.get("MY_LLM_KEY")` before `workflow.run()`.

18.8 Output Inspection Patterns in Notebooks

Notebook execution requires replacing terminal commands (`cat`, `find`, `ls`) with Python-native output reading. The new example scripts demonstrate the standard pattern:

```
1 from pathlib import Path
2
3 # After workflow.run():
4 task_dir = Path(task.get_scratch_dir())
5
6 # List the task's outputs
7 print(list(task_dir.iterdir()))
8
9 # Read a specific output file
10 content = (task_dir / "result.txt").read_text(encoding="utf-8")
11 print(content)
12
13 # Read a checkpoint
14 checkpoint_path = workdir / "checkpoint.json"
15 print(checkpoint_path.read_text()[:500])
16
17 # Inspect the generated launcher
18 launcher = task_dir / ".dagon" / "launcher.sh"
19 print(launcher.read_text())
```

Listing 18.9: Standard Python-native output inspection in a notebook cell.

This pattern is directly analogous to the three-file debugging procedure developed in ?? (`launcher.sh`, `context.sh`, `stdout.txt`), expressed in Python rather than in shell commands — consistent with the notebook’s preference for keeping all interaction inside Python cells.

18.9 The PyPI Release: A New Installation Path

The `docs/jupyter_notebook.md` and `docs/colab.md` guides both document a PyPI release (`pip install dagonstar`) as the primary installation path for users who do not need the full repository tree. This is a significant change relative to the `development-native-web` branch, which documented only the source-checkout path. The coexistence of three installation paths — PyPI release, `pip install git+...`, and editable source checkout — maps directly onto three user roles:

User role	Installation path	When to use
End-user / student	<code>pip install dagonstar</code>	Stable, packaged release
Latest features	<code>pip install git+...</code>	No checkout needed
Developer / examples	<code>pip install -e .</code>	Repository checkout

For the classroom use-case that motivated this chapter, the Colab installation pattern of Option 3 (full checkout) is almost always the appropriate choice, because the tutorial scripts and example notebooks depend on files within the repository tree (e.g. `examples/tutorial/lesson_12_llm_tasks.py`).

18.10 Chapter Summary

The master branch adds a complete interactive-execution layer to DAGonStar: a PyPI package release as a first installation path; a local Jupyter Notebook setup guide; a Google Colab guide with explicit capability matrix and persistence guidance; two self-contained Colab example scripts with paired `.ipynb` notebooks; and a 32-cell tutorial notebook covering all fourteen lessons in a single, Colab-runnable document. The core mechanisms — `workflow://` dependencies, in-memory configuration, task lifecycle, and output inspection — are identical to the script-based lessons of Parts I and II, with three adaptations specific to the notebook context: the `local_config()/workspace()` helper pattern, Python-native output reading via `task.get_scratch_dir()` and `pathlib.Path`, and explicit guidance on which backends (Docker, long production workflows) are unsupported in hosted Colab.

Exercise 18.1. Using the `workflow_files_colab.py` example as a template, add a fourth task (`validate_summary`) that reads the `workflow:///summarize_data/summary.txt` output, checks that the total reported is the expected value ($12 = (2 + 3) \times 2$), and exits with a non-zero status if it is not. Implement this as a notebook cell using the `local_config()` helper, and inspect the generated `launcher.sh` using the `pathlib` pattern of `??`.

Part IV

Advanced Capabilities: FAIR, Interoperability, and Extended Infrastructure

Chapter 19

FAIR by Design

A workflow is a scientific method. Its metadata should be as rigorous as its code.

*DAGonStar documentation,
docs/fair_principles.md*

The *FAIR principles* — Findable, Accessible, Interoperable, Reusable — were articulated for research data and have since been extended to research software and computational workflows. DAGonStar implements FAIR support as an **opt-in, native workflow capability**. Calling `Workflow.enable_fair(FairProfile(...))` attaches a `FairRecorder` to the same lifecycle events that drive execution; it does not alter task scheduling, staging, commands, or behavior for workflows that do not opt in.

19.1 Design Philosophy

The recorder is deliberately **local and dependency-free**: it records machine-readable metadata *next to a run* rather than copying data or publishing it to a remote repository. This reflects an important epistemic boundary: the recorder can substantiate what it locally observes — command hashes, artifact sizes, SHA-256 fixity, lifecycle timestamps, dependency edges — but cannot claim global findability, institutional accessibility, or long-term preservation, which require repository infrastructure and governance that are external to the workflow engine.

Local FAIR recording is therefore a *necessary precondition* for any subsequent repository deposit, not a substitute for it.

19.2 The Opt-In API

19.2.1 FairProfile

```
1 from dagon.fair import FairProfile, Agent, AccessPolicy, Artifact
2
3 profile = FairProfile(
4     title="WRF Forecast Chain",
5     description="72-hour NWP workflow for the Campania region.",
6     creators=[Agent(
7         name="Raffaele Montella",
8         orcid="0000-0003-3401-8432",
9         affiliation="University of Naples Parthenope",
10    )],
```

```

11     license="https://creativecommons.org/licenses/by/4.0/",
12     keywords=["NWP", "WRF", "environmental modeling"],
13     access=AccessPolicy(metadata="open", data="restricted"),
14     strict=False,
15 )
16 workflow = Workflow("ForecastChain", config=config)
17 recorder = workflow.enable_fair(profile)

```

Listing 19.1: Enabling FAIR recording on a workflow.

Agent supports orcid, ror, email, and affiliation. AccessPolicy records the meta-data/data access assumption without copying or restricting data. The optional `strict=True` flag causes the recorder to fail early when the profile lacks a title, description, creators, or license.

19.2.2 Task-Level Declarations

Tasks declare intentional inputs and outputs through a chainable API:

```

1 run_model.declare_inputs(Artifact(
2     "workflow:///prepare/wrfinput_d01",
3     media_type="application/x-netcdf",
4 )) \
5 .declare_outputs(Artifact(
6     "output/wrfout_d01",
7     media_type="application/x-netcdf",
8     identifier="https://doi.org/placeholder",
9 )) \
10 .annotate(description="WRF simulation", parameters={"ntasks": 64})

```

Listing 19.2: Task-level FAIR declarations.

These declarations remain **inert** until the owning workflow has FAIR mode enabled. A `workflow://` reference in a declared input is recognized as a producer/consumer relationship and exported as a provenance edge when `on_dependencies_made` fires.

19.3 Lifecycle Capture

At workflow start, the recorder creates a UUID-based run identifier and output directory, recording workflow/profile metadata, DAGOnStar version, Python/platform summary, configured data and stager movers, and an allowlisted subset of environment variables (only when `FairProfile.capture_environment` is set; names containing `TOKEN`, `SECRET`, `PASSWORD`, `PRIVATE_KEY`, `AWS_`, `GCP_`, or `AZURE_` are always redacted).

Dependency construction fires `on_dependencies_made`. The recorder maps task edges and recognizes `workflow://` references as producer/consumer relationships. For each task the recorder captures: type, original command and SHA-256 command hash, declared artifacts and annotations, predecessor/successor names, wait state, working directory, backend information, staging boundaries, timestamps, final status, and a `checkpoint_reused` flag for tasks skipped by checkpoint reuse. Existing declared local outputs receive size and SHA-256 fixity.

19.4 FAIR Principle Mapping

Principle	DAGOnStar behavior
Findable	Workflow, run, task, and output-artifact URNs generated; title, description, creators, keywords, and identifiers structured.
Accessible	Metadata as local UTF-8 JSON/JSON-LD, Markdown, and HTML. <code>AccessPolicy</code> states the assumption without copying data.
Interoperable	Emits RO-Crate JSON-LD, PROV-like JSON, DataCite JSON, CodeMeta JSON-LD. <code>workflow://</code> edges become provenance links.
Reusable	Licenses, task context, checkpoint reuse markers, checksums, file sizes, and runtime summaries retained.

19.5 Exports

The default output is `<scratch>/dagon/fair/<workflow>/<run-uuid>/`. The recorder writes: `run.json`, `ro-crate-metadata.json`, `prov.json`, `datacite.json`, `codemeta.json`, `checksums.sha256`, `fairness-report.json`, `report.md`, and `report.html`.

Caution

The recorder does **not** export raw `dagon.ini` content, credentials, keys, or tokens. FaaS activities add only sanitized provider and function identity, request identifiers, and materialized-output fixity. This is local export only: it does not validate a full Workflow Run RO-Crate profile, emit PROV-O Turtle, package or copy data, or publish to repositories.

19.6 Lesson 15 — Make a Workflow FAIR by Design

Lesson 15 — Make a Workflow FAIR by Design

Track: Advanced. **Runtime:** Local. **Colab:** Supported.

Objective. Record supported FAIR metadata and distinguish local export from publication.

Run:

```
python -m unittest tests.test_fair -v
```

Verify. Tests confirm profiles, records, sanitization, and local exports. They do not publish to any repository.

Controlled experiment. Remove a required profile field temporarily, predict strict-mode failure, then restore.

Scientific practice. FAIRness is a stewardship practice; local metadata alone is not global findability or accessibility.

Exercise 19.1. Add FAIR declarations to the environmental case study workflow from ???: assign `FairProfile` with a title, creator, and license; declare the WRF output as an `Artifact` with `media_type="application/x-netcdf"`. Run the workflow and inspect the generated `ro-crate-metadata.json`. Identify which `workflow://` edges appear as provenance relationships.

19.7 Chapter Summary

DAGonStar’s opt-in FAIR recorder assembles locally substantiated metadata and provenance without altering scheduling, staging, or execution for workflows that do not enable it. It exports RO-Crate, PROV, DataCite, and CodeMeta formats together with SHA-256 checksums and human-readable reports. The implementation is explicit about its boundary: local export, not repository publication. Enabling FAIR recording is the first step in a stewardship chain that ends at deposit; this chapter covers what DAGonStar contributes to that chain.

Chapter 20

CWL Export and Portable Emulation

A portable description of a workflow is not the same thing as portable execution. Stating the boundary explicitly is the first step toward useful interoperability.

*DAGonStar documentation,
docs/cwl_export.md*

The Common Workflow Language (CWL) v1.2 is a community standard for describing portable computational workflows. DAGonStar provides two complementary mechanisms: `Workflow.saveAsCWL()` exports the current DAG as a self-contained CWL v1.2 document, and the `portable_emulation` flag runs any workflow locally without contacting real infrastructure.

20.1 CWL Export: `Workflow.saveAsCWL()`

```
1 workflow = Workflow("analysis")
2 prepare = DagonTask(TaskType.BATCH, "prepare",
3                     "echo sample > sample.txt")
4 analyze = DagonTask(TaskType.BATCH, "analyze",
5                     "wc -c sample.txt > count.txt")
6 workflow.add_task(prepare)
7 workflow.add_task(analyze)
8 analyze.add_dependency_to(prepare)
9
10 workflow.saveAsCWL("analysis.cwl")
```

Listing 20.1: Exporting a workflow to CWL v1.2.

The exporter:

- runs `make_dependencies()` if not already called;
- converts task names to portable CWL identifiers, resolving collisions;
- maps each task to an embedded `CommandLineTool` executing `/bin/sh -c <command>`;
- adds a `DockerRequirement` hint for Docker tasks that carry an image tag;
- uses JSON syntax, which is also valid CWL, requiring no YAML library.

20.1.1 Validation and Execution

```
1 pip install cwltool
2 cwltool --validate analysis.cwl
3 cwltool analysis.cwl
```

Listing 20.2: Validating with cwltool.

The checked-in reference export in `examples/cwl/` is regenerated and validated by the automated test suite:

```
1 python -m unittest tests.test_cwl_example -v
```

20.1.2 The Portability Boundary

Caution

CWL export describes commands and graph ordering. It is **not** a lossless translation of every DAGOnStar executor. CWL runs exported commands in its own working directories; DAGOnStar scratch management, checkpointing, remote SSH, Slurm, cloud, Kubernetes, Nomad, service registration, and specialized staging are not reproduced by a generic CWL runner. A `workflow://` reference in a command is retained verbatim: it creates the correct step dependency in the exported graph, but a generic CWL runner does not implement DAGOnStar's URI staging.

Native, Web, and LLM tasks are exported only when their task object provides a non-empty shell command.

20.2 Portable Emulation Mode

The `portable_emulation=True` constructor parameter makes every task type run locally without contacting Docker, SSH, Slurm, cloud, Kubernetes, Apptainer, or Nomad:

```
1 workflow = Workflow(
2     "portable-demo",
3     config=local_config(workdir),
4     portable_emulation=True,
5 )
6 # LLM, Web, and FaaS use credential-free deterministic emulators.
7 # Docker, SLURM, Kubernetes use local scratch execution.
8 workflow.add_task(DagonTask(TaskType.BATCH, "prepare",
9     "echo prepared > result.txt"))
10 workflow.add_task(DagonTask(TaskType.NATIVE, "transform",
11     "mymodule:process",
12     inputs={"src": "workflow:///prepare/result.txt"},
13     outputs={"out": "out.txt"}))
14 workflow.make_dependencies()
15 workflow.run()
```

Listing 20.3: Portable emulation for CI, teaching, and Colab.

Original task types remain visible in FAIR metadata, JSON, checkpoints, and the DAG. This mode is specifically designed for:

- continuous integration that cannot access Docker or Slurm;
- teaching and workshop environments, including Google Colab;

- structural validation of a workflow’s graph before deploying on real infrastructure;
- dry runs that generate `launcher.sh` without executing.

20.3 The Interoperability Contract

The test in `tests/test_tasktype_interoperability.py` constructs one graph containing all registered task types and verifies that each:

1. supports factory construction via `DagonTask`;
2. discovers `workflow://` dependencies;
3. participates in FAIR recording (when enabled);
4. reuses checkpoints when the working directory is still accessible;
5. produces deterministic CWL v1.2 JSON;
6. coexists with every other type in the same workflow.

Adding a new task type requires passing all six verifications.

20.4 Lesson 17 — CWL Export

Lesson 17 — Export and Validate Common Workflow Language

Track: Advanced. **Runtime:** Local. **Colab:** Supported.

Objective. Export CWL v1.2 and explain the portability boundary.

Run:

```
python -m unittest tests.test_cwl_example -v
```

Controlled experiment. Change one command, predict the changed CWL field, regenerate, and compare with `diff`.

Scientific practice. Portable descriptions need explicit interfaces and documented semantic boundaries.

20.5 Chapter Summary

DAGonStar provides deterministic CWL v1.2 export validated by `cwltool`, and a portable emulation mode that runs any task type locally without real infrastructure. The portability boundary is explicitly documented: CWL captures the command graph; it does not reproduce staging, checkpointing, or backend-specific execution. Together, export and emulation enable interoperability with CWL-aware tools and provide a safe, credential-free path for CI, teaching, and Colab validation.

Chapter 21

Function-as-a-Service Tasks

Running this chapter in Google Colab

All lessons in this chapter run in Google Colab with Option 3 installation (`git clone` then `pip install -e .`, see Appendix E). Use `local_config(workdir)` instead of `dagon.ini`, and use `Path(task.get_scratch_dir())` to read task outputs instead of shell `find/cat` commands (see ?? and Appendix E for the complete guide).

`TaskType.FAAS` invokes an already-deployed function as a workflow node. It does not build, deploy, publish, or delete cloud infrastructure.

*DAGonStar documentation,
docs/faas_tasks.md*

Function-as-a-Service (FaaS) platforms allow scientific workflows to offload computation to event-driven, auto-scaling cloud functions without managing virtual machines. DAGonStar's `TaskType.FAAS` integrates with these platforms through a provider-neutral lifecycle model that records invocation identity, retry provenance, declared outputs, and provider-neutral execution metadata using the same `workflow://` dependency mechanism that governs every other task type.

21.1 When to Use FAAS

Need	FAAS	WEB	NATIVE
Already-deployed cloud function	✓		
Provider invocation provenance	✓		
Retry management + idempotency	✓		
General HTTP/HTTPS resource		✓	
Local Python function			✓
No external service (CI, Colab)	mock only	local mock	✓

Use **FAAS** when the workflow invokes an already-deployed function and needs provider

identity, retry, idempotency, and invocation provenance. Use `WEB` for a general HTTP resource; `NATIVE` for a local Python function.

21.2 Constructor and Parameters

```

1 from dagon.task import DagonTask, TaskType
2
3 classify = DagonTask(
4     TaskType.FAAS,
5     "classify",
6     provider = "mock",          # required
7     function  = "classifier",    # required
8     inputs   = {
9         "document": "workflow:///prepare/outputs/input.json",
10        "threshold": 0.8,
11    },
12    outputs   = {"label": "classification.json"},
13    invocation= "sync",          # "sync" (default) / "async"
14    timeout   = 120,
15    retry     = {"max_attempts": 3, "backoff": "exponential"},
16 )

```

Listing 21.1: FaaS task constructor.

`inputs` maps parameter names to JSON scalars, local file paths, or `workflow://` references. References infer dependencies, copy the producer file into `inputs/`, and become artifact descriptors containing task-relative path, size, media type, and SHA-256 at execution time. `outputs` maps names to paths relative to `outputs/`; absolute paths and paths containing `.` are rejected.

Retryable categories default to throttling, provider unavailability, and transport failures. Retries retain one deterministic idempotency key and record distinct attempt numbers. Runtime records are written to `.dagon/faas_spec.json`, `faas_request.json`, `faas_response.json`, and `faas_provider_metadata.json`, with all credential values excluded.

21.3 Providers

Provider	Description
<code>mock</code>	Deterministic, in-process, credential-free. The required starting point for CI, examples, and local development.
<code>http</code>	Posts the DAGOnStar JSON envelope. Headers from <code>header_env</code> ; literal secrets rejected.
<code>knative</code>	HTTP adapter with optional CloudEvents binary headers.
<code>aws</code>	AWS Lambda via boto3 default credential chain. Install <code>dagonstar[faas-aws]</code> .
<code>azure</code>	Azure Functions via DefaultAzureCredential or <code>token_env</code> . Install <code>dagonstar[faas-azure]</code> .
<code>gcp</code>	GCP Cloud Run via Application Default Credentials or <code>token_env</code> . Install <code>dagonstar[faas-gcp]</code> .

Named profiles in `dagon.ini`:

```

1 [faas.aws.prod]
2 region = eu-south-1
3 ; Never place literal credentials here --- use IAM profiles.

```

Explicit `provider_options` override profile values. Profiles contain endpoints, regions, and environment-variable *names*, never credential values. The full configuration reference is in Appendix G.

21.4 Security Constraints

Caution

Provider options may reference secrets **only through names ending in `_env`**, which cause the adapter to read the value from the named environment variable at runtime. Literal secrets in options are rejected. Output traversal (`../`) is rejected. TLS remains enabled. Authorization headers, signed query strings, and raw SDK objects are excluded from all provenance records.

21.5 Mixed Workflow Example

```
1 workflow.add_task(DagonTask(TaskType.BATCH, "prepare",
2     "mkdir -p output; echo sample > output/value.txt"))
3 workflow.add_task(DagonTask(TaskType.NATIVE, "transform",
4     "myproject.tasks:transform",
5     inputs={"source": "workflow:///prepare/output/value.txt"},
6     outputs={"result": "result.txt"}))
7 workflow.add_task(DagonTask(TaskType.FAAS, "publish",
8     provider="mock", function="echo",
9     inputs={"document": "workflow:///transform/outputs/result.txt"}))
10
11 workflow.make_dependencies()
12 workflow.run()
```

Listing 21.2: BATCH -> NATIVE -> FAAS three-task chain.

The `workflow://` reference in the FaaS input creates the `transform->publish` dependency automatically, stages the file, and converts it to an artifact descriptor in the request envelope.

21.6 Chapter Summary

`TaskType.FAAS` provides a provider-neutral, lifecycle-aware model for invoking already-deployed cloud functions. It differs from `WEB` in semantic specificity (function identity, retry, provenance) and from `NATIVE` in execution location (the function runs on the cloud platform). Six provider adapters cover mock to AWS, Azure, and GCP. Credentials are always referenced through environment variable names, never embedded in source. The complete provider configuration is in Appendix G.

Chapter 22

IoT and Compute-Continuum Tasks

`TaskType.IOT` models bounded observation, desired-state configuration, physical actuation, and portable edge computation. It is not an alias for HTTP, SSH, or shell execution.

*DAGonStar documentation,
docs/iot_tasks.md*

The *compute continuum* spans embedded IoT sensors through edge processors, gateways, and cloud or HPC clusters. Environmental science increasingly depends on this full spectrum: monitoring stations, smart buoys, and edge processors produce observations that feed directly into numerical models. DAGonStar’s `TaskType.IOT` models this boundary explicitly through four semantically distinct operations while maintaining the same dependency, FAIR, CWL, and checkpoint model as every other task type.

22.1 Four Operations

observe

A sensor reading that produces a finite, bounded sequence of events. Unbounded streams cannot be represented as workflow tasks; completion *must* be bounded.

configure

A desired-state configuration task. The provider adapter resolves how to achieve the target state.

actuate

Physical actuation. Non-idempotent by default: one attempt. An unknown acknowledgement raises `IoTUnknownOutcomeError` and is not replayed.

compute

Portable edge/gateway/cloud computation with declared artifacts and placement preferences.

Caution

DAGonStar is **not** a physical safety interlock. Device safety interlocks remain the responsibility of firmware and hardware design. DAGonStar models the logical workflow; it does not guarantee physical delivery, timing, or safety of actuation commands.

22.2 Constructor

```

1 from dagon.task import DagonTask, TaskType
2
3 observe = DagonTask(
4     TaskType.IOT,
5     "temperature_station_01",
6     operation = "observe",          # observe / configure / actuate /
        compute
7     provider   = "mock",           # always start with mock
8     completion = {"events": 3},    # bounded completion is mandatory
9 )

```

Listing 22.1: IoT task construction: bounded observation.

Key parameters:

operation

Required. One of the four operations above.

provider

Required. `mock` is deterministic, credential-free, and the required starting point.

completion

Required for `observe`. Must be bounded.

credential_ref

Environment-variable name holding the device credential. Values must never appear in specifications.

22.3 Workflow:// in IoT Specifications

Nested `workflow://` references are fully supported in structured IoT specification fields:

```

1 calibrate = DagonTask(
2     TaskType.IOT,
3     "apply_calibration",
4     operation      = "configure",
5     provider       = "mock",
6     completion     = {"acknowledge": True},
7     specification = {
8         "calibration_file":
9         "workflow:///download_cal/outputs/cal.json",
10    },
11 )

```

Listing 22.2: IoT configure task consuming an upstream calibration file.

`make_dependencies()` discovers the reference, stages the file beneath `.dagon/inputs`, and resolves a deep copy at execution time that preserves the logical strings in JSON. Outputs are scratch-relative and traversal-safe.

22.4 Compute-Continuum Operations

An IoT `compute` task declares an artifact and placement preferences:

```

1 edge_qc = DagonTask(
2     TaskType.IOT,
3     "edge_quality_control",
4     operation = "compute",
5     provider  = "mock",
6     completion = {"artifact": True},
7     placement  = ["edge", "gateway", "cloud"], # preference order
8     artifact   = "workflow:///observe/outputs/temperature.json",
9     runtime    = "python3 qc_model.py",
10 )

```

Listing 22.3: Compute-continuum task with tiered placement.

Results record requested/selected targets, tier, placement, migrations, and protocol meta-data. Mock placement is deterministic and does not claim real deployment. DAGonStar owns workflow dependencies and evidence; provider adapters or external orchestrators own discovery, deployment, and migration.

22.5 Lesson 18 — Compute-Continuum Operations

Lesson 18 — Operate Across the Compute Continuum

Track: Advanced. **Runtime:** Local deterministic mock. **Colab:** Supported.

Objective. Create bounded IoT tasks, connect structured data, inspect placement evidence, and explain safe recovery.

Run:

```
python3 examples/iot/observe_mock.py
```

Expected result. The output JSON contains event sequences 1, 2, and 3 exactly once.

Controlled experiment. Change the completion bound from {"events": 3} to {"events": 1} and verify the assertion failure.

Scientific practice. IoT tasks model intent, not physical delivery. Device safety interlocks remain authoritative.

Exercise 22.1. Extend the environmental case study (??) with an upstream IoT observation stage: construct a `TaskType.IOT` task with `operation="observe"`, `completion={"events": 6}`, and `provider="mock"`. Connect its output to the `prepare_forcing BATCH` task via a `workflow://` reference. Verify the dependency edge appears in the exported CWL.

22.6 Chapter Summary

`TaskType.IOT` models the compute continuum through four semantically specific operations: bounded observation, desired-state configuration, physical actuation, and portable edge computation. It participates in the same `workflow://` dependency, FAIR recording, CWL export, and checkpoint model as every other task type while adding IoT-specific concepts: bounded completion, placement preferences, and an explicit not-a-safety-interlock boundary. A credential-free deterministic mock enables full-path testing without physical devices. Provider configuration is in Appendix G.

Chapter 23

Extended Infrastructure Backends

A producer’s execution backend does not constrain the consumer’s backend: the edge is expressed through `workflow://`, and the configured stager moves or exposes the artifact between their working directories.

*DAGonStar documentation,
docs/tasktype_interoperability.md*

Three additional infrastructure backends extend DAGonStar’s execution surface beyond the eight discussed in ??: Kubernetes container orchestration, Apptainer (the HPC-native container runtime that supersedes Singularity), and HashiCorp Nomad job scheduling. All three participate in the same dependency model, FAIR recording, CWL export, checkpoint semantics, and portable emulation as the original task types.

23.1 The Interoperability Contract

The automated test `tests/test_tasktype_interoperability.py` constructs one graph containing all registered task types and verifies that each supports:

1. factory construction through `DagonTask(TaskType.X, ...)`;
2. explicit and `workflow://` dependency discovery;
3. opt-in FAIR declarations and lifecycle provenance;
4. checkpoint reuse when the working directory is accessible;
5. deterministic CWL v1.2 JSON export;
6. coexistence with every other type in the same workflow.

Adding a new task type to DAGonStar requires passing all six verifications. This prevents heterogeneous workflows from breaking silently at the dependency, FAIR, or export layer.

23.2 Kubernetes (`TaskType.KUBERNETES`)

KUBERNETES submits the task as a Kubernetes Job on a reachable cluster:

```

1 k8s_regrid = DagonTask(
2     TaskType.KUBERNETES,
3     "regrid",
4     "python regrid.py "
5     "--input workflow:///model/wrfout_d01",
6     image = "lab/gdal-regrid:3.7@sha256:<digest>",
7     namespace = "research",
8     resources = {"cpu": "4", "memory": "8Gi"},
9 )

```

Listing 23.1: Kubernetes task for geospatial regridding.

`image=` is required; pin to a digest for reproducibility. `kubeconfig=` defaults to `~/.kube/config`.

When to use. Kubernetes is appropriate when the analysis step requires a specific container image, the facility provides a Kubernetes cluster alongside Slurm, or the workflow targets a managed cloud Kubernetes service.

```

1 pip install "dagonstar[kubernetes]"

```

23.3 Apptainer (TaskType.APPTAINER)

Apptainer (formerly Singularity) runs without a privileged daemon and is the container runtime of choice at most HPC facilities. APPTAINER wraps the command in Apptainer execution:

```

1 apptainer_post = DagonTask(
2     TaskType.APPTAINER,
3     "postprocess",
4     "python render.py "
5     "--wrfout workflow:///wrf/output/wrfout_d01",
6     image = "oras://registry.example.org/lab/gdal:3.7",
7     bind = ["/data:/data"],      # host:container bind mounts
8 )

```

Listing 23.2: Apptainer task for HPC post-processing.

When to use. Use when the HPC facility provides Apptainer or Singularity but not Docker. Bind the task's scratch directory inside the container via `bind=` to make staged files accessible.

```

1 pip install "dagonstar[apptainer]"

```

23.4 Nomad (TaskType.NOMAD)

HashiCorp Nomad schedules workloads across heterogeneous clusters, supporting Docker, exec, Java, and custom task drivers.

```

1 nomad_qc = DagonTask(
2     TaskType.NOMAD,
3     "batch_qc",
4     "python qc.py "
5     "--input workflow:///ingest/raw_data.nc",
6     driver = "exec",
7     nomad_address= "http://nomad.cluster.example.org:4646",
8 )

```

Listing 23.3: Nomad task for batch quality control.

```
pip install "dagonstar[nomad]"
```

23.5 The Complete Backend Matrix

Table 23.1: All thirteen DAGonStar execution backends.

TaskType	Execution requirement	Notebook / Colab
BATCH	POSIX shell	Fully supported.
CHECKPOINT	POSIX shell	Supported; use durable storage.
SLURM	Reachable Slurm client	Construct anywhere; run needs cluster.
CLOUD	Provider extra + identity	Live run needs provider.
DOCKER	Docker extra + daemon	Local Jupyter may; hosted Colab normally cannot.
KUBERNETES	Extra + kubeconfig	Live run needs reachable cluster.
APPTAINER	Apptainer + image	Live run needs Apptainer-capable host.
NOMAD	Nomad API + client	Live run needs Nomad deployment.
LLM	Endpoint or local mock	Mock runs in notebooks and Colab.
NATIVE	Importable Python fn	Fully supported.
WEB	Endpoint or local mock	Mock supported.
FAAS	Provider or mock	Mock runs locally.
IOT	Provider or mock	Deterministic mock runs locally.

Use `portable_emulation=True` (??) to run all thirteen types locally for CI, teaching, and Colab validation.

23.6 Extended Backend Selection Guide

Building on ?? from ??, the full table for thirteen backends:

Scientific need	TaskType	Key parameters
Small local utility	BATCH	command string
Remote SSH host	BATCH+ip=	ip=, ssh_username=
HPC Slurm scheduler	SLURM	partition=, ntasks=
Local Docker image	DOCKER	image=
Kubernetes cluster	KUBERNETES	image=, namespace=
HPC container (no daemon)	APPTAINER	image=, bind=
Nomad cluster	NOMAD	driver=, address
Cloud VM	CLOUD	provider, flavor
Cloud function	FAAS	provider=, function=
AI interpretation	LLM	provider=, input_files=
REST publication	WEB	specification dict
Python post-processing	NATIVE	module:function
Edge IoT observation	IOT	operation=, provider=

23.7 Chapter Summary

Kubernetes, Apptainer, and Nomad extend DAGonStar's execution surface to cover container orchestration, HPC-native containers, and heterogeneous workload schedulers. All three satisfy the interoperability contract that governs every backend: `factory construction`, `workflow://`

dependency discovery, FAIR recording, checkpoint reuse, CWL export, and coexistence. The configuration of external service providers for all backends is in Appendix G.

Part V

Appendices

Appendix A

Reference Guide: Public API

This appendix consolidates the engine’s public API for use as a standing reference once the narrative chapters of Parts I–III have been read. It mirrors the project’s own `reference_guide.md` in full, and additionally interleaves short discussion paragraphs connecting each API surface back to the chapter in which its design rationale was developed, so that this appendix can serve a reader either as a pure lookup table or as a condensed second pass through the book’s argument.

A.1 Import Conventions

```
1 from dagon import Workflow, DataMover, StagerMover
2 from dagon.task import DagonTask, TaskType
```

These two import lines are, in a precise sense, the entire public surface a workflow author needs for the great majority of workflows discussed in this book: every example from ?? through ?? is expressible using only the names imported here, plus the task-type-specific keyword arguments documented later in this appendix. The optional `dagon.api`, `dagon.dag_tps`, and `dagon.config` modules, discussed in their respective sections below, are imported separately precisely because they are optional in the sense already developed in ??: a workflow that never uses the DAGon service, meta-workflows, or programmatic configuration access has no need to import them at all.

A.2 `dagon.Workflow`

```
1 Workflow(
2     name,
3     config=None,
4     config_file="dagon.ini",
5     max_threads=10,
6     jsonload=None,
7     checkpoint_file=None,
8 )
```

Listing A.1: Constructor signature.

name

workflow name.

config

optional in-memory configuration dictionary.

config_file

INI file path used when `config` is not provided.

max_threads

semaphore size currently stored for task use.

jsonload

JSON-like workflow object to load.

checkpoint_file

file path for writing checkpoint metadata.

Recall from ??’s discussion of the layered override mechanism that `config` and `config_file` are mutually exclusive in effect, not additive: supplying `config` bypasses `config_file` entirely, rather than overlaying it. The `checkpoint_file` parameter, examined in full in ?? and Appendix D, is what activates the `<workflow-name>.<task-name>` checkpoint-key scheme; omitting it entirely disables checkpoint recording, which is the correct choice for the many short, cheap-to-rerun tutorial-scale workflows used throughout Part I, and the incorrect choice for any workflow expensive enough to warrant resumability.

Selected methods.

- `add_task(task)`: add a task and attach it to this workflow.
- `make_dependencies()`: parse dataflow references and validate the graph.
- `run(resume_checkpoint_file=None)`: run tasks in the current thread.
- `launch(resume_checkpoint_file=None)`: run tasks in a background thread.
- `wait(timeout=None)`: wait for a launched workflow; returns whether it ended.
- `add_listener(event_name, callback)`: register a workflow or task lifecycle callback.
- `set_dry(dry) / get_dry()`: set/return the dry-run flag.
- `set_data_mover(data_mover) / get_data_mover()`: set/return the workflow default data mover.
- `set_stager_mover(stager_mover)`: set the workflow default staging mode.
- `get_scratch_dir_base()`: compute or return the workflow scratch base.
- `find_task_by_name(workflow_name, task_name)`: locate a task.
- `as_json()`: serialize workflow metadata.
- `load_json(Json_data)`: load tasks from JSON-like data.
- `enable_fair(profile)`: attach FAIR recorder (??).
- `saveAsCWL(filename)`: write CWL v1.2 export (??).
- `set_dry(dry) / get_dry()`: dry-run mode.
- `Validate_WF()`: reject cycles.

Two of these methods deserve a cross-reference a pure API listing would otherwise omit. `as_json()/load_json()` are the mechanism underlying the cross-process workflow transport implied by `??`'s optional DAGon service, and are also what makes a serialized workflow — including its LLM task provider references, per `??` — portable across machines, provided the receiving machine has the same named provider configured locally. `get_scratch_dir_base()` is the method whose return value ultimately determines where every artifact discussed in `??`'s scratch-directory anatomy (`??`) physically lives.

A.3 `dragon.Status`

Value
<code>Status.READY</code>
<code>Status.WAITING</code>
<code>Status.RUNNING</code>
<code>Status.FINISHED</code>
<code>Status.FAILED</code>

This five-state model is the concrete realization of the abstract, topological-order-driven scheduling argument developed in `??`: `READY` corresponds exactly to a node with no remaining incoming edges in the source-removal construction given there, and the transition from `WAITING` to `READY`, triggered when a task's last predecessor reaches `FINISHED`, is the runtime enactment of that construction's repeated source-removal step.

A.4 `dragon.task.TaskType`

Value
<code>TaskType.CHECKPOINT</code>
<code>TaskType.BATCH</code>
<code>TaskType.SLURM</code>
<code>TaskType.CLOUD</code>
<code>TaskType.DOCKER</code>
<code>TaskType.LLM</code>
<code>TaskType.KUBERNETES</code>
<code>TaskType.APPTAINER</code>
<code>TaskType.NOMAD</code>
<code>TaskType.FAAS</code>
<code>TaskType.IOT</code>
<code>TaskType.NATIVE</code>
<code>TaskType.WEB</code>

A.5 `dragon.task.DragonTask`

```
task = DragonTask(TaskType.BATCH, "A", "echo A")
```

The concrete class is selected from the `tasks_types` mapping:

Task type	Module	Class
CHECKPOINT	<code>tagon.checkpoint</code>	<code>Checkpoint</code>
BATCH	<code>tagon.batch</code>	<code>Batch</code>
CLOUD	<code>tagon.remote</code>	<code>CloudTask</code>
DOCKER	<code>tagon.docker_task</code>	<code>DockerTask</code>
LLM	<code>tagon.llm</code>	<code>LLMTask</code>
NATIVE	<code>tagon.native</code>	<code>NativeTask</code>
WEB	<code>tagon.web</code>	<code>WebTask</code>
SLURM	<code>tagon.batch</code>	<code>Slurm</code>

This table is, in effect, the implementation-level evidence for the factory- pattern discussion of ??: every row maps an identical calling convention — `DagonTask(TaskType.X, name, payload, ...)` — onto a distinct concrete class living in its own module, several of which (`tagon.docker_task`, `tagon.llm`, `tagon.native`, `tagon.web`) are, per ??, lazily imported, so that constructing a BATCH task never requires the Docker SDK or any web-request library to even be installed.

A.6 `tagon.task.Task`

```
1 Task(name, command, working_dir=None, transversal_workflow=None,
    globusendpoint=None)
```

Listing A.2: Base constructor.

Important attributes. `name`, `command`, `working_dir`, `nexts`, `prevs`, `reference_count`, `workflow`, `status`, `data_mover`, `stager_mover`, `globusendpoint`.

Important methods. `add_dependency_to(task)`, `add_transversal_point(task)`, `pre_run()`, `pre_process_command(command)`, `post_process_command(command)`, `include_command(body)`, `create_working_dir()`, `execute()`, `run()`, `as_json()`, `get_scratch_dir()`, `get_endpoint()`, `set_endpoint(globusendpoint)`, `set_data_mover(data_mover)`, `set_stager_mover(stager_mover)`.

Subclasses are expected to implement or specialize command execution (??). Of the attributes listed, `prevs` and `nexts` are the two a workflow author will inspect most often during the local-structural-verification pattern recommended throughout ????. `pre_run()` and `pre_process_command()` are the two methods that jointly implement the dependency-inference algorithm formalized in ??.

A.7 Batch and Slurm Tasks

```
1 DagonTask(TaskType.BATCH, "local", "hostname")
```

Remote batch is selected by passing SSH parameters:

```
1 DagonTask(
2     TaskType.BATCH, "remote", "hostname",
3     ip="example.org", ssh_username="user", keypath="/path/to/key",
4 )
```

```
1 DagonTask(
2     TaskType.SLURM, "simulation", "./run.sh",
3     partition="compute", ntasks=4, memory=4096, time="01:00:00",
4 )
```

Supported Slurm constructor options include: `comment`, `partition`, `ntasks`, `memory`, `time`, `nodes`, `ntasks_per_node`, `working_dir`, `globusendpoint`. Remote Slurm is selected by passing `ip`. See ?? for the queueing-cost and structural-verification discussion that should accompany any first use of these parameters against a real cluster.

A.8 Docker Tasks

```
1 DagonTask(
2     TaskType.DOCKER, "container-task", "python --version",
3     image="python:3.12-slim",
4 )
```

Options include: `image`, `container_id`, `working_dir`, `globusendpoint`, `remove`, `volume`. Remote Docker is selected by passing SSH information including `ip`. The `image` value, as discussed in ??’s tag-versus-digest reproducibility comparison, should be considered for digest-pinning whenever the workflow’s results are intended for archival or publication.

A.9 Cloud Tasks

Cloud tasks are backed by `dagon.remote.CloudTask` and Apache Libcloud. They require provider information, an SSH user, key options, and either an existing instance or instance flavor data. Cloud behavior is provider-specific and should be tested in the target environment. As noted in ??’s discussion of configuration sections not exercised in this book’s worked examples, no runnable cloud example is given here; the provider-specific authentication and provisioning model must be verified against the target provider’s own current documentation.

A.10 LLM Tasks

```
1 DagonTask(TaskType.LLM, name, prompt, provider, params=None, input_files=
2     None,
3     working_dir=None, output_file="response.json", timeout=120)
```

Listing A.3: LLMTask constructor.

`prompt` is a JSON object (or a JSON string) containing `messages`; string values may use `{parameter}` placeholders from `params` or `input_files`. Provider configuration is read from `[llm.<provider>]`. `input_files` maps parameter names to `workflow://` references, which infer dependencies and stage local UTF-8 text before the request. The default `timeout=120` should be reconsidered, per ??’s discussion of operational constraints, against the documented typical latency of whichever real provider a workflow ultimately targets. See ??.

A.11 Native Tasks

```
1 DagonTask(TaskType.NATIVE, name, function, inputs=None, outputs=None,
2     executor="local", resources=None, python=sys.executable,
3     working_dir=None, environment=None)
```

Listing A.4: NativeTask constructor.

Inputs may be JSON scalars, existing local files, or `workflow://` file references. File values are staged below `inputs/`; `outputs` maps parameter names to safe relative paths below `outputs/`. JSON return metadata is written to `.dagon/native_result.json`. `executor`

accepts `local` and `slurm`; functions must be importable module-level callables, for the serialization-related reasons developed in full in ???. See ??.

A.12 Web Tasks

```

1 DagonTask(TaskType.WEB, name,
2     {"method": "GET", "url": "https://example.org",
3      "outputs": {"body": "response.json"}},
4     executor="local")

```

Listing A.5: WebTask specification.

Specifications support `query`, `headers`, `json`, `data`, `multipart`, `auth`, `timeout`, `retry` settings, expected status codes, and `outputs`. Recursive `workflow://` values infer dependencies; referenced files are copied below `inputs/`. Body, headers, and metadata outputs are safe relative paths below `outputs/`; `.dagon/web_result.json` stores non-secret status metadata. Supported authentication is bearer, basic, API-key header, or API-key query, always through environment-variable *names* rather than literal secrets. `executor="slurm"` accepts existing scheduler options in `resources`. The retry settings mentioned here interact directly with the idempotency considerations developed in ???; review that chapter's discussion before enabling automatic retries against a non-idempotent endpoint. See ??.

A.13 Data Movement Enumerations

DataMover. DONTMOVE, LINK, COPY, SCP, HTTP, HTTPS, FTP, SFTP, GRIDFTP, SKYCDS. The cost/self-containment trade-off between COPY and LINK specifically is examined quantitatively in ??.

StagerMover. NORMAL, PARALLEL, SLURM.

A.14 Configuration API

```

1 from dagon.config import read_config
2
3 all_sections = read_config("dagon.ini")
4 batch = read_config("dagon.ini", "batch")

```

If a requested section does not exist, `read_config(..., section)` returns `None`. This function underlies the file-based configuration path discussed in ???; note that it is bypassed entirely when a `Workflow` is constructed with an in-memory `config` dictionary instead.

A.15 Workflow Schema

The `workflow://` schema is documented in full in ???. In brief:

```

1 workflow://<workflow-name>/<task-name>/<path-inside-task-working-directory>
2 workflow:///<task-name>/<path-inside-task-working-directory>

```

A.16 Meta-Workflows

```
1 from dagon.dag_tps import DAG_TPS
2
3 meta = DAG_TPS("ExperimentSet")
4 meta.add_workflow(workflow_a)
5 meta.add_workflow(workflow_b)
6 meta.make_dependencies()
7 meta.run()
```

Useful for transversal workflows that reference data across workflow boundaries (??). The meta-workflow code is legacy-sensitive and less covered by automated tests than local `Workflow` behavior; production use should be verified with representative workflows, preserving the verification command with the experiment record. See also this appendix’s framing of this caveat within ??’s closing, honesty-focused revisiting of the book’s central claims.

A.17 Service API

`dagon.api.API` communicates with a DAGon service endpoint. It supports: `create_workflow(workflow)`, `add_task(workflow_id, task)`, `update_task_status(workflow_id, task, status)`, `get_task(workflow_id, task)`, `get_workflow_by_name(workflow_name)`, `update_task(workflow_id, task, attribute, value)`, `add_dependency(workflow_id, task, dependency)`.

The service is optional and disabled in the default sample configuration (??). A reader who has worked through this book’s local-first examples exclusively, as the examples-catalog discussion of ?? recommends, will not have exercised this API directly; it becomes relevant only when a deployment genuinely requires the multi-process or multi-host coordination discussed in ??’s comparison of layered-in-process and microservice architectural alternatives.

Appendix B

Developer Guide: Contributing to DAGonStar

This appendix is addressed to contributors and agents modifying DAGonStar itself, rather than to workflow authors; it is reproduced here so that readers of this textbook who wish to extend the engine — for instance, by adding a new task type alongside LLM, WEB, and NATIVE — have the project’s own development discipline available as a reference.

B.1 Why a Textbook Includes a Developer Guide

It is worth pausing to justify, briefly, why this appendix exists at all in a book primarily addressed to workflow *authors* rather than engine *contributors*. Two reasons motivate its inclusion. First, a reader who encounters one of this book’s many documented limitations (?? being the most concentrated instance) and wishes to address it directly — rather than merely working around it — needs exactly the procedural knowledge this appendix provides. Second, and more broadly, reading how a project’s own contributors are asked to extend it is one of the most reliable ways to understand a system’s actual design boundaries, often more reliably than reading the system’s own marketing or high-level architecture description: a project’s testing and extension discipline reveals, concretely, which parts of the system its own maintainers consider stable and trustworthy, and which parts they explicitly flag as requiring caution. The seven-step new-task-type recipe below, for instance, is the same recipe that produced every task type this book treats as a stable, well-documented mechanism in Part II.

B.2 Ground Rule: Implementation First

Documentation, examples, and tests must describe the *current* implementation in the repository. If intended behavior differs from current behavior, the project’s policy is to either implement and test it, or label it explicitly as future work — the same fidelity principle this textbook has followed throughout.

This ground rule is worth connecting explicitly to the architectural-honesty discussion of ?? (??): the documented constraints reproduced there exist, precisely, *because* this ground rule was followed when that section of the documentation was written — a project that instead documented intended, aspirational behavior would have no structural reason to mention incomplete shell quoting or partial test coverage at all. The rule is therefore not merely a contributor instruction; it is the generative principle behind much of what makes the documentation this book is built from trustworthy as source material in the first place.

B.3 Development Setup

```

1 git clone https://github.com/DagOnStar/dagonstar.git
2 cd dagonstar
3 python3 -m venv .venv
4 . .venv/bin/activate
5 python -m pip install --upgrade pip
6 python -m pip install -e .

```

Install optional integration dependencies only when needed:

```

1 python -m pip install -e ".[docker]"
2 python -m pip install -e ".[cloud]"
3 python -m pip install -e ".[globus]"
4 python -m pip install -e ".[api]"
5 python -m pip install -e ".[all]"

```

Note the use of `-e` (editable install) here, in contrast to the plain `pip install -r requirements.txt` path given for workflow *authors* in `??`: an editable install links the installed package directly to the working source tree, so that changes made to `dagon/` source files take effect immediately on the next interpreter invocation, without requiring a reinstall — the correct mode for someone actively modifying the engine, as opposed to someone merely authoring workflows against an unmodified installation.

B.4 Test Commands

```

1 python -m unittest discover -s tests -v

```

Listing B.1: Run unit tests.

```

1 python -m py_compile dagon/*.py dagon/api/*.py dagon/communication/*.py
   dagon/ftp_publisher/*.py dagon/peer2peer/*.py tests/*.py

```

Listing B.2: Compile source and tests.

```

1 python -m ruff check dagon/shell.py tests
2 python -m mypy dagon/shell.py

```

Listing B.3: Lightweight quality checks.

(Install `mypy==1.13.0` and `ruff==0.6.9` first if not already available.) When packaging metadata changes:

```

1 python -m pip install -e .

```

B.5 Test Strategy

The current unit tests deliberately avoid external services. They verify stable core behavior: configuration parsing; workflow defaults; dependency discovery; cycle detection; serialization; runtime-secret validation.

When adding features:

1. add a unit test for local deterministic behavior;
2. mock external services where useful;

3. keep Docker/Slurm/SSH/Globus/cloud tests separate from generic CI unless the environment is explicitly provisioned;
4. document manual integration verification commands in the change notes.

This four-point strategy is, point for point, the same mock-service discipline already generalized in ??’s discussion of mock services as a verification strategy: just as a workflow *author* verifies an LLM- or WEB-task mechanism locally against a mock before relying on a real provider, an engine *contributor* verifies new engine behavior locally, against mocked external services, before that behavior is exercised against a real Docker daemon, SSH host, or Slurm scheduler. The same underlying principle — separate verifying the mechanism from verifying the integration — operates at both the workflow- authoring layer and the engine-development layer.

Scientific Practice Note

The test suite is intentionally a starting point, not a complete quality gate. It includes checkpoint reuse, staging command generation, command failure propagation, packaging extras, and mocked Docker, cloud, Globus, and API-extra boundaries. CI applies Ruff to the tests and shell helper, and mypy to the shell helper; both scopes are expanded gradually as modules gain annotations. Priority areas for further expansion are command rewriting for `workflow://` references, local batch integration workflows, and mocked SSH and Slurm boundaries.

B.6 Adding a New Task Type

1. Add a value to `TaskType`.
2. Add a mapping in `tasks_types`.
3. Implement a concrete task class.
4. Ensure the class initializes the base `Task` correctly.
5. Override `on_execute()` or other environment-specific hooks.
6. Add unit tests for construction and dependency behavior.
7. Update `reference_guide.md`, `architecture.md`, and relevant tutorials (in this book’s terms: Appendix A, ??, and the corresponding lesson chapter).

This is precisely the process by which LLM, WEB, and NATIVE were added to the original BATCH/SLURM/ DOCKER/CLOUD backend family — the resulting uniformity across all seven concrete task types, exploited throughout Part II of this book, is a direct consequence of every task type following the same seven-step recipe.

Example B.1. Consider, as a thought experiment grounded entirely in this recipe, what adding a hypothetical `TaskType.QUEUE` (a task that publishes a message to a message-queue system rather than waiting for a synchronous HTTP response, in contrast to `TaskType.WEB`’s synchronous request model) would require: a new `TaskType.QUEUE` enum value (step 1); a new mapping to, say, a `dagon.queue.QueueTask` class (step 2); the class itself, implementing message construction and publication (step 3); correct base-class initialization preserving `prevs/nexts` dependency bookkeeping (step 4); an `on_execute()` override implementing the actual publish call (step 5); unit tests verifying that a `workflow://` reference inside the message

payload is correctly parsed and staged, mirroring the existing `WebTask` tests (step 6); and updates to Appendix A, ??’s task-type table, and a new lesson chapter analogous to ?? (step 7). No part of this hypothetical extension would require modifying `Workflow`, `Task.pre_run()`, or the `workflow://` parser itself — exactly the architectural payoff the factory-pattern discussion of ?? predicted.

B.7 Modifying Dependency Resolution

Dependency resolution is implemented primarily in `Task.pre_run()` and `Task.pre_process_command()` (??). Be careful with: parsing `workflow://` references; same-workflow versus transversal references; reference counts used by garbage collection; recursive calls to `workflow.make_dependencies()` for parallel modes; service-backed task lookup when the DAGon service is enabled.

Any change here should include tests for: same-workflow references; missing references; cycles; command rewriting; reference counts.

B.8 Modifying Staging

Staging logic lives in `Stager.stage_in()` and `Stager.generate_command()` (??). Risks: shell quoting; remote/local path distinction; external command availability; differences between files, directories, and glob patterns; service-specific credentials.

Scientific Practice Note

Prefer adding helpers and unit tests before broad rewrites. New staging or command-generation code should avoid raw string concatenation for paths or user-controlled values; prefer explicit quoting helpers and tests covering spaces, special characters, missing files, directories, and glob patterns.

This guidance connects directly to the whitespace boundary rule already encountered as a workflow-author-facing constraint in ??: the parser limitation that forces a workflow author to avoid spaces in referenced paths is, from the contributor’s side of the same boundary, exactly the kind of shell-quoting risk this section warns against introducing further instances of. A contributor strengthening staging-path quoting is, in effect, working to relax the constraint that ?? currently asks workflow authors to observe manually.

B.9 Security Expectations

- Never add real credentials to source files, docs, sample config, tests, or CI.
- Use placeholders in examples.
- Read runtime secrets from environment variables or local ignored files.
- Avoid logging secrets.
- Quote shell arguments defensively in new code.

These expectations are the implementation-level counterpart of the configuration-level rules given in ?? and exercised throughout the LLM- and Web-task chapters (????). Note the symmetry: `api_key_env`, `token_env`, and the other environment-variable-indirection mechanisms documented from a workflow author’s perspective in ?? exist specifically *because* contributors are held, by this section’s expectations, to never hard-code credential-handling logic that would make such indirection unnecessary or bypassable.

B.10 Documentation Expectations

When changing implementation: update the relevant documentation file in `docs/`; update `README.md` if the change affects onboarding; update `AGENTS.md` if contributor policy changes; add or update tests.

Scientific Practice Note

Scientific documentation should be precise enough for a researcher to reproduce the computational method, and honest about operational assumptions — the same standard this textbook has attempted to meet throughout, including its explicit reproduction of every documented limitation and open technical-debt item.

B.11 Source-Level Changes in the Master Branch

Two targeted source improvements were introduced between the `development-native-web` branch and the master branch:

- `tagon/communication/ssh.py`: added `import logging` and initialized `self.logger = logging.getLogger()` in the `SSHManager.__init__` constructor. Prior to this change, `SSHManager` used the root logger directly without obtaining a named logger object, which made log filtering by module name unreliable.
- `tagon/task.py`: removed two `self.workflow.logger.debug()` calls from the garbage-collection path that printed the internal `remove_scratch_dir` flag and `nexts` list. These calls were diagnostic artifacts from development and produced misleading output during normal execution of any workflow that reached the cleanup phase.

Neither change affects the public API documented in Appendix A or the behavior of any workflow described in this textbook.

B.12 New Mandatory Requirements (v2.2)

The `AGENTS.md` file in the repository root governs all contributions and adds two mandatory requirements that apply to every code change as of the `main` branch:

FAIR-by-design. Treat FAIR metadata as a first-class, opt-in concern when adding or changing workflow, task, staging, checkpoint, or provenance behavior:

- Preserve the behavior of workflows that do not enable FAIR recording.
- Record only information the implementation can substantiate locally. Do not present local exports as repository publication, remote validation, or copied data.
- Keep `tagon/fair/` optional and dependency-free; FAIR metadata must never include credentials.
- Maintain FAIR examples and tutorials as self-contained local demonstrations; state Colab support precisely.
- Update FAIR tests, examples, tutorials, and affected documentation when `tagon/fair/` behavior changes.

CWL interoperability. DAGonStar workflow, task, and dependency changes must preserve the CWL v1.2 interoperability contract exposed by `Workflow.saveAsCWL(filename)`:

- Keep CWL exports standards-valid, dependency-free in the base installation, deterministic, and credential-safe.
- Be explicit about the boundary between portable command-graph semantics and DAGonStar-specific execution or staging.
- Any behavior change affecting CWL export must update the automated CWL tests (`tests/test_cwl_example.py`), `examples/cwl/`, `docs/cwl_export.md`, the CWL example guide, the CWL tutorial lesson, and all affected API documentation.

B.13 Current Technical Debt

- Automated tests cover core local behavior, checkpointing, staging, command failure propagation, packaging extras, and selected mocked optional-boundary behavior; broader integration coverage is still needed.
- Public APIs have broad annotations across workflow, task, configuration, staging, task-subclass, and API-client code; type coverage continues to be tightened when touching public methods.
- Shell command construction uses centralized POSIX quoting for core task launchers, Slurm, Docker, remote lifecycle commands, staging paths, and SCP remote-endpoint prefixes; it should still be extended when touching legacy/site-specific integrations.
- Optional integrations now have package extras, while `requirements.txt` remains a full development/demo environment; lazy imports and explicit extras for Docker, cloud, Globus, API, and other service-specific requirements continue to be preserved.
- Some legacy examples require site-specific data or services (??).

Appendix C

Consolidated Configuration Reference

This appendix reproduces a complete, consolidated `dragon.ini` reference, assembling every section discussed individually in ??, for use as a single lookup document while authoring workflows.

C.1 How to Use This Appendix

Two reading strategies suit two different needs. A reader setting up a new workflow’s configuration for the first time should treat the consolidated listing below as a *starting template*: copy it, delete every section the workflow’s task types do not actually require (per the backend-selection guidance of ??), and fill in only the remaining, genuinely needed sections — resisting the temptation to retain unused sections “just in case,” since an unused but populated section is, per ??’s auditing-procedure discussion, one more place a stray credential placeholder could later be mistaken for a real value. A reader instead debugging an existing, unfamiliar `dragon.ini` should use the quick-reference table at the end of this appendix as an index, then follow each entry’s chapter cross-reference back to ??’s fuller discussion of that section’s semantics and failure modes.

```
1 ; -----
2 ; Optional DAGOn service (disabled by default; see Sec. 5.2.1)
3 ; -----
4 [dragon_service]
5 route = http://localhost:57000
6 use = False
7
8 ; -----
9 ; FTP publishing / transversal data download (Sec. 5.2.2)
10 ; -----
11 [ftp_pub]
12 ip      = localhost
13 user    = anonymous
14 password =
15
16 ; -----
17 ; Service/node addressing (Sec. 5.2.3)
18 ; -----
19 [dragon_ip]
20 ip = localhost
21
22 ; -----
23 ; Local batch execution and scratch layout (Sec. 5.2.4)
24 ; -----
25 [batch]
```

```

26 scratch_dir_base=/tmp/
27 run_base=
28 remove_dir=False
29 threads=1
30
31 ; -----
32 ; Slurm scheduler defaults (Sec. 5.2.5)
33 ; -----
34 [slurm]
35 partition=
36
37 ; -----
38 ; LLM providers: any number of [llm.<name>] sections (Sec. 5.2.6)
39 ; -----
40 [llm.example]
41 endpoint=https://api.example.org
42 api_key_env=DAGON_LLM_EXAMPLE_API_KEY
43 model=example-chat-model
44
45 ; -----
46 ; Cloud providers (Sec. 5.2.8)
47 ; -----
48 [ec2]
49 key=<aws_access_key_id>
50 secret=<aws_secret_access_key>
51 region=
52
53 [digitalocean]
54 key=
55
56 [gce]
57 key=
58 secret=
59 project=
60
61 ; -----
62 ; Standard Python logging configuration (Sec. 5.2.9)
63 ; -----
64 [loggers]
65 keys=root
66
67 [handlers]
68 keys=stream_handler
69
70 [formatters]
71 keys=formatter
72
73 [logger_root]
74 level=DEBUG
75 handlers=stream_handler
76
77 [handler_stream_handler]
78 class=StreamHandler
79 level=DEBUG
80 formatter=formatter
81 args=(sys.stderr,)
82
83 [formatter_formatter]
84 format=%(asctime)s %(name)-12s %(levelname)-8s %(message)s

```

Listing C.1: A consolidated, illustrative `dagong.ini` (placeholders only — never commit real secrets).

C.2 Worked Example: Configuring a Two-Backend Workflow

To make the abstract template above concrete, consider configuring a workflow that mixes a local BATCH preparation task with a SLURM-scheduled simulation and an LLM-based summarization step — a three-backend combination representative of the synthesis chapter’s forecasting-chain pattern (??). Applying the “delete what is unused” principle from the previous section, the correctly trimmed configuration retains only:

```

1  [dagon_service]
2  use = False
3
4  [batch]
5  scratch_dir_base=/data/dagon-scratch/
6  run_base=%Y%m%d-%H%M%S
7  threads=4
8
9  [slurm]
10 partition=compute
11
12 [llm.research]
13 endpoint=https://api.example.org
14 api_key_env=DAGON_RESEARCH_LLM_KEY
15 model=research-chat

```

Note what was deliberately omitted relative to the full template: no cloud provider section (no CLOUD task is used), no ftp_pub or dagon_ip section (no transversal or service-backed task is used), and a scratch_dir_base deliberately moved off /tmp/ to durable storage, per ??’s caution about treating /tmp as durable across a checkpointed, multi-session operational deployment. The logging sections are also omitted here on the assumption that this configuration is supplied as an in-memory config dictionary rather than loaded via config_file — recall from ??’s discussion of the layered override mechanism that fileConfig() is invoked only on the file-loading path, so a purely programmatic configuration has no logging-section obligation the file-based path would otherwise impose.

C.3 Runtime Environment Variables (not in dagon.ini)

```

1  ; SKYCDS staging credentials
2  export DAGON_SKYCDS_CLIENT_TOKEN=...
3  export DAGON_SKYCDS_CATALOG_TOKEN=...
4  export DAGON_SKYCDS_API_TOKEN=...
5  export DAGON_SKYCDS_IP=...
6
7  ; Globus transfer
8  export DAGON_GLOBUS_TRANSFER_TOKEN=...
9
10 ; LLM provider credential, named by api_key_env above
11 export DAGON_LLM_EXAMPLE_API_KEY=...
12
13 ; Web task authentication, named by token_env / username_env / password_env
14 export SERVICE_TOKEN=...

```

This environment-variable list is, collectively, the concrete instantiation of the security-expectations discipline reproduced in Appendix B: every one of these variables exists specifically so that the corresponding *_env-suffixed configuration key (or, in SKYCDS’s and Globus’s case, a documented fixed variable name) can name a secret *indirectly*, without that secret ever appearing as a literal string anywhere in versioned configuration, workflow source, or serialized workflow JSON (recall ??’s note that as_json() can serialize a full workflow, including its

LLM task provider references — a further, concrete reason the credential itself must never be embedded directly in anything that serialization might capture).

C.4 Quick-Reference Table

Section	Purpose
[dagon_service]	Optional remote registration/status service; <code>use=False</code> for local-only operation.
[ftp_pub]	FTP host for transversal workflow data download.
[dagon_ip]	Service/node addressing; not required locally.
[batch]	Scratch directory base, run grouping, staging thread count.
[slurm]	Default Slurm partition for staging command generation.
[llm.<name>]	One section per OpenAI-compatible LLM provider.
[ec2], [digitalocean], [gce]	Cloud provider credentials for CLOUD tasks (via Apache Libcloud).
[loggers], [handlers], [formatters], and related sections	Standard Python <code>logging.config.fileConfig</code> setup, required when <code>Workflow</code> is constructed without an explicit <code>config</code> dictionary.

Scientific Practice Note

As a closing reminder consolidating ??’s scattered cautions: commit only safe templates (such as `dagon.ini.sample`); never commit real secrets; record the configuration values that affect scientific reproducibility (scratch layout, model parameters, container images, cloud instance types, Slurm resource requests); and keep operational secrets strictly separate from reproducibility metadata by referencing only environment-variable *names* in versioned configuration.

Section	Purpose
[dagon_service]	Optional remote registration/status service; <code>use=False</code> for local-only operation.
[ftp_pub]	FTP host for transversal workflow data download.
[dagon_ip]	Service/node addressing; not required locally.
[batch]	Scratch directory base, run grouping, staging thread count.
[slurm]	Default Slurm partition for staging command generation.
[llm.<name>]	One section per OpenAI-compatible LLM provider.
[ec2], [digitalocean], [gce]	Cloud provider credentials for CLOUD tasks (via Apache Libcloud).
[loggers], [handlers], [formatters], and related sections	Standard Python <code>logging.config.fileConfig</code> setup, required when <code>Workflow</code> is constructed without an explicit <code>config</code> dictionary.

Scientific Practice Note

As a closing reminder consolidating ??’s scattered cautions: commit only safe templates (such as `dragon.ini.sample`); never commit real secrets; record the configuration values that affect scientific reproducibility (scratch layout, model parameters, container images, cloud instance types, Slurm resource requests); and keep operational secrets strictly separate from reproducibility metadata by referencing only environment-variable *names* in versioned configuration.

Appendix D

Checkpoint Semantics Reference

This appendix collects, in full, the checkpoint semantics introduced in ??, for use as a standing reference when designing resumable, long-running scientific workflows.

D.1 Checkpointing as a Special Case of the Provenance Graph

Before working through the mechanics, it is worth connecting checkpoint semantics explicitly to the provenance-graph concept developed in ??. A checkpoint file, in the precise sense developed there, is a *partial, persisted snapshot* of the provenance graph’s node annotations — specifically, the `working_dir` and final exit code of every task that has completed at least once — written to a location (the checkpoint JSON file) that survives independently of the Python process that produced it. This framing clarifies precisely what a checkpoint file does and does not capture: it captures exactly the same two facts ??’s debugging procedure already treats as foundational — where a task’s output lives, and whether the task succeeded — and nothing more. It does not, in particular, capture the `launcher.sh` content itself, which is why, as the limitations section below reiterates, checkpoint reuse cannot by itself detect that a task’s *command* changed between the run that produced the checkpoint entry and the run attempting to reuse it.

D.2 What Checkpointing Records

When a `Workflow` is created with `checkpoint_file`, DAGonStar writes a JSON file containing: the workflow scratch base; per-task working directories; workflow name; task name; and the task exit code after execution.

```
1 {
2   "_scratch_dir": "/tmp/",
3   "Experiment.A": {
4     "working_dir": "/tmp/1710000000000-A",
5     "workflow": "Experiment",
6     "name": "A",
7     "code": 0
8   }
9 }
```

Listing D.1: Representative checkpoint JSON shape.

D.3 Creating and Resuming a Checkpointed Workflow

```

1 from dagon import Workflow
2 from dagon.task import DagonTask, TaskType
3
4
5 workflow = Workflow("Experiment", checkpoint_file="checkpoint.json")
6
7 task = DagonTask(TaskType.BATCH, "A", "echo result > result.txt")
8 workflow.add_task(task)
9 workflow.make_dependencies()
10 workflow.run()

```

Listing D.2: Creation.

```

1 workflow = Workflow("Experiment", checkpoint_file="checkpoint.json")
2 task = DagonTask(TaskType.BATCH, "A", "echo result > result.txt")
3 workflow.add_task(task)
4 workflow.make_dependencies()
5 workflow.run(resume_checkpoint_file="checkpoint.json")

```

Listing D.3: Resumption.

If the checkpoint records `code: 0` for `Experiment.A` and the recorded working directory still exists, the task can be treated as already complete and is skipped on the resumed run.

D.4 Task Identity

Checkpoint keys are built from `<workflow-name>.<task-name>`. Stable workflow and task names are therefore essential: renaming a task is, from the checkpoint system’s perspective, indistinguishable from introducing an entirely new task with no recorded history.

This naming-stability requirement deserves a slightly fuller treatment than a single warning sentence, since it interacts with ordinary software maintenance in a way that is easy to overlook. As a workflow’s Python source evolves over the lifetime of a research project — task names refactored for clarity, a workflow renamed to reflect a renamed project — every such rename silently orphans the corresponding checkpoint entries: the old key remains in the checkpoint JSON, referring to a working directory that may still exist on disk, while the renamed task generates a new key with no checkpoint history of its own. This is not a bug; it follows directly from the dictionary-key-based identity scheme described above. It does mean, however, that a workflow author renaming tasks in an actively used, checkpointed workflow should treat the rename as equivalent, from a checkpoint-continuity perspective, to discarding all prior checkpoint history for the renamed tasks, and plan accordingly — for instance, by completing any in-flight, expensive run under the old names before applying the rename, rather than renaming mid-run and hoping for seamless resumption.

D.5 Working Directory Persistence

Checkpoint reuse requires the recorded working directory to exist. If scratch directories are deleted, moved, or garbage-collected, checkpoint reuse cannot recover those task outputs. For long-running scientific workflows, scratch directories should be placed on storage with a retention policy appropriate to the experiment’s expected lifetime.

D.6 Checkpointing and Dataflow

Checkpointing is most valuable for dataflow workflows whose upstream tasks produce large intermediate files. Given the chain

```
1 download -> preprocess -> simulate -> postprocess
```

if `simulate` fails, a later run may reuse `download` and `preprocess` if their working directories and checkpoint entries remain valid — avoiding, potentially, hours of redundant data acquisition and preprocessing for a single failed simulation attempt.

This four-stage example is worth re-examining quantitatively, building on ??’s staging-cost discussion: the value checkpointing delivers in this scenario is directly proportional to the cost of the stages it allows a researcher to *skip* on resumption, relative to the cost of the stage that actually failed. If `download` and `preprocess` together consume four hours of wall-clock time and `simulate` consumes thirty minutes before failing, checkpoint reuse converts a worst-case four-and-a-half-hour recovery cost into a thirty-minute one — a substantial, quantifiable return on the comparatively small overhead of supplying a `checkpoint_file` argument. The same calculation, run in reverse, also clarifies when checkpointing delivers little practical benefit: a workflow whose every stage is roughly equally cheap gains comparatively little from checkpoint reuse, since even a full restart from scratch is itself inexpensive.

D.7 Explicit Checkpoint Tasks

`TaskType.CHECKPOINT` maps to `dagom.checkpoint.Checkpoint`. This task type is part of the implementation for explicit checkpoint behavior; the workflow-level `checkpoint_file` mechanism described above remains the primary, user-facing mechanism for recording and resuming task metadata.

D.8 Remote, Docker, and Cloud Checkpointing

Remote checkpoint reuse checks for the existence of the recorded directory on the remote host through the task’s SSH connection, which means:

- the same host must be reachable;
- credentials must still work;
- the remote scratch directory must still exist;
- paths must be valid on the remote filesystem.

Caution

Docker and cloud task checkpointing depends on the persistence of the host scratch directory and, for cloud tasks, on the persistence or reconstruction of the remote execution environment. Do not assume a terminated cloud instance preserves local scratch data unless it was explicitly written to persistent storage.

These three additional preconditions — host reachability, credential validity, and remote filesystem persistence — compose directly with the backend-specific operational considerations already developed in ???: a remote or Slurm-scheduled task’s queueing and latency costs (already discussed there) determine how *expensive* a failure partway through that task is, while this section’s preconditions determine whether checkpoint reuse can actually *recover* from that failure without re-incurring those costs. A workflow author relying on checkpoint reuse for a remote or cloud-backed stage should verify both sets of considerations together, not in isolation.

D.9 Scientific Usage Pattern

For expensive workflows, the documentation recommends the following seven-step discipline:

1. Use a stable workflow name.
2. Use stable task names.
3. Set `checkpoint_file`.
4. Set a scratch base with enough storage.
5. Preserve the checkpoint JSON with experiment metadata.
6. Preserve or archive task working directories needed for reuse.
7. On rerun, use `resume_checkpoint_file`.

D.10 Limitations and Cautions

Caution

- Checkpoints record task *success* and working directory, not full semantic equivalence of code, inputs, or parameters.
- If a command changes but the task name and previous success remain, checkpoint reuse may incorrectly skip recomputation.
- For rigorous scientific reproducibility, store code version, configuration, input checksums, and software environment *alongside* the checkpoint file, not only inside it.
- Garbage collection or manual cleanup can invalidate checkpoint reuse.

The first two items of this list deserve to be read together, since they describe the same underlying limitation from two angles: a checkpoint entry’s identity is purely nominal (task name) and purely behavioral (recorded exit code), never *content-based*. A content-based scheme — recording, say, a hash of the task’s fully resolved command, as captured in `launcher.sh` — would in principle detect a changed command automatically and correctly invalidate the corresponding checkpoint entry, at the cost of additional implementation complexity the current, simpler name-and-status scheme avoids. A workflow author who needs this stronger guarantee today must provide it manually, by following the fifth bullet’s recommendation to preserve code version and configuration alongside the checkpoint file, and by manually deleting or ignoring stale checkpoint entries whenever a task’s command is deliberately changed.

D.11 Verification

```
1 cd examples/dataflow/batch
2 python dataflow-checkpoint-demo.py
```

Then inspect the generated checkpoint file named by the example script. A minimal custom test illustrating two successive resumed runs:

```
1 from dagon import Workflow
2 from dagon.task import DagonTask, TaskType
3
4
5 def build():
6     workflow = Workflow("CheckpointDemo", checkpoint_file="checkpoint.json")
7     task = DagonTask(TaskType.BATCH, "A", "echo data > data.txt")
8     workflow.add_task(task)
9     workflow.make_dependencies()
10    return workflow
11
12
13 build().run(resume_checkpoint_file="checkpoint.json")
14 build().run(resume_checkpoint_file="checkpoint.json")
```

The second run can reuse the completed task if the recorded directory remains on disk.

Appendix E

Google Colab Reference

This appendix provides a concise, standalone reference for running DAGonStar workflows in Google Colab. It complements the extended discussion in ?? and is intended for readers who wish to use Colab as their primary teaching or experimentation environment without consulting the full chapter.

E.1 What Is Google Colab?

Google Colaboratory (Colab) is a hosted, browser-based Jupyter notebook service provided by Google. Each Colab notebook runs inside an ephemeral virtual machine (*runtime*) that provides a free-tier CPU environment with Python pre-installed and network connectivity. Runtimes are not persistent: all files stored outside a mounted Google Drive directory are deleted when the runtime is recycled, which typically occurs after a period of inactivity or at the end of a session. Premium tiers offer GPU, TPU, and longer session lifetimes, but the standard free tier is entirely adequate for all DAGonStar tutorial lessons.

Caution

Colab is a teaching and prototyping environment, not a production scheduler. Long-running forecasting chains, workflows that require a persistent Docker daemon, or pipelines that must reliably survive a machine restart should be run on a local workstation, a dedicated server, or an HPC cluster — not on a hosted Colab runtime.

E.2 Capability Summary

DAGonStar Capability	Colab?	Notes
Local BATCH and Bash tasks	Yes	Best-supported mode.
Notebook-driven workflow creation	Yes	Interactive cell execution.
<code>workflow://</code> references	Yes	Keep files in <code>/content</code> .
Checkpointing	Yes, with care	Use Drive for persistence.
LLM tasks (mock provider)	Yes	No credentials required.
Web tasks (local echo server)	Yes	Bound to <code>127.0.0.1</code> .
Native Python tasks	Yes	Module must be importable.
Asynchronous <code>launch()</code>	Yes	Keep tab connected.
SSH/remote execution	Possible	Needs reachable host and short-lived credentials.
Slurm (as client)	Possible	Colab submits; it is not the cluster.
Docker tasks	No	No Docker daemon in hosted runtimes.
Cloud tasks	Possible	Credentials must not appear in notebook source.
Long production workflows	Not recommended	Runtimes are ephemeral.

E.3 Installation Paths

Three installation paths are available in a Colab cell:

```
1 !pip install -U pip
2 !pip install dagonstar
```

Listing E.1: Option 1: PyPI stable release.

```
1 !pip install git+https://github.com/DagOnStar/dagonstar.git
```

Listing E.2: Option 2: latest repository version, no checkout.

```
1 !git clone https://github.com/DagOnStar/dagonstar.git
2 %cd dagonstar
3 !pip install -e .
```

Listing E.3: Option 3: full repository checkout (required for tutorial scripts and examples/).

Option 3 is required when running tutorial scripts that reference files within the repository tree, such as the LLM and Web lesson examples:

```
1 !python examples/tutorial/lesson_12_llm_tasks.py
2 !python examples/tutorial/lesson_13_web_tasks.py
```

Listing E.4: Running tutorial scripts from a Colab cell (requires Option 3).

E.4 The Standard Configuration Helper

All tutorial notebook cells use the following in-memory configuration helper, which avoids the need for a `dagon.ini` file:

```

1 from pathlib import Path
2 import tempfile
3
4 def local_config(workdir: Path) -> dict:
5     return {
6         "batch": {
7             "scratch_dir_base": str(workdir),
8             "run_base": "",
9             "remove_dir": "False",
10            "threads": "1",
11        },
12        "dagon_service": {"use": "False"},
13        "ftp_pub": {
14            "ip": "localhost",
15            "user": "anonymous",
16            "password": "",
17        },
18    }
19
20 def workspace(name: str) -> Path:
21     base = Path(tempfile.mkdtemp(prefix="dagonstar-nb-"))
22     (base / name).mkdir(parents=True, exist_ok=True)
23     return base / name

```

Listing E.5: Standard `local_config()` helper used throughout the tutorial notebook.

The three fields in `ftp_pub` (`ip`, `user`, `password`) are all required. Omitting `user` or `password` raises a `KeyError` at runtime when the transversal data-movement path is exercised.

E.5 Persistence: Google Drive

All file artifacts written to `/content/` (the default Colab runtime filesystem) are deleted when the runtime is recycled. To preserve scratch directories and checkpoint files across sessions, mount Google Drive and redirect the `scratch_dir_base`:

```

1 from google.colab import drive
2 drive.mount('/content/drive')
3
4 WORKDIR = "/content/drive/MyDrive/dagonstar-experiments"
5
6 config = {
7     "batch": {
8         "scratch_dir_base": WORKDIR,
9         "run_base": "",
10        "remove_dir": "False",
11        "threads": "1",
12    },
13    "dagon_service": {"use": "False"},
14    "ftp_pub": {"ip": "localhost", "user": "anonymous", "password": ""},
15 }

```

Listing E.6: Mounting Google Drive for persistent storage.

E.6 Output Inspection in a Notebook Cell

Rather than using shell commands (`cat`, `find`), notebook cells use `pathlib.Path` and `task.get_scratch_dir()` to read task outputs:

```

1 from pathlib import Path
2
3 # After workflow.run():
4 task_dir = Path(task.get_scratch_dir())
5 print(list(task_dir.iterdir()))           # all outputs
6 print((task_dir / "result.txt").read_text()) # a specific file
7 print((task_dir / ".dagon" / "launcher.sh").read_text()) # generated command
8 print((task_dir / ".dagon" / "stdout.txt").read_text()) # captured output

```

Listing E.7: Python-native task output inspection.

E.7 Running the Tutorial Notebook

The repository provides a 32-cell unified tutorial notebook at:

`docs/tutorial/DAGonStar_tutorial.ipynb`

Open it in Colab using one of the following methods:

1. Navigate to https://github.com/DagOnStar/dagonstar/blob/master/docs/tutorial/DAGonStar_tutorial.ipynb and click **Open in Colab** (if the GitHub badge is present), or copy the URL into the **File** → **Open notebook** → **GitHub** tab in Colab.
2. Clone the repository via Option 3 above and open the notebook file from the Colab file browser.

Run cells in order. Lessons 08 (Docker) and 09 (Slurm, live execution) will display an informative message rather than executing, since the corresponding services are unavailable in hosted Colab. All other fourteen lessons run to completion.

E.8 The Ready-to-Use Colab Examples

Two self-contained scripts with paired notebooks are available in `examples/colab/`:

hello_colab.py / hello_colab.ipynb

A three-task chain (`write_hello` → `decorate_hello` → `validate`) using `workflow://` references. Demonstrates in-memory configuration, explicit `make_dependencies()`, and `task.get_scratch_dir()`.

workflow_files_colab.py / workflow_files_colab.ipynb

A three-stage CSV pipeline (`prepare_data` → `transform_data` → `summarize_data`) illustrating a realistic data-processing chain with two sequential `workflow://` dependency edges.

```

1 python examples/colab/hello_colab.py
2 python examples/colab/workflow_files_colab.py

```

Listing E.8: Running the Colab examples after a repository checkout.

E.9 Security in Colab Notebooks

The documentation states the security rule without exception: *never commit secrets or paste permanent private keys into a notebook*. In practice this means:

- Use **Colab Secrets** (Tools → Secrets) to store API keys, tokens, and passwords; retrieve them at runtime with `from google.colab import userdata`.
- For LLM tasks, set `api_key_env = MY_KEY` in the configuration, then expose the key via `os.environ["MY_KEY"] = userdata.get("MY_KEY")` before calling `workflow.run()`.
- For Web task authentication, use the `token_env`, `username_env`, or `password_env` indication mechanisms documented in ??.
- Treat each Colab session as a fresh, ephemeral machine: reinstall packages and re-authenticate after every runtime reset.

E.10 Troubleshooting

ModuleNotFoundError: No module named 'dagon'

The package was not installed in the current runtime session. Run the installation cell again; each fresh runtime requires re-installation.

Relative-path errors when running examples

Use `%cd /content/dagonstar` before running any script that references files relative to the repository root.

Scratch directory not found after resuming

The runtime was recycled; `/content/` does not persist. Mount Google Drive and set `scratch_dir_base` to a Drive path as shown in ??.

Docker task fails immediately

Docker is not available in hosted Colab. Run Docker lessons on a local workstation or a cloud VM with Docker installed.

KeyError: 'user' in ftp_pub

The in-memory configuration dictionary is missing the `user` and `password` keys in the `ftp_pub` section. Use the complete `local_config()` helper from ??.

Appendix F

GitHub: Repository Structure, Usage, and Contribution

This appendix guides both users who wish to obtain and track DAGonStar from GitHub, and contributors who wish to improve the engine’s code, documentation, or examples. The material is grounded in the repository’s own `AGENTS.md` and `docs/developer_guide.md` files, which constitute the authoritative contributor reference.

F.1 The DAGonStar Repository

The canonical repository is hosted at:

<https://github.com/DagOnStar/dagonstar>

It is organized under the **DagOnStar** GitHub organization. The default branch is `master`. The repository’s top-level structure reflects the layered architecture described in ??:

Path	Contents
<code>dagon/</code>	Core library: <code>task.py</code> , <code>__init__.py</code> (Workflow), <code>batch.py</code> , <code>llm.py</code> , <code>web/</code> , <code>native.py</code> , etc.
<code>docs/</code>	All documentation: <code>tutorial/</code> (14 lessons + notebook), <code>examples/</code> , architecture, reference guide, user guide.
<code>examples/</code>	Runnable examples: <code>colab/</code> , <code>dataflow/</code> , <code>envapp/</code> , <code>taskflow/</code> , <code>tutorial/</code> , <code>llm/</code> , <code>web/</code> , <code>native/</code> , etc.
<code>tests/</code>	Unit tests (fast, local, no external services required).
<code>dagon.ini.sample</code>	Safe sample configuration (no credentials).
<code>AGENTS.md</code>	Working principles and verification commands for contributors and AI agents.
<code>README.md</code>	Project overview, motivation, acknowledgments, publication list, installation instructions.

F.2 Obtaining DAGonStar from GitHub

F.2.1 Cloning the Repository

To obtain a full working copy of the repository:

```
1 git clone https://github.com/DagOnStar/dagonstar.git
2 cd dagonstar
```

Listing F.1: Cloning the default master branch.

To clone a specific branch (for example, the `development-native-web` branch used as the basis for earlier editions of this textbook):

```
1 git clone --branch development-native-web \
2     https://github.com/DagOnStar/dagonstar.git
```

F.2.2 Staying Up to Date

To update a local clone with the latest changes from the remote:

```
1 cd dagonstar
2 git pull origin master
3 pip install -e .      # reinstall after source changes
```

Listing F.2: Pulling the latest changes.

The `pip install -e .` step is necessary after any change to the package metadata (`setup.cfg`, `pyproject.toml`) or to files that are imported rather than re-read at runtime.

F.2.3 Pinning to a Specific Commit

For reproducible research environments, it is often preferable to pin the installation to a specific, reviewed commit:

```
1 pip install git+https://github.com/DagOnStar/dagonstar.git@<commit-sha>
```

Listing F.3: Installing a pinned commit.

Record the commit SHA alongside the workflow source and configuration in the experiment’s reproducibility supplement.

F.3 Repository Branches and Releases

master

The primary branch. Receives bug fixes, documentation improvements, and new features after review. The PyPI release tracks this branch.

development-native-web

A feature branch introducing the LLM, Web, and Native task types alongside the web-service-backed workflow schema. This branch was the basis for earlier editions of this textbook and remains available for reference.

Other branches

Feature branches for ongoing development work. Contributors typically open a dedicated branch from **master** for each change set.

F.4 Using GitHub Issues

GitHub Issues are the primary channel for reporting defects, requesting features, and asking questions about the DAGOnStar engine or its documentation:

1. Navigate to <https://github.com/DagOnStar/dagonstar/issues>.
2. Click **New issue**.
3. Provide a concise title and a detailed description that includes: the Python version (`python -version`), the installed DAGonStar version or commit SHA, the minimal code that reproduces the issue, and the full error message or unexpected output.
4. Assign relevant labels if available (`bug`, `documentation`, `enhancement`).

Scientific Practice Note

The project documentation's ground rule — *documentation must remain consistent with the current implementation* — applies equally to issue reports: a well-formed issue includes a verification command that can be run locally to reproduce the reported behavior, exactly as `AGENTS.md` requires for code changes.

F.5 Forking and the Pull-Request Workflow

Contributions to DAGonStar follow the standard GitHub fork-and-pull-request model:

1. **Fork** the repository by clicking **Fork** on the GitHub page. This creates a personal copy under your account.
2. **Clone** your fork locally:

```
1 git clone https://github.com/<your-username>/dagonstar.git
2 cd dagonstar
3 git remote add upstream https://github.com/DagOnStar/dagonstar.git
```

3. **Create a feature branch** from master:

```
1 git checkout master && git pull upstream master
2 git checkout -b feature/my-improvement
```

4. **Make your changes**, following the working principles in ??.
5. **Verify** using the commands in ??.
6. **Commit** with a concise, imperative-mood commit message (e.g. Add user and password fields to ftp_pub section):

```
1 git add -p # stage changes interactively
2 git commit -m "Brief description of the change"
```

7. **Push** to your fork:

```
1 git push origin feature/my-improvement
```

8. **Open a pull request** on GitHub by navigating to the upstream repository and clicking **Compare & pull request**. Provide a description that explains *what* was changed, *why*, and the verification command used to confirm correctness.

F.6 Working Principles for Contributors

The following principles are extracted verbatim from `AGENTS.md`, the repository’s authoritative contributor guidance:

- Preserve existing research examples unless a change is clearly required.
- Do not commit real credentials, tokens, host-specific passwords, SSH keys, or cloud secrets.
- Treat `tagon.ini.sample` as documentation, not as a private configuration file.
- Keep fixes conservative: many examples encode historical research workflows.
- Prefer small, reviewable changes with an explicit verification command.

Additionally, from `docs/developer_guide.md`:

- Documentation, examples, and tests must describe the *current* implementation. If intended behavior differs from current behavior, either implement and test it, or label it explicitly as future work.
- When adding or changing features, add a unit test under `tests/` for the expected stable behavior.
- Quote new path and user-controlled inputs defensively in any new shell-command construction.

F.7 Verification Commands

Every pull request must pass, at minimum, the following verification sequence:

```

1 # 1. Run the unit test suite
2 python -m unittest discover -s tests -v
3
4 # 2. Compile all source files (catches syntax errors in non-tested modules)
5 python -m py_compile \
6     dagon/*.py \
7     dagon/api/*.py \
8     dagon/communication/*.py \
9     dagon/ftp_publisher/*.py \
10    dagon/peer2peer/*.py \
11    tests/*.py
12
13 # 3. Style and type checks (install first if not present)
14 #     pip install mypy==1.13.0 ruff==0.6.9
15 python -m ruff check dagon/shell.py tests
16 python -m mypy dagon/shell.py
17
18 # 4. If packaging metadata changed:
19 python -m pip install -e .

```

Listing F.4: Required verification commands (from `AGENTS.md`).

Caution

The unit test suite is deliberately designed to avoid external services (Docker, SSH, Slurm, cloud providers, Globus). If your change touches a live-service integration boundary, provide a separate manual verification command in the pull-request description that

can be run by a reviewer with access to the relevant service.

F.8 Contributing to Documentation

Documentation improvements are among the most impactful and accessible contributions to an open research software project. The DAGonStar documentation lives entirely within the repository tree under `docs/` and `examples/`, so documentation changes follow the same fork-and-pull-request workflow as code changes.

F.8.1 Scope of Documentation Changes

- **Typo corrections and clarifications:** the lowest barrier; open a pull request directly with the corrected `.md` file.
- **New worked examples:** add a `.py` script under the appropriate `examples/` subdirectory, together with a `README.md` classifying the example per the four-category verification scheme described in ??.
- **New tutorial lessons:** follow the existing lesson structure (objective → concept → code → expected behavior → verification → scientific practice note), add the `.md` file to `docs/tutorial/`, and add a corresponding cell to `docs/tutorial/DAGonStar_tutorial.ipynb`.
- **New documentation guides:** place the `.md` file under `docs/` and cross-reference it from `docs/README.md` and the `README.md` at the repository root.

F.8.2 The Documentation Consistency Principle

The most important rule for documentation contributors, stated by the project and reproduced throughout this textbook:

Documentation must remain consistent with the software in this repository. If intended behavior differs from current behavior, either implement and test it, or label it explicitly as future work.

This principle is not a bureaucratic formality; it is the mechanism that makes the documentation usable as a primary reference for reproducibility-conscious researchers. A documentation change that describes aspirational behavior without the corresponding code implementation actively misleads users and violates the trust that makes research software documentation credible.

F.9 Contributing a New Task Type

Adding a new execution backend follows the seven-step recipe documented in Appendix B and repeated here for completeness:

1. Add a value to the `TaskType` enumeration.
2. Add a mapping in the `tasks_types` dispatch dictionary.
3. Implement the concrete task class, extending `Task`.
4. Initialize the `Task` base class correctly in `__init__`.

5. Override `on_execute()` with backend-specific logic.
6. Add unit tests for construction, dependency behavior, and any new configuration key.
7. Update `docs/reference_guide.md`, `docs/architecture.md`, and the relevant tutorial lesson or example.

A well-formed pull request for a new task type includes: the source file(s), at least one unit test, documentation updates, and a runnable local example with a local mock or structural-verification variant that requires no external service.

F.10 Using GitHub Actions and CI

The repository uses GitHub Actions for continuous integration. On each push and pull request, CI runs:

- The unit test suite (`python -m unittest discover -s tests -v`).
- A compile check of all Python source files.
- `ruff` style checks on `tagon/shell.py` and `tests/`.
- `mypy` type checks on `tagon/shell.py`.

A pull request that fails any CI check will not be merged until the failure is resolved. CI results are visible on the pull request page and on the repository's **Actions** tab.

F.11 Finding Help and Staying Connected

- **Issues:** <https://github.com/DagOnStar/dagonstar/issues> — for bug reports, feature requests, and questions.
- **Pull requests:** <https://github.com/DagOnStar/dagonstar/pulls> — for code and documentation contributions.
- **Repository discussions** (if enabled): for open-ended questions about workflow design, research use cases, or integration strategies.
- **Author contact:** `raffaele.montella@uniparthenope.it` — for academic collaboration enquiries.
- **Production deployment:** the Center for Monitoring and Modeling Marine and Atmosphere applications at the University of Naples “Parthenope” (<https://meteo.uniparthenope.it>) operates DAGOnStar in production for the MytilX, MytilEx, and ADMIRE projects described in the Bibliography.

Appendix G

External Service Provider Configuration

This appendix is the authoritative configuration reference for every external service provider supported by DAGonStar. It covers cloud execution backends, FaaS providers, IoT providers, Kubernetes, Apptainer, Nomad, and storage/staging integrations. The organizing principle is the same as the rest of this textbook: credentials must never be embedded in committed files; environment variables or provider-standard credential chains are always the correct mechanism.

G.1 General Configuration Principles

1. **Never commit credentials.** The `tagon.ini.sample` file is documentation. It must contain only placeholder values, never real passwords, tokens, or private keys.
2. **Reference secrets through environment variable names**, never through literal values, in any DAGonStar configuration key or task constructor argument.
3. **Use provider-standard credential chains** (AWS IAM instance profiles, Azure `DefaultAzureCredential`, GCP Application Default Credentials) in preference to static credentials wherever the provider supports them.
4. **Validate connectivity before wiring tasks.** SSH, Slurm, Docker, and Kubernetes connectivity must be verified independently before a workflow is composed and executed.
5. **Declare the verification level** of any example or workflow that depends on an external service (see ??).

G.2 Remote Execution: SSH

Remote BATCH and SLURM tasks require SSH access. Verify connectivity before constructing tasks:

```
1 ssh -i ~/.ssh/id_rsa user@hpc.example.org "hostname && module list"
```

Listing G.1: SSH connectivity verification.

In `tagon.ini`:

```
1 [batch]
2 scratch_dir_base = /tmp/
3 run_base         =
4 remove_dir       = False
```

```

5 threads          = 1
6
7 [remote]
8 host             = hpc.example.org
9 username         = jsmith
10 keypath          = /home/jsmith/.ssh/id_rsa

```

Listing G.2: Remote task configuration.

In the task constructor:

```

1 remote_task = DagonTask(
2     TaskType.BATCH,
3     "remote_step",
4     "bash run.sh workflow:///prepare/forcing.nc",
5     ip           = "hpc.example.org",
6     ssh_username = "jsmith",
7     keypath      = "/home/jsmith/.ssh/id_rsa",
8 )

```

Listing G.3: RemoteBatch task construction.

G.3 Slurm Configuration

```

1 [slurm]
2 partition =
3 ; Additional sbatch defaults may be added here.
4 ; Never include credentials.

```

Listing G.4: Slurm section in dagon.ini.

```

1 sim = DagonTask(
2     TaskType.SLURM,
3     "run_wrf",
4     "bash run_wrf.sh workflow:///prepare/wrfinput_d01",
5     partition = "compute",
6     ntasks    = 64,
7     time      = "06:00:00",
8     nodes     = 2,
9     memory    = "128G",
10    account   = "my_project",
11 )

```

Listing G.5: Slurm task with resource specification.

Structural verification without a live cluster:

```

1 python -c "
2 from dagon.task import DagonTask, TaskType
3 t = DagonTask(TaskType.SLURM, 'sim', 'bash run.sh',
4               partition='compute', ntasks=64, time='06:00:00')
5 print(t.generate_command('launcher.sh'))
6 "

```

G.4 Docker and Container Configuration

```

1 [docker]
2 ; No credentials in this file.
3 ; Use docker login / DOCKER_CONFIG for private registry auth.

```

Listing G.6: Docker section (credentials never here).

```

1 post = DagonTask(
2     TaskType.DOCKER,
3     "postprocess",
4     "python render.py --wrfout /in/wrfout_d01",
5     image = "lab/gdal-postprocess@sha256:<digest>",
6 )

```

Listing G.7: Docker task with digest-pinned image.

For private registry authentication, use `docker login` or set `DOCKER_CONFIG` to a credentials directory before running the workflow. Never place registry passwords in `dagon.ini`.

G.5 Kubernetes

```

1 pip install "dagonstar[kubernetes]"
2 kubectl config current-context      # verify kubeconfig is valid

```

```

1 [kubernetes]
2 kubeconfig = ~/.kube/config
3 namespace  = research
4 image_pull_policy = IfNotPresent
5 ; Container registry credentials: use imagePullSecrets in the
6 ; namespace, not literal credentials in this file.

```

Listing G.8: Kubernetes section in dagon.ini.

```

1 k8s = DagonTask(
2     TaskType.KUBERNETES,
3     "regrid",
4     "python regrid.py --input /mnt/data/wrfout_d01",
5     image = "registry.example.org/gdal:3.7@sha256:<digest>",
6     namespace = "research",
7     resources = {"cpu": "4", "memory": "8Gi"},
8     node_selector = {"accelerator": "none"},
9 )

```

Listing G.9: Kubernetes task parameters.

For private registries, create a Kubernetes `imagePullSecret` in the target namespace and reference it via `image_pull_secret=` in the task constructor. Do not embed registry credentials in `dagon.ini`.

G.6 Apptainer / Singularity

```

1 pip install "dagonstar[apptainer]"
2 apptainer --version      # verify installation

```

```

1 [apptainer]
2 image_dir = /scratch/containers
3 bind_paths = /data,/scratch

```

Listing G.10: Apptainer section in dagon.ini.

```

1 apt = DagonTask(
2     TaskType.APPTAINER,
3     "gdal_regrid",
4     "python /scripts/regrid.py",
5     image = "oras://registry.example.org/gdal:3.7",
6     bind = ["/scratch/dagon:/mnt/scratch",
7             "/data/model:/mnt/model:ro"],
8 )

```

Listing G.11: Apptainer task with bind mounts.

G.7 HashiCorp Nomad

```

1 pip install "dagonstar[nomad]"
2 nomad status # verify Nomad is reachable

```

```

1 [nomad]
2 address      = http://nomad.cluster.example.org:4646
3 namespace    = default
4 token_env    = NOMAD_TOKEN
5 ; token_env names an environment variable; never a literal token.

```

Listing G.12: Nomad section in dagon.ini.

```

1 nmd = DagonTask(
2     TaskType.NOMAD,
3     "batch_qc",
4     "python qc.py --input workflow:///ingest/raw.nc",
5     driver      = "exec",
6     nomad_address = "http://nomad.cluster.example.org:4646",
7     token_env    = "NOMAD_TOKEN",
8 )

```

Listing G.13: Nomad task using exec driver.

G.8 Cloud Tasks (Apache Libcloud)

```

1 [cloud]
2 provider      = OPENSTACK
3 ; Credentials always from environment variables, never literal:
4 user_env      = OS_USERNAME
5 password_env  = OS_PASSWORD
6 auth_url      = https://keystone.example.org:5000/v3/

```

Listing G.14: Cloud section in dagon.ini.

```

1 pip install "dagonstar[cloud]"
2 # OpenStack:
3 export OS_USERNAME=jsmith
4 export OS_PASSWORD=<from vault>
5 # AWS: configure via aws configure or IAM instance profile
6 # GCP: gcloud auth application-default login

```

```

1 cloud_sim = DagonTask(
2     TaskType.CLOUD,
3     "cloud_model",

```

```

4     "bash run_model.sh workflow:///prepare/ic.nc",
5     provider = "OPENSTACK",
6     flavor   = "m1.xlarge",
7     image    = "ubuntu-22.04-wrf",
8 )

```

Listing G.15: Cloud task with flavour and image specification.

G.9 FaaS Provider Configuration

G.9.1 Mock (CI, Colab, local development)

No configuration required. Use `provider="mock"` in the task constructor.

G.9.2 HTTP / Knative

```

1 [faas.http.staging]
2 endpoint    = https://faas.example.org/function/classify
3 ; Use header_env to supply authorization tokens at runtime.
4 header_env  = FAAS_AUTH_HEADER

```

```

1 fn = DagonTask(
2     TaskType.FAAS, "classify", provider="knative",
3     function="classifier",
4     provider_options={"knative": {
5         "endpoint": "https://classify.example.svc.cluster.local",
6         "cloudevents": True,
7     }},
8     inputs={"doc": "workflow:///prepare/outputs/input.json"},
9 )

```

Listing G.16: Knative with CloudEvents headers.

G.9.3 AWS Lambda

```

1 pip install "dagonstar[faas-aws]"
2 aws configure # or use an IAM instance profile

```

```

1 [faas.aws.prod]
2 region    = eu-south-1
3 ; Credentials from IAM instance profile or default AWS chain.
4 ; Never place aws_access_key_id here.

```

```

1 fn = DagonTask(
2     TaskType.FAAS, "classify", provider="aws",
3     function = "arn:aws:lambda:eu-south-1:123456:function:classify",
4     inputs   = {"doc": "workflow:///prepare/outputs/input.json"},
5     outputs  = {"label": "label.json"},
6 )

```

Listing G.17: AWS Lambda invocation.

G.9.4 Azure Functions

```
1 pip install "dagonstar[faas-azure]"
2 az login # or use managed identity
```

```
1 [faas.azure.prod]
2 endpoint = https://myapp.azurewebsites.net/api/classify
3 token_env = AZURE_FUNCTION_TOKEN
```

G.9.5 GCP Cloud Run

```
1 pip install "dagonstar[faas-gcp]"
2 gcloud auth application-default login
```

```
1 [faas.gcp.prod]
2 endpoint = https://classify-xxxxx-ew.a.run.app
3 audience = https://classify-xxxxx-ew.a.run.app
```

G.10 IoT Provider Configuration

G.10.1 Mock

No configuration required. Use `provider="mock"` in the task constructor. The mock is deterministic and credential-free.

G.10.2 HTTP-Based Providers

```
1 [iot.http.production]
2 endpoint = https://iot.example.org/api/v1
3 credential_env = IOT_API_KEY
4 ; credential_env names the environment variable; never its value.
```

```
1 export IOT_API_KEY=<from vault>
```

Provider deployments, device registration, and IAM remain external prerequisites. Object-store upload/download adapters are not yet implemented; use direct JSON or an already accessible, policy-approved object URI.

G.11 Storage and Staging Configuration

G.11.1 Local Staging (COPY and LINK)

```
1 [batch]
2 scratch_dir_base = /data/dagon-scratch/
3 run_base =
4 remove_dir = False
5 threads = 1
```

```
1 from dagon import DataMover, Workflow
2 workflow = Workflow("Experiment", config=config)
3 workflow.set_data_mover(DataMover.COPY) # or DataMover.LINK
```

Listing G.18: Setting the workflow data mover.

G.11.2 Globus

```
1 pip install "dagonstar[globus]"
2 globus login # authenticate once
```

```
1 [globus]
2 client_id      = <Globus native-app client UUID>
3 ; Transfer token obtained at runtime; never stored here.
4 source_endpoint = <UUID of source endpoint>
5 destination_endpoint = <UUID of destination endpoint>
```

G.11.3 SKYCDS

```
1 [skycds]
2 url      = https://skycds.example.org
3 user_env = SKYCDS_USER
4 pass_env = SKYCDS_PASS
```

G.12 LLM Provider Configuration

```
1 [llm.openai]
2 api_url      = https://api.openai.com/v1
3 api_key_env   = OPENAI_API_KEY
4 model        = gpt-4o-mini
5 max_tokens    = 2048
6
7 [llm.local_ollama]
8 api_url      = http://localhost:11434/v1
9 api_key_env   = OLLAMA_KEY ; set to any non-empty value
10 model        = llama3.1
```

```
1 export OPENAI_API_KEY=<from vault>
```

```
1 summarize = DagonTask(
2     TaskType.LLM,
3     "summarize_forecast",
4     provider      = "openai",
5     input_files   = {"report": "workflow:///postprocess/report.txt"},
6     outputs       = {"summary": "summary.txt"},
7     system        = "You are a meteorology summarization assistant.",
8 )
```

Listing G.19: LLM task referencing a named provider profile.

G.13 Web Task Provider Configuration

```
1 [web]
2 ; Global defaults. Per-request overrides via task specification dict.
3 timeout      = 30
4 max_retries   = 3
5 ; Never place authentication tokens or passwords here.
```

Authentication in Web tasks is always through environment-variable indirection in the request specification:

```

1 upload = DagonTask(
2     TaskType.WEB,
3     "upload",
4     {
5         "method":      "POST",
6         "url":         "https://portal.example.org/api/upload",
7         "token_env":   "PORTAL_TOKEN",
8         "multipart":  {"product": {"file":
9                         "workflow:///postprocess/maps/map.png"}}},
10    },
11 )

```

Listing G.20: Authenticated Web task.

G.14 Complete dagon.ini Sample

The following sample covers all sections currently recognized by DAGonStar. Never commit a version with real credential values.

```

1  [dagon_service]
2  route = http://localhost:57000
3  use   = False
4
5  [ftp_pub]
6  ip      = localhost
7  user    = anonymous
8  password =
9
10 [batch]
11 scratch_dir_base = /tmp/
12 run_base         =
13 remove_dir       = False
14 threads          = 1
15
16 [slurm]
17 partition =
18
19 [docker]
20 ; No credentials here. Use docker login.
21
22 [kubernetes]
23 kubeconfig      = ~/.kube/config
24 namespace       = default
25
26 [apptainer]
27 image_dir       = /tmp/apptainer-cache
28 bind_paths      =
29
30 [nomad]
31 address         = http://localhost:4646
32 namespace       = default
33 token_env       = NOMAD_TOKEN
34
35 [cloud]
36 provider        = OPENSTACK
37 user_env        = OS_USERNAME
38 password_env    = OS_PASSWORD
39 auth_url        =
40
41 [faas.aws.default]

```

```
42 region =
43
44 [faas.azure.default]
45 endpoint =
46 token_env = AZURE_FUNCTION_TOKEN
47
48 [faas.gcp.default]
49 endpoint =
50 audience =
51
52 [faas.http.default]
53 endpoint =
54 header_env =
55
56 [iot.http.default]
57 endpoint =
58 credential_env = IOT_API_KEY
59
60 [llm.default]
61 api_url = https://api.openai.com/v1
62 api_key_env = OPENAI_API_KEY
63 model = gpt-4o-mini
64 max_tokens = 1024
65
66 [globus]
67 client_id =
68 source_endpoint =
69 destination_endpoint =
70
71 [skycds]
72 url =
73 user_env = SKYCDS_USER
74 pass_env = SKYCDS_PASS
```

Listing G.21: Complete dagon.ini.sample (all sections).

Bibliography and Source Provenance

Primary source

All technical content in this book is derived from the official DAGonStar documentation tree. This edition reflects the master branch:

DagOnStar/dagonstar, branch `master`, directory `docs/` (core documents, `docs/tutorial/` fourteen lessons plus the unified tutorial notebook `DAGonStar_tutorial.ipynb`, and `docs/examples/` catalog).
<https://github.com/DagOnStar/dagonstar>

Key additions in the master branch relative to the `development-native-web` branch documented in prior editions:

- `docs/colab.md` — Google Colab integration guide with capability matrix and persistence guidance (??).
- `docs/jupyter_notebook.md` — local Jupyter Notebook setup and usage guide (??).
- `docs/tutorial/DAGonStar_tutorial.ipynb` — 32-cell unified tutorial notebook covering all fourteen lessons, designed for Colab and Jupyter (??).
- `examples/colab/hello_colab.py/.ipynb` and `workflow_files_colab.py/.ipynb` — two credential-free Colab example workflows with paired notebooks (??).
- PyPI package release: `pip install dagonstar` (????).
- Two minor source improvements: logging initialization in `dagon.communication.ssh.SSHManager`; removal of debug-only `logger.debug` calls from the task garbage-collection path in `dagon.task.Task`.

Workshop abstract

The motivating abstract for this material is:

R. Montella, “*A workshop about DAGonStar in Practice: Orchestrating AI-HPC Workflows for Science and Engineering*”, High-Performance Scientific Computing Laboratory, Department of Science and Technology, University of Naples “Parthenope.”

Engine publications

The following publications, listed in the project’s own `README.md`, document the workflow engine and its predecessors:

- Sánchez-Gallegos, D. D., Di Luccio, D., Kosta, S., Gonzalez-Compean, J. L., Montella, R. “An efficient pattern-based approach for workflow supporting large-scale science: The DagOnStar experience.” *Future Generation Computer Systems*, 122 (2021): 187–203.
- Barron-Lugo, J. A., Gonzalez-Compean, J. L., Carretero, J., Lopez-Arevalo, I., Montella, R. “A novel transversal processing model to build environmental big data services in the cloud.” *Environmental Modelling & Software*, 144 (2021): 105173.
- Sánchez-Gallegos, D. D., Di Luccio, D., Gonzalez-Compean, J. L., Montella, R. “Internet of things orchestration using dagon workflow engine.” In *2019 IEEE 5th World Forum on Internet of Things (WF-IoT)*, pp. 95–100.
- Sánchez-Gallegos, D. D., Di Luccio, D., Gonzalez-Compean, J. L., Montella, R. “A microservice-based building block approach for scientific workflow engines: Processing large data volumes with dagonstar.” In *2019 15th International Conference on Signal-Image Technology & Internet-Based Systems (SITIS)*, pp. 368–375.
- Montella, R., Di Luccio, D., Kosta, S. “Dagon: Executing direct acyclic graphs as parallel jobs on anything.” In *2018 IEEE/ACM Workflows in Support of Large-Scale Science (WORKS)*, pp. 64–73.

Application publications

- Mellone, G., De Vita, C. G., Zambianchi, E., Expósito Singh, D., Di Luccio, D., Montella, R. “Democratizing the computational environmental marine data science: using the high-performance cloud-native computing for inert transport and diffusion lagrangian modelling.” In *2022 IEEE International Workshop on Metrology for the Sea*, pp. 267–272.
- De Vita, C. G., Mellone, G., Florio, A., Torres Charles, C. A., Di Luccio, D., Lapegna, M., Benassai, G., Budillon, G., Montella, R. “Parallel and hierarchically-distributed Shoreline Alert Model (SAM).” In *2023 31st Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*, pp. 109–113.
- Montella, R., Di Luccio, D., De Vita, C. G., Mellone, G., Lapegna, M., Ortega, G., Marcellino, L., Zambianchi, E., Giunta, G. “A highly scalable high-performance Lagrangian transport and diffusion model for marine pollutants assessment.” In *2023 31st Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*, pp. 17–26.

Survey

- Aldinucci, M., Agosta, G., Andreini, A., Ardagna, C. A., Bartolini, A., Cilardo, A., Cosenza, B., et al. “The Italian research on HPC key technologies across EuroHPC.” In *Proceedings of the 18th ACM International Conference on Computing Frontiers*, pp. 178–184, 2021.

Funding acknowledgments (as stated by the project)

- Research agreement “*Modelling mytilus farming at scale*” (MytilX, CUP D13C24000470002, funded by the Istituto Zooprofilattico Sperimentale dell’Umbria e delle Marche).
- Research contract “*Mytilus farming System with High-Performance Computing and Artificial Intelligence*” (MytilEx, CUP I63C23000180002, funded by the Campania Region, Veterinary sector).

- EuroHPC H2020 project “*Adaptive Multi-tier Intelligent data manager for Exascale*” (ADMIRE, 956748-ADMIRE-H2020-JTI-EuroHPC-2019-1, funded by the European Commission), Work Package 7.